TRNG Based on Multiple Entropy Sources Using CTDSM

A dissertation submitted in partial fulfillment of
the requirements for the degree of

Master of Science by Research

by

Apurv Pandey
(Roll No. EE21MS001)

Under the guidance of
Prof. Naveen Kadayinti

॥ सा विद्या या विमुक्तये ॥
ಭಾ.ತಂ.ಸಂ. ಧಾರವಾಡ
भा. प्रौ. सं. धारवाड
I.I.T. DHARWAD

DEPARTMENT OF ELECTRICAL, ELECTRONICS AND
COMMUNICATION ENGINEERING
INDIAN INSTITUTE OF TECHNOLOGY DHARWAD

November, 2025

Dissertation Approval

The dissertation entitled

[illegible]

by

[illegible]

[illegible]

[illegible]

[illegible]

....

Internal Examiner

[illegible]

....

External Examiner

[illegible]

[illegible]

....

Supervisor

[illegible]

....

Chairman

[illegible]

Date: ....

Place: ....

Declaration

I declare that this written submission represents my ideas in my own words and where others' ideas or words have been included, I have adequately cited and referenced the original sources. I also declare that I have adhered to all principles of academic honesty and integrity and have not misrepresented or fabricated or falsified any idea/data/fact/source in my submission. I understand that any violation of the above will be cause for disciplinary action by the Institute and can also evoke penal action from the sources which have thus not been properly cited or from whom proper permission has not been taken when needed.

Apurv Pandey
Roll No. EE21MS001

Date: 12 November, 2025

Abstract

A True Random Number Generator (TRNG) serves as a fundamental primitive in ensuring the security, reliability, and integrity of various systems and applications that require high-quality randomness. It is crucial in hardware security applications and tasks such as cryptographic key generation and secure communication protocols. TRNGs extract entropy from random physical processes to generate statistically independent, non-reproducible bitstreams possessing cryptographic-grade randomness. Silicon-based TRNGs utilize random noise in the circuit as a source of entropy to emulate an equiprobable Bernoulli random process.

This thesis proposes the use of a Continuous-Time Delta-Sigma Modulator (CTDSM) to extract entropy from multiple uncorrelated sources for TRNG design. It qualifies as an ideal candidate for combining entropy as its noise floor depends on multiple uncorrelated noise such as thermal noise, quantization noise, clock jitter, and metastability-induced noise. A CTDSM can be made entropy-rich by leveraging its negative feedback mechanism and non-idealities due to clock jitter and the quantizer's metastability. By introducing thermal noise as the reference input of the delta-sigma loop, simultaneously employing jittery clocks for both the ADC and DAC and utilizing a slowly resolving high-time constant comparator, one can intermix multiple uncorrelated noise sources into CTDSM.

This thesis delves into the conceptual development of CTDSM and investigates the various non-idealities that affect its noise spectrum. It conducts qualitative and quantitative analyses of various noise sources to evaluate their impact on the CTDSM's noise floor across the entire frequency spectrum. Drawing insights from these analyses, it provides theoretical justification for the design choices aimed at ensuring a similar contribution of noise from each source to the overall noise signal variance within the CTDSM.

This thesis proposes a TRNG based on a 1st-order, 4-bit CTDSM implemented using the TSMC 65nm CMOS process at a supply voltage of 1.2 volts. The circuit design for the various blocks involved in the CTDSM-based TRNG has been done to maximize the stochasticity of

the random process incorporated into the modulator. The thermal noise at the input is amplified, buffered, and applied to the input of the CTDSM by utilizing a high-gain OTA. The quantizer within the CTDSM is designed to reduce the dynamic range to match the mean and standard deviation of the random process within the loop. The design choices of the DAC simultaneously ensure the aliasing of high-frequency noise and good jitter sensitivity. The quantizer is clocked using a Chaotic Ring oscillators, which generates a 100.6 MHz clock with an RMS jitter-to-period ratio of approximately 20%. The proposed design has been evaluated through extensive simulations, successfully demonstrating the generation of cryptographic-grade random sequences at a throughput of 100.6 MHz.

Contents

List of Tables

List of Figures

Abbreviations

TRNG True Random Number Generator.

CRO Chaotic Ring oscillators.

SBD Soft-Breakdown.

MOS Metal-Oxide-Semiconductor.

RTN Random Telegraph Noise.

SAR Successive Approximation Register.

ADC Analog-to-Digital Converter.

DS Delta-Sigma.

CTDSM Continuous-Time Delta-Sigma Modulator.

STF Signal Transfer Function.

NTF Noise Transfer Function.

ACF Auto-Correlation Function.

PSD Power Spectral Density.

RO Ring oscillator.

DFF D Flip-Flop.

VCO Voltage Controlled Oscillator.

DTDSM Discrete-Time Delta-Sigma Modulator.

NRZ Non-Return to Zero.

DAC Digital-to-Analog Converter.

LSB Least-Significant Bit.

OTA Operational Transconductance Amplifier.

PRNG Pseudo-Random Number Generator.

NIST National Institute of Standards and Technology.

LFSR Linear Feedback Shift Register.

RZ Return to Zero.

TD Trapping/De-trapping.

FRO Fast Ring oscillator.

SRO Slow Ring oscillator.

TG Transmission Gate.

DFT Discrete Fourier Transform.

DSP Digital Signal Processors.

AAF Anti-Aliasing Filter.

OSR Oversampling Ratio.

SCR Switched-Capacitor Resistor.

EDRZ Exponentially Decaying Return to Zero.

IID Independent and Identically Distributed.

VCVS Voltage Controlled Voltage Source.

NOCG Non-Overlapping Clock Generator.

Chapter 1

Introduction

Random numbers can be generated using a Pseudo-Random Number Generator (PRNG), which is an algorithm designed to produce a sequence of numbers with properties that mimic those of random number sequences. It utilizes an input, known as a seed, to produce a sequence of numbers that appear random but can be reproduced or extrapolated if the adversary has knowledge of the algorithm and seed used. The deterministic aspect of this process is the reason for the term "pseudorandom". While PRNGs are often sufficient for many applications where cryptographic strength is not required, they are not suitable for security-critical applications like encryption or key generation. It is highly desirable to choose True Random Number Generator (TRNG)s over PRNGs for random key generation to meet the stringent criteria of cryptography. The tighter level of security in TRNGs is obtained at the cost of low throughput, high energy, and area requirements.

TRNGs find widespread applications in various fields, including cryptography, hardware security, secure communication protocols, scientific research, and simulation. TRNGs are essential for generating unpredictable and high-quality random numbers, which are fundamental for cryptographic key generation, secure authentication, secure communication channels, secure boot processes, and data encryption. In hardware security, TRNGs are utilized for device authentication, firmware integrity verification, and generating unique device identifiers. In scientific research and simulation, TRNGs are employed for the imitation of unpredictable real-world noise inputs for simulations, stochastic experiments, and statistical analyses through various randomized algorithms. Overall, TRNGs play a critical role in ensuring the security, reliability, and integrity of various systems and applications that require randomness.

A TRNG is a primitive in hardware security. The quality of the TRNG has a significant bearing on the security of such systems. TRNG provides a stream of ones and zeros that can be split into sub-streams or blocks of random numbers for use in cryptography applications. A random sequence of bits can be seen as the outcome of the tossing of a fair coin with 0 and 1 labels on its faces. Each coin flip has an equal probability of $\frac{1}{2}$ of producing a 0 or 1. Further, these flips are independent, meaning the outcome of one flip does not influence the subsequent ones. This unbiased coin is an ideal random bit stream generator, ensuring a random and uniform distribution of 0 and 1 values. Silicon-based TRNGs imitate a Bernoulli random process by generating two equiprobable symbols, resulting in a truly random bitstream with nearly unity entropy.

Entropy quantifies the degree of randomness present in a physical process, and it represents the average amount of information or uncertainty in a random variable. In the context of TRNGs, the entropy source represents the unpredictable physical phenomenon that gives the random number generators their inherent randomness. Mathematically, entropy quantifies the information carried by each bit in a sequence, typically ranging from 0 to 1. A value of 0 means complete predictability, while 1 indicates maximum uncertainty, where each bit holds one full bit of information. For example, in a fair coin flip, where there are two equally likely outcomes, the system's entropy would be 1 bit. This is because one bit of information is required to represent the outcome of the coin flip, as there are two equiprobable possibilities. Ensuring a good quality entropy source is very crucial for TRNG design. This involves the identification of an unbiased and uncorrelated random physical process as a source of randomness for TRNG.

Designing a TRNG circuit that consistently provides high-quality entropy across process variation, temperature, voltage, and frequency variations is a formidable challenge. Various random number generator designs have been proposed, utilizing mixed or pure digital circuits. These designs primarily differ based on their entropy sources, harvesting methods, and post-processing techniques. Silicon solutions for TRNG design extract entropy from random processes within a circuit, such as Random Telegraph Noise (RTN), Thermal noise, Non-deterministic jitter, Metastability in latches, and Chaotic circuits. Harvesting techniques depend on the entropy source and the throughput requirements. In cases of poor entropy extraction, complex post-processing becomes necessary, incurring area and energy penalties.

Autocorrelation and Shannon entropy are commonly used metrics for assessing the quality of randomness in data or random number sequences, but they have limitations. Autocorrelation primarily detects linear dependencies but overlooks higher-order relationships or non-linear patterns. Meanwhile, Shannon entropy measures overall uncertainty but misses localized patterns and assumes independence between symbols. Due to the critical nature of applications that use TRNGs and the variety of physical processes employed, it is crucial to develop a rigorous approach to describe the quality of randomness produced by a TRNG. A series of statistical tests suggested in Special Publication 800-22 by the National Institute of Standards and Technology (NIST) [1] are frequently used to evaluate the randomness of the generated bit sequence. The NIST SP 800-22 test suite examines a truly random source's output for characteristics desirable for use in cryptography [2] [3].

1.1 Bias and Correlation

Silicon-based TRNGs are expected to produce a completely unbiased and uncorrelated bit stream. That means that at every trigger, it should produce either 0 or 1 with equal probability and be completely uncorrelated to the previous outcomes. This imitates the Bernoulli random process with $p = 0.5$. Any bias due to non-idealities in implementing TRNG causes deviation from the zero mean random process. Any deterioration in the overall entropy of the TRNG output makes it unsuitable for cryptography applications. TRNGs often need some kind of de-biasing/post-processing to suppress the bias to maximize the entropy. Figure 1.1 displays biased and unbiased outcomes of a random number generator, with two Bernoulli random variables depicted. On the left, the distribution is unbalanced, while on the right, after de-biasing, it becomes equiprobable.

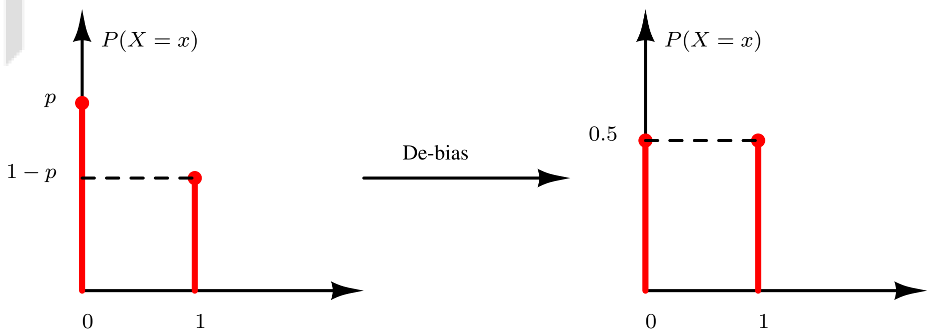

Figure 1.1: Biased outcomes before post-processing (left) versus the unbiased outcomes after post-processing (right) of a TRNG.

TRNG outcomes are supposed to be uncorrelated, which indicates that each generated number is statistically independent of its predecessors, essential for randomness. This can be detected by plotting the Auto-Correlation Function (ACF) of the produced random bitstream. ACF of a TRNG output would exhibit unity autocorrelation at zero lag and zero correlation at all other lags, indicating perfect independence between consecutive samples. Correspondingly, from the Wiener–Khintchine relationship, this would result in a flat Power Spectral Density (PSD), indicating that the signal's power is distributed uniformly across all frequencies and there are no observable patterns.

Another non-ideal aspect of generating a truly random bit stream is the presence of correlations among the samples induced by memory within the system. These correlations can arise due to various factors, such as internal state variables or memory elements used in the generation process. Figure 1.2 displays the ACF and corresponding PSD of both correlated and uncorrelated outcomes of a random number generator. A unity autocorrelation factor at zero lag and zero elsewhere indicates a white random process, while non-zero autocorrelation factors at non-zero lags indicate correlation among samples, leading to a colored spectrum in the PSD [4].

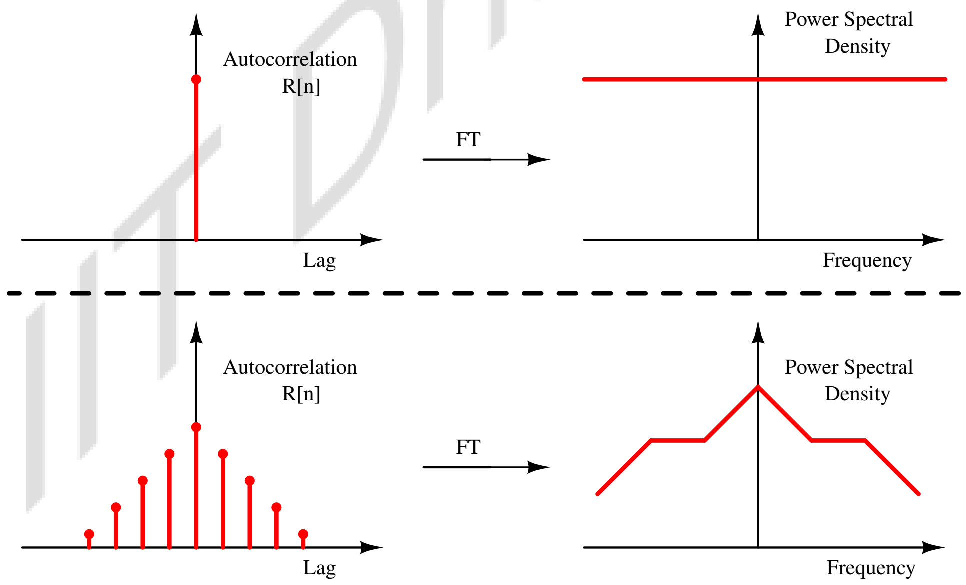

Figure 1.2: Autocorrelation Function and Power Spectral Density comparison between ideal True Random Number Generator output with unity ACF and flat PSD (top) versus TRNG output with non-zero ACF leading to colored PSD (bottom) [4].

1.2 Randomness Testing

The testing procedure for quality of randomness produced through various random number generators has been standardized by the NIST SP 800-22 statistical test suite. It comprises 15 randomness tests designed to assess the quality of random number generators. These tests cover a range of statistical and computational aspects to ensure the reliability of generated random sequences. The list of tests and their brief descriptions are as follows:

1. **Frequency (Monobit) Test**: Checks for a fair balance between ones and zeroes in the produced sequence.
2. **Block Frequency Test**: Divides the sequence into non-overlapping blocks to examine the frequency of ones in each block.
3. **Runs Test**: Identifies consecutive identical bits (runs) and compares their number to what's expected in a truly random sequence.
4. **Longest Run of Ones in a Block Test**: Focuses on finding the longest run of consecutive ones within a block.
5. **Binary Matrix Rank Test**: Evaluates sub-matrices rank from the entire sequence to detect any linear dependencies.
6. **Discrete Fourier Transform (Spectral) Test**: Analyzes the spectral content of the sequence to detect patterns.
7. **Non-Overlapping Template Matching Test**: Searches for predefined patterns within the sequence to detect random generators that produce excessive occurrences of a given aperiodic pattern.
8. **Overlapping Template Matching Test**: Searches for predefined patterns within the sequence, allowing for overlapping matches.
9. **Maurer's Universal Statistical Test**: Measures the compressibility of the sequence.
10. **Linear Complexity Test**: Evaluates the length of a Linear Feedback Shift Register (LFSR) to determine the randomness of a sequence, with longer LFSRs indicating greater complexity and randomness while shorter LFSRs implying non-randomness.

11. **Serial Test**: Examines the frequency of overlapping pairs of bit-patterns across the entire sequence.

12. **Approximate Entropy Test**: Measures the unpredictability of the sequence by assessing the likelihood of repeated patterns.

13. **Cumulative Sums (Cusum) Test**: Evaluate the sequence's maximal excursion (from zero) of the random walk defined by the cumulative sum of adjusted (-1, +1) digits in the sequence.

14. **Random Excursions Test**: Focuses on the number of cycles having specific frequencies in the cumulative sum of the sequence.

15. **Random Excursions Variant Test**: Examines the total number of times that a particular state is visited in a cumulative sum random walk.

Overall, these tests collectively ensure a thorough evaluation of the randomness of generated sequences, making the NIST SP 800-22 suite a valuable tool for assessing the quality of random number generators [1] [5].

1.3 Motivation

As discussed, Silicon-based TRNGs extract entropy from random processes within a circuit, such as white electrical noise. This type of noise possesses the ideal characteristics of an entropy source, samples of which are unbiased and independent outcomes. However, despite electrical noise typically following a Gaussian distribution, circuit non-idealities in entropy harvesting components can introduce deviations from a perfectly white and unbiased noise profile. Due to these non-idealities, relying solely on one entropy source can make it difficult to consistently extract white and unbiased randomness.

This thesis proposes utilizing a Continuous-Time Delta-Sigma Modulator (CTDSM) as an entropy extractor for TRNG design. The method exploits the inherent negative feedback mechanism and non-idealities of CTDSM to introduce noise from multiple uncorrelated sources into the loop. This is illustrated in Figure 1.3. By introducing thermal noise as the reference input of the delta-sigma loop, simultaneously employing jittery clocks for both the ADC and DAC, and utilizing a slowly resolving high-time constant comparator, we can intermix multiple uncorrelated noise sources into the loop. The noise from these sources, characterized by different distributions, converges to a Gaussian distribution, as per the Central Limit Theorem. This results in the input of the ADC within the loop to be a white random process $y(t)$, which includes variance contributions from all uncorrelated random processes present in the loop. Hence, the random bitstream sampled from such a CTDSM can potentially yield very high entropy, making it well-suited for TRNG design.

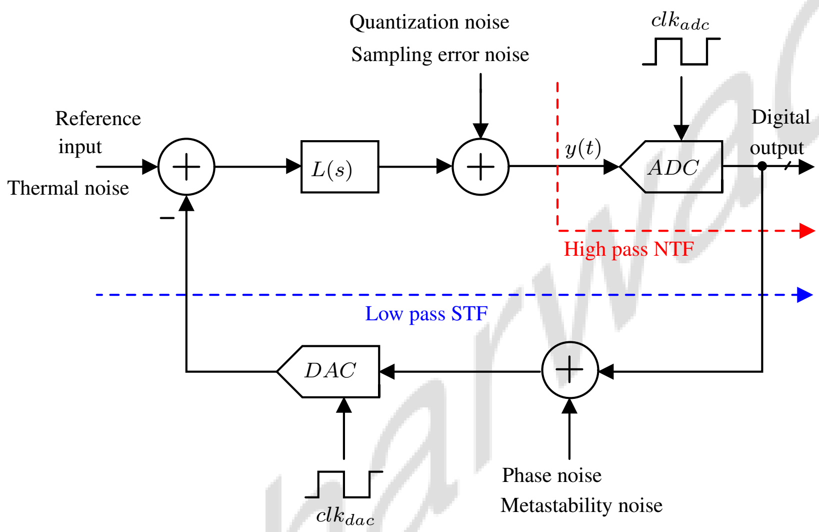

Figure 1.3: Multiple uncorrelated noise sources reaching to the output of the CTDSM through different Transfer Functions.

1.4 Contribution of the Thesis

This study extensively explores various aspects within a CTDSM, which can be used to combine entropy from multiple uncorrelated noise sources for TRNG design. The main contributions of this thesis are:

- Qualitative and quantitative analysis of thermal noise, quantization noise, clock jitter, metastability, and their impact on the noise floor of CTDSM.
- Theoretical justification for the design choices aimed at ensuring similar noise contribution from each noise source onto the overall noise signal variance within CTDSM.
- The design of CTDSM-based TRNG circuits using TSMC 65nm technology. The contributions in the design of the various blocks involved in the CTDSM-based TRNG are

– Circuit design techniques to boost thermal noise variance to inject it at the input of the CTDSM.

– An effective method of dithering white noise at the input of the ADC within the CTDSM loop.

– The design of a multi-bit quantizer to reduce the dynamic range to match the mean and standard deviation of the random process within the loop.

– Circuit design to generate a weighted combination of Return to Zero and exponentially decaying pulses to leverage poor alias rejection and high jitter sensitivity simultaneously.

– The design of Chaotic Ring oscillators (CRO) to generate jittery clocks for ADC and DAC.

1.5 Dissertation outline

The remainder of the thesis is organized as follows:

- Chapter 2 provides a comprehensive review of the literature on reported TRNG circuits and post-processing techniques, offering valuable insights into fundamental methods for extracting entropy from various noise sources.
- Chapter 3 discusses the fundamental aspects of delta-sigma modulation, including the conceptual development of CTDSM, its transfer functions, and the modeling and characterization of multiple noise sources that impact the noise floor of the CTDSM.
- Chapter 4 presents the design details of CTDSM-based TRNG along with circuit simulation and NIST test results.
- Finally, Chapter 5 discusses the conclusion and future direction of this work.

Chapter 2

Literature survey

This chapter presents a comprehensive literature survey on reported TRNG circuits, focusing primarily on mixed or pure digital designs. These circuits differ primarily in their approaches to extracting entropy from various random processes and post-processing techniques. A wide variety of random processes can be utilized in circuits as sources of entropy, such as current fluctuation in MOS due to Soft-Breakdown (SBD), Random Telegraph Noise, thermal noise, clock jitter, the uncertainty of resolving latches during metastable states, and circuits possessing chaos dynamics. Each reported work showcases unique techniques for extracting entropy from these physical processes, along with their shortcoming, followed by improvements if they happened in another reported work. When poor entropy extraction arises due to non-idealities caused by Process, Voltage, and Temperature (PVT) variations, sophisticated post-processing techniques are employed to improve the quality and reliability of the generated random data. We will briefly discuss several standard post-processing techniques used to enhance the reliability and quality of generated random data.

This review aims to offer insights into the methodologies employed for TRNG circuit design and the advancements made in addressing challenges associated with generating high-quality random data. Following this introduction, subsequent sections will discuss the details of various TRNGs reported in the literature.

2.1 TRNG Based on Soft-Breakdown MOS

Signal fluctuations in CMOS devices due to SBD have an amplitude substantially higher than that of thermal noise. A high electric field stressed under a constant voltage of 7 volts was shown to cause a Soft-Breakdown in the MOS capacitor [6]. It was observed that the fluctuation in MOS current after the occurrence of soft breakdown was substantially higher, nearly six orders of magnitude, compared to the fluctuation due to thermal noise before the soft breakdown of the same MOS. The interaction between defects created by high-energy electrons in the insulating layer and electrons conducting through the film was presumed to cause the current fluctuation [7, 8]. The power spectrum of current fluctuation in SBD-MOS was observed to exhibit f^{-1}-like features, suggesting a potential for causing long-term correlation in random number sequences if directly converted. The TRNG proposed in [6] was designed to digitize the random process caused due to SBD-MOS and eliminate the influence of frequency-dependent PSD. The reported TRNG based on SBD-MOS is shown in Figure 2.1, which consists of an astable multi-vibrator whose oscillation period becomes a random process.

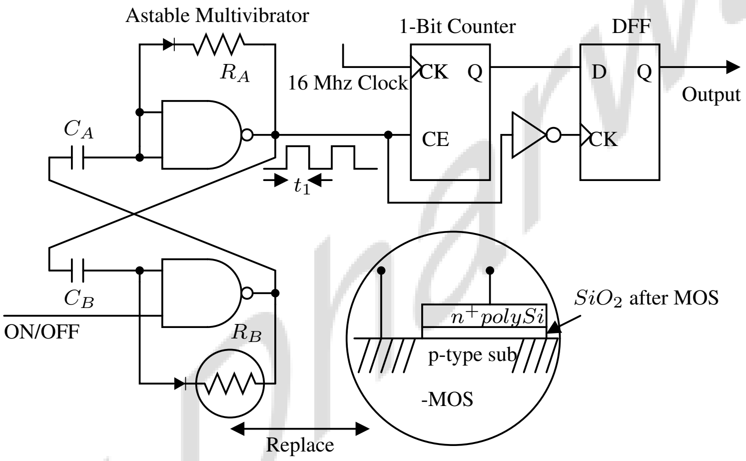

Figure 2.1: Noise from SBD-MOS randomly modulating pulse width (t_1) which gets digitized and converted to 1-bit random bitstream [6].

The random fluctuation in time constant determined by $R_A R_B$ and $C_A C_B$ of astable multi-vibrator translates to jitter the output. The resistance fluctuation of SBD-MOS has been used as an entropy source that randomly varies the time period of the rectangular output wave (t_1). The TRNG uses a counter to measure the period of a rectangular wave generated by an astable multi-vibrator. Each period of the multi-vibrator output fluctuates due to the resistance variation of the SBD-MOS, resulting in a 1-bit random bitstream at every high state of the multi-vibrator. This circuit was designed with discrete components using 20 CMOS gates and a few passive devices. The supply voltage for the astable multi-vibrator was 5V, and a 16 MHz clock frequency was used to generate random numbers at the throughput of 50 kb/s. The generated data showed consistency in passing all recommended NIST tests [6].

2.2 TRNG Based on RTN in Scaled Transistors

The stochastic Trapping/De-trapping (TD) of numerous charge traps in the gate oxide causes low-frequency noise in MOSFETs. The TD of electrons due to imperfections near the Si–SiO_2 interface causes Random Telegraph Noise. The traps are either introduced during the manufacturing process or generated due to stress. It can collect and emit a channel carrier dependent on its parameters, modifying the threshold voltage. As a result, the magnitude of a device's drain voltage/current has discrete levels. The noise resulting from RTN exhibits a sufficiently high amplitude compared to the resolution of the comparator in the technology, making it easier to digitize. At advanced technology nodes, such as 28 nanometers and below, RTN amplitude may surpass 60% of the drain current, according to silicon data [9]. The properties of the traps in different devices differ due to process variations. Each device's RTN signal has a distinct signature as a result of this. The TRNG proposed in [9] used RTN as the source of entropy for TRNG design.

Previous generations of large-area devices feature numerous traps, and the superposition of their spectra eventually culminates in a f^{-1}-like characteristic. Due to aggressive technology scaling, the number of traps in a device has decreased substantially. This results in a discrete temporal shift in output current known as RTN and a frequency-independent region up to a specific cutoff frequency that relies on the trap and device parameters (Refer to Figure 2.2).

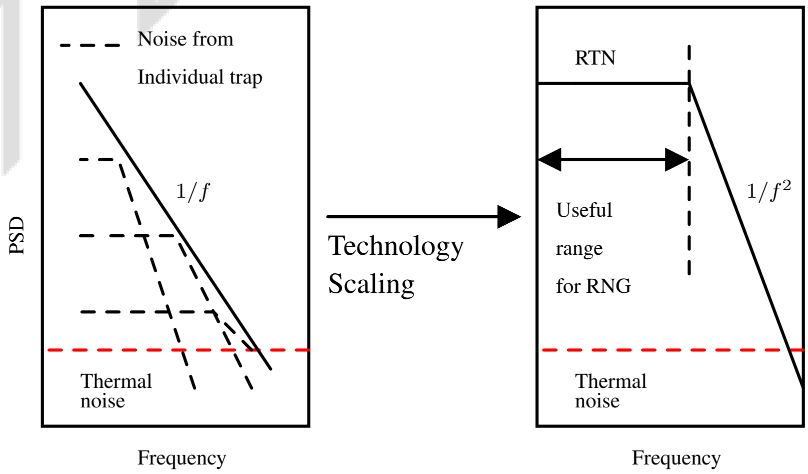

Figure 2.2: Noise due to multiple traps contributing to $1/f$ for large area devices (left), as compared to the Lorenztian profile of a single trap in scaled devices (right). White noise-like characteristics of a single-trap RTN can be used for TRNG [9].

Drain voltage or current fluctuations are manifestations of an RTN signal. Thresholding by using a noise generator, filter, and comparator circuits can be used to amplify and digitize the signal. In Figure 2.3, consider the amount of time spent in i^{th} cycle in the state of electron capture to be τ_c^i and the amount of time spent in the state of electron emission to be τ_e^i. Each rising edge of the RTN signal is used to toggle the output bit in this method. The resulting signal remains in the OFF state for one complete RTN cycle, with a duration of $(\tau_c^i + \tau_e^i)$, before switching to the ON state for the following RTN cycle, with a duration of $(\tau_c^{i+1} + \tau_e^{i+1})$. As a result, after the positive edge triggered toggling of the RTN signal, the resultant signal has OFF and ON durations with the duration given by:

$$\begin{aligned} \tau_{OFF} &= \tau_c^i + \tau_e^i \\ \tau_{ON} &= \tau_c^{i+1} + \tau_e^{i+1} \end{aligned} \tag{2.1}$$

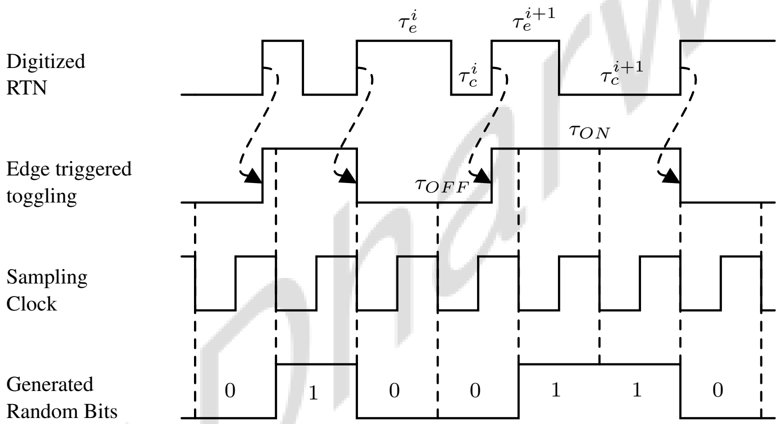

Figure 2.3: From RTN signal to generated random bits following the proposed algorithm. The TRNG output is toggled each time a rising edge is observed in the digitized RTN. This uses the fact that the average time between two rising edges caused by the stochastic event of electron emission $\tau_c + \tau_e$ is constant. $\tau_{c/e}$ is the time for i_{th} capture/emission [9].

The time constants (τ_c and τ_e) in RTN are frequently unequal, and the sequence obtained immediately by digitization of RTN is skewed towards either bit, depending on which of (τ_c and τ_e) has the bigger value. The suggested technique focuses on the physical features of RTN to reduce the bias in RTN bit streams. When averaged over a sufficiently long time, the values of capture and emission times (τ_c and τ_e) associated with a given trap remain constant, which is an inherent property of RTN. The average τ_{OFF} and τ_{ON} can be determined through equation 2.2.

$$\begin{aligned} \overline{\tau_{OFF}} &= \overline{\tau_c^i + \tau_e^i} = \overline{\tau_c^i} + \overline{\tau_e^i} = \tau_c + \tau_e \\ \overline{\tau_{ON}} &= \overline{\tau_c^{i+1} + \tau_e^{i+1}} = \overline{\tau_c^{i+1}} + \overline{\tau_e^{i+1}} = \tau_c + \tau_e \end{aligned} \tag{2.2}$$

Because the average values of τ_{ON} and τ_{OFF} are equal, bias caused by unequal capture and emission time constants in an RTN signal is eliminated. Without the need for additional post-processing methods, this straightforward and practical solution automatically ensures a 50% bit probability based on RTN's physical characteristics. The reported TRNG utilized a 65 nm discrete nMOS device biased with a 1.2 μA constant current to demonstrate RTN signal extraction and random bitstream generation. The RTN for the 65nm discrete NMOS exhibited a flat PSD up to the cutoff frequency of 3.33 Hz. Therefore, it was sampled below the cutoff frequency of the Lorentzian PSD, and the bitstream generated from the TRNG, with a throughput of 3.33 Hz, successfully passed NIST tests [9].

2.3 TRNG Based on Coupled Ring Oscillators

TRNGs based on Coupled Ring oscillator (RO) are designed to sample the randomness of phase jitter from multiple high-frequency, well-matched ROs. This technique aims to sample jittery pulses precisely at their zero crossings. To achieve this, the outputs of these ROs are fed into a multi-input XOR gate, and the XORed output is sampled by a reference clock with a fixed frequency using a DFF to generate the random bitstream. The TRNG reported in [10] is shown in Figure 2.4. It outlines the design principle for coupled ring oscillators to extract sufficient entropy for TRNG applications using purely digital footprints. The author formulated a "coupon collector's problem" to determine the relative length and number of ROs required in the suggested topology to sufficiently meet the entropy requirements. For a sample design, 114 equal-length ROs were combined using an XOR tree, the output of which was sampled by a D Flip-Flop (DFF). A sampling frequency of 40 MHz was suggested, and post-processing was done using an extended BCH code [256, 16, 113]. Here, the samples were grouped into blocks of 256 bits and fed into the post-processing function, which returns only 16 bits. The resulting throughput of the TRNG was reported to be 2.5 Mbps, passing all NIST tests [10].

However, in such designs, handling a large amount of switching activity from free-running ROs is challenging for both the XOR tree and the sample DFF. When multiple ROs operate simultaneously, the number of transitions within a sampling period becomes so high that the setup and hold times for the look-up table (LUT) and the internal register element in the FPGA are shorter than the specifications bound for the device. An enhancement to the TRNG based on the coupled ROs is suggested in [11], where an extra DFF is added after each ring to address the problem of multiple transitions in the sampling period (refer to Figure 2.5). Adding extra flip-flops does not alter the collection of randomness gathered from each ring. Due to this

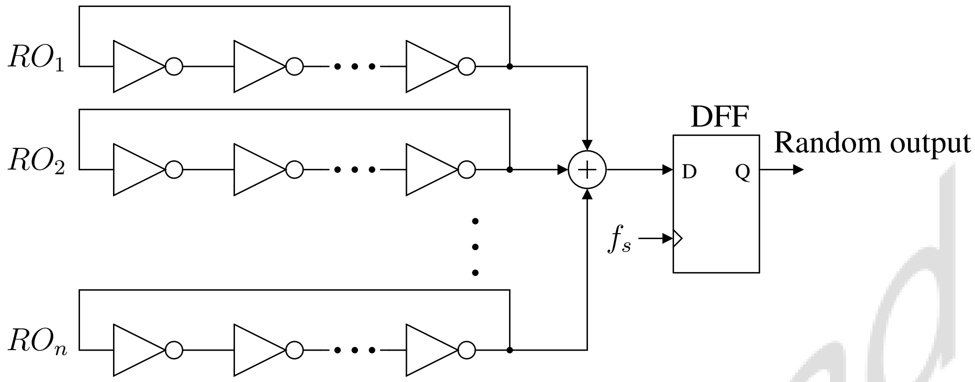

Figure 2.4: Sampling jitter by combining multiple equal length RO for random number generation [10]

enhancement, the signals on the XOR's input could be synchronized with the sampling clock and changed only once throughout the sample period. Consequently, the setup and hold times for the internal logic will now fall within acceptable limits due to the reduction in transitions on the input to the XOR tree. The improved TRNG topology was tested on Altera's Cyclone II FPGA with a supply voltage of 1.2 volts in 90nm technology. Random data was generated at 50MHz and passed all NIST tests [11].

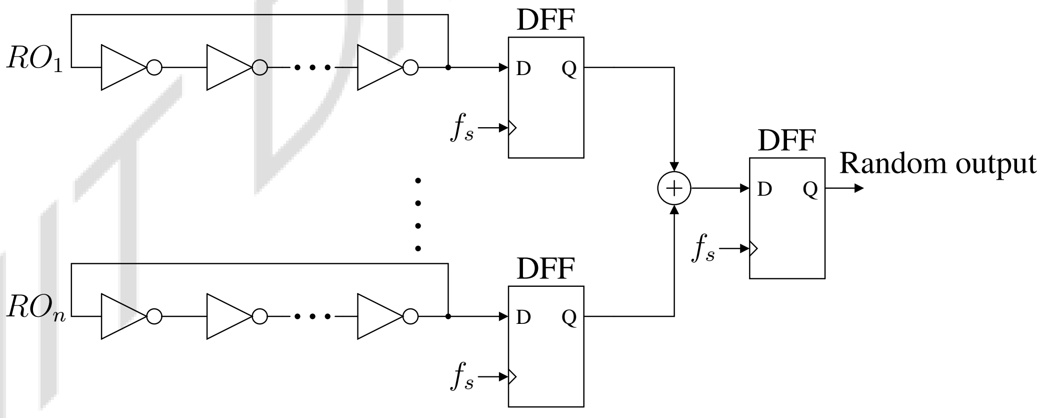

Figure 2.5: Improved coupled ring oscillator based TRNG [11].

The entropy of the resulting bit sequence from the TRNG would be substantially decreased if equal-length ring oscillators, designed in FPGAs, exhibit high correlation due to physical proximity. Thus, the XOR of the output from these rings may mostly return zeros. To address this concern, Programmable Delay Lines (PDLs) are utilized in [12] to implement ROs. This introduces increased variability in RO oscillations from cycle to cycle, leading to jitter in the generated RO clocks. Figure 2.6 illustrates the Programmable Delay Inverter using a 4-input (A_1 A_2 A_3 A_4) Look-Up Table (LUT) implemented in Xilinx Spartan-3A FPGA.

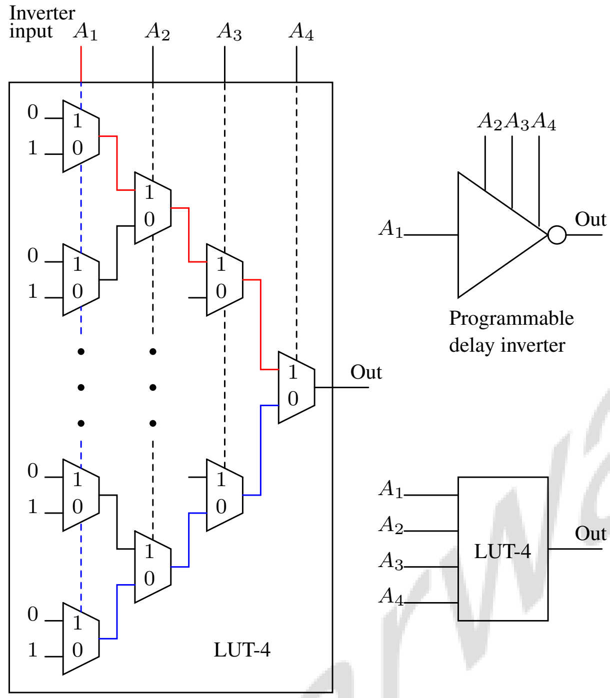

Figure 2.6: Programmable delay line using 4-input LUT used to implement RO for coupled ring oscillator based TRNG [12].

Here, A_1 represents the inverter input, while the other 3 bits control delay, determining the signal propagation path from A_1 to the output. For 4-input LUTs, the red line highlights the shortest signal propagation path from A_1 to the output with $A_2A_3A_4 = 000$, while the blue lines denote the longest path with $A_2A_3A_4 = 111$. The TRNG architecture employs a coupled ring oscillator to sample the jitter, as recommended in [11]. The complete design includes 32 ROs, an XOR tree, DFF, and a Von-Neumann post-processing unit. Von-Neumann post-processing involves pairing consecutive bits and discarding identical pairs to reduce bias and enhance randomness. The TRNG successfully passed all the recommended NIST tests with a throughput of 6 Mbps [12].

2.4 TRNG Based on Chaotic Ring Oscillators

When designed to sample in jittery, non-deterministic zones, coupled ring oscillators become bulky due to the necessity of a large number of ring oscillators to attain sufficient entropy. The large number of ring oscillators needed to generate a high-quality TRNG can be overcome by amplification of jitter using chaotic dynamics as reported in [13]. One conventional way to sample jitter for random number generation is by using a DFF to sample a relatively jitter-free clean clock from a Fast Ring oscillator (FRO) (f_1), using a jittery Slow Ring oscillator (SRO) (f_2) (refer Figure 2.7).

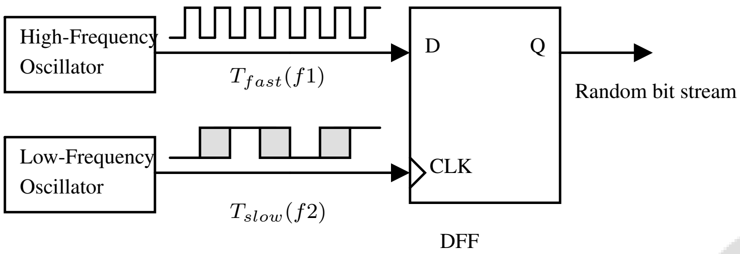

Figure 2.7: Sampling High-frequency clean clock with a slow, jittery clock which has peak to peak jitter of magnitude close to the period of the clean clock.

To ensure good randomness of the output stream, the jitter of the SRO (i.e., clock) must be comparable to the oscillation period of the FRO. Thus, f_1 is much higher than f_2, so the system's total power consumption tends to be high, mainly due to the high-speed f_1. Jitter amplification in f_2 results in a slowing down of f_1, as shown in Figure 2.8.

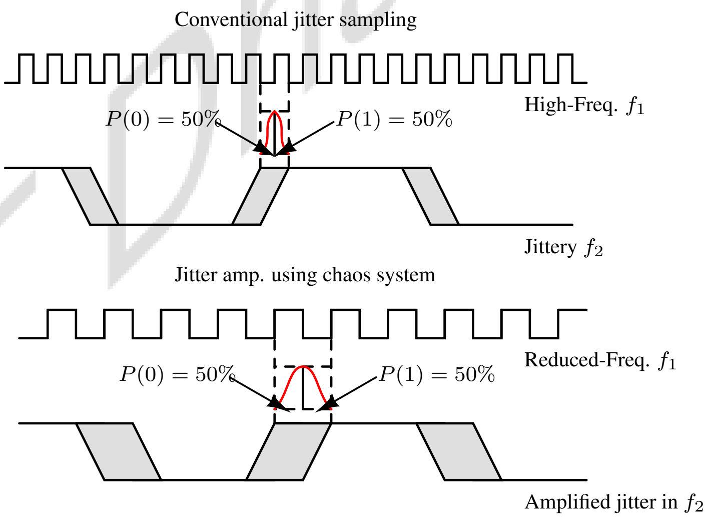

Figure 2.8: Jitter amplification in sampling clock f_2 relaxes high-frequency clock f_1 [13].

The design reported in [13] is shown in Figure 2.9. It introduces a chaotic system where two distinct oscillators are non-linearly coupled, leading to intrinsic jitter amplification. This amplification can be utilized to generate a random bit stream, as per the approach given in Figure 2.7. The whole setup has been referred to as the Chaotic Ring Oscillator Random Number Generator (CRNG). The proposed CRNG circuit is an electronic counterpart of a Double Pendulum.

The double pendulum exhibits chaotic behavior due to the non-linear coupling of its two pendulum arms and its sensitivity to initial conditions, resulting in unpredictable frequency variations in its motion. Similarly, in the electronic domain, merging two oscillators with different frequencies can cause chaos. It comprises two ROs: a SRO and a FRO and NMOS M_{NR}, which behaves like a non-linear resistor. The gate of M_{NR} is controlled by V_{FRO}, which modulates the amplitude of V_{SRO}. Furthermore, an always-on Transmission Gate (TG) connects V_{SRO} and V_{FRO} to introduce more non-linearity. As a result of this setup, the signals V_{SRO} and V_{FRO} continuously impact each other, pushing the trajectory of V_{SRO} with respect to V_{FRO} to follow chaotic dynamics. The chaotic signal V_{SRO} is used to sample a high-frequency clock f_{High} for the generation of a random bit-stream, as shown in Figure 2.9. f_{High} can be generated from the system clock, or another dedicated RO can be used. The reported TRNG was implemented in a 45nm CMOS process, and the generated random bitstream passed all NIST tests with a throughput of 127 Mbps [13].

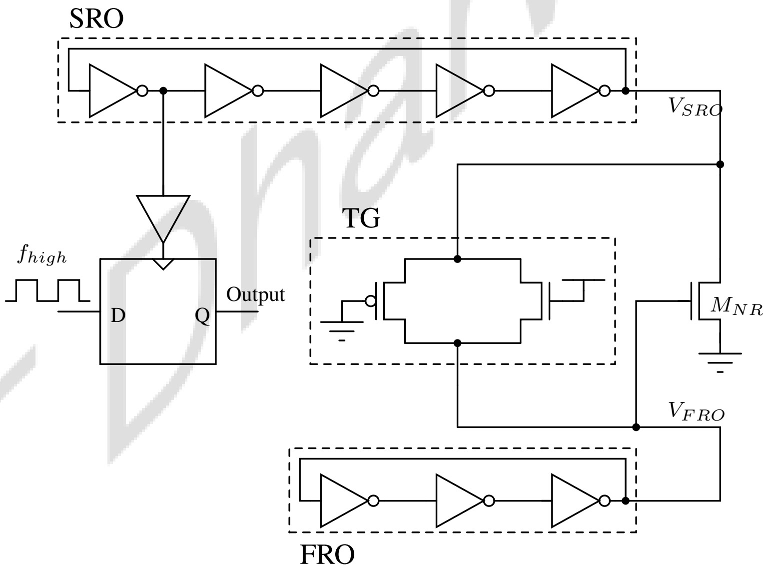

Figure 2.9: Chaotic Random number generator circuit: Electronic analogue to Double Pendulum [13].

The TRNG reported in [14] is designed for low-voltage IoT applications with limited power supplies and silicon area. As the supply voltage falls, the non-linear coupling effect due to the TG rapidly diminishes. In the proposed TRNG, a capacitive coupling approach is used, ensuring that the coupling effect between its two terminals persists even at low supply voltages. This approach is demonstrated to be more robust than diode or TG coupling methods. Using this coupling, the authors reported a 24-fold amplification in the standard deviation of jitter, even as VDD scales down to 0.4 volts. Figure 2.10 shows the proposed TRNG architecture, which consists of one FRO f_1 and two unequal length SROs f_{21} and f_{22}. Here, non-linear variations arise due to time-constant variations caused by non-linear output impedance and coupling capacitor.

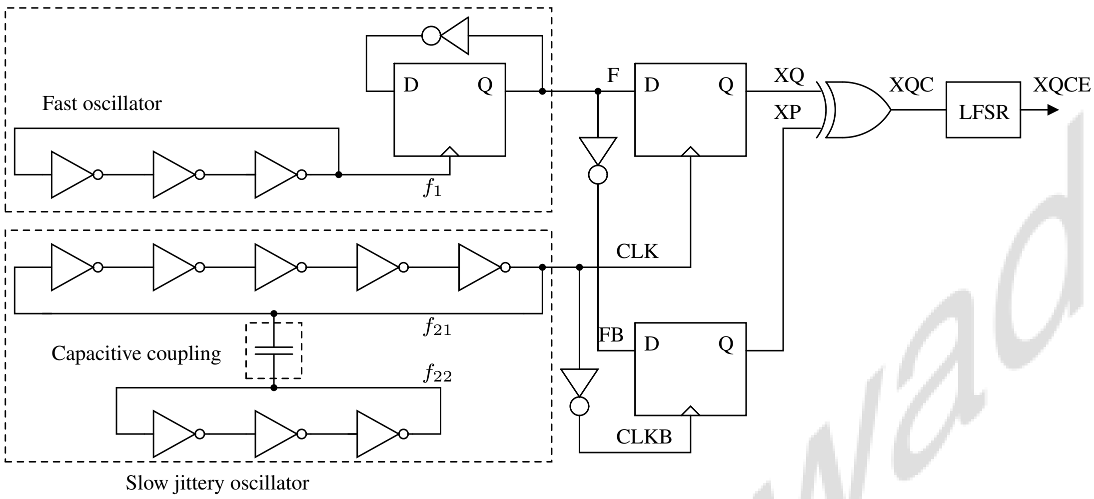

Figure 2.10: Reported design using capacitive coupling and dual-edge sampling. LFSR has been used to enhance the uniformity [14].

ROs generating clock with frequencies f_{21} and f_{22} are capacitively coupled, resulting in a chaotic dynamic system that amplifies the jitter noise of f_{21}. To achieve a 50% duty cycle, FRO output (f_1) is sent into a divide-by-two frequency divider. The frequency divider's output (F) has a frequency of $f_1/2$ and is connected to the first DFF's input, which is sampled by a jittered clock (CLK at the output of (f_{21}) to generate XQ. The DFF samples a random sequence because the jitter of f_{21} is significantly increased due to chaotic coupling, making it comparable to the period of f_1. XP is also produced by feeding the inverted versions of F and CLK (FB and CLKB) to the D and CK inputs of the second DFF, respectively. Due to the very high jitter in CLK and CLKB, XQ and XP are uncorrelated and XOR-ed to produce XQC. Finally, XQC is processed via a 4-bit linear feedback shift register to improve uniformity, resulting in XQCE (random sequence). The reported TRNG was implemented in a 65nm CMOS process, and it passed all NIST tests with a maximum throughput of 67 Mbps [14].

2.5 TRNG Based on Metastability

TRNG can also be designed by exploiting the sensitivity of memory elements near the metastable state to circuit noise. In this approach, random processes influence the final state between two equally desired outputs. Circuit noise determines the ultimate state of a bistable circuit at the metastable point, as illustrated in Figure 2.11. Here, the latch is forced into metastability using pre-charge devices to set both nodes a and b when CLK=0. The bistable element enters a metastable state at the rising edge of CLK where nodes a and b settle to a value V_{meta}, representing the intersection point of the Voltage Transfer Characteristics of inverters I_0 and I_1.

The magnitude of differential noise at nodes a and b during the metastable period determines whether stable states resolve to $(a = 0, b = 1)$ or $(a = 1, b = 0)$. A random noise source, such as thermal noise at nodes a and b, introduces variability during each clock evaluation phase, resulting in a random resolution state. Thermal noise peak-to-peak magnitudes of up to 2 mV on nodes a and b quickly push the system out of metastability due to the cross-coupled inverter's high gain from positive feedback. However, differential noise on nodes a and b and device mismatch in cross-coupled inverters I_0 and I_1 may disrupt the system's metastable state.

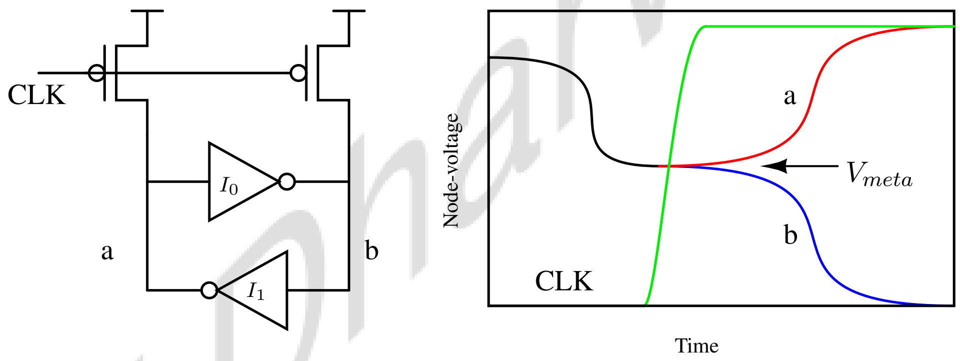

Figure 2.11: Perfectly matched Cross-coupled inverter pair pre-charged to VDD enters metastability. Random noise at nodes a and b introduces variability during each clock evaluation phase, resulting in a random resolution state.

A device mismatch in the cross-coupled inverters can produce an intrinsic bias, which will always push the bi-stable element in the bias's direction when it exits meta-stability. When there is a mismatch between the inverter devices, common-mode supply noise propagates differential components to nodes a and b, causing the system to be disrupted from the meta-stable point because the magnitude of random thermal noise is insufficient to overcome the mismatch-induced bias. This imbalance can have a considerable impact on the generated bit's entropy. This undesirable behavior can be avoided by reducing the mismatch between the bi-stable element's cross-coupled inverters. To harness the entropy of thermal noise using a bi-stable element, a way to compensate for static and dynamic mismatches in the cross-coupled inverters is required.

This ensures that the system's resolution state is determined solely by differential thermal noise at the two nodes, resulting in a random series of bits at nodes a and b.

The TRNG presented in [15] is shown in Figure 2.12. This architecture features cross-coupled inverters I_0 and I_1 biased into metastability and incorporates three digital calibration techniques to mitigate the impacts of PVT-induced mismatch: *(i) Coarse-grained tuning* using programmable inverters, *(ii) Fine-grained tuning* using delayed clocks, *(iii) self-calibrating feedback loop*. The reported design tolerated up to 20% PVT-induced mismatch using these calibrations techniques.

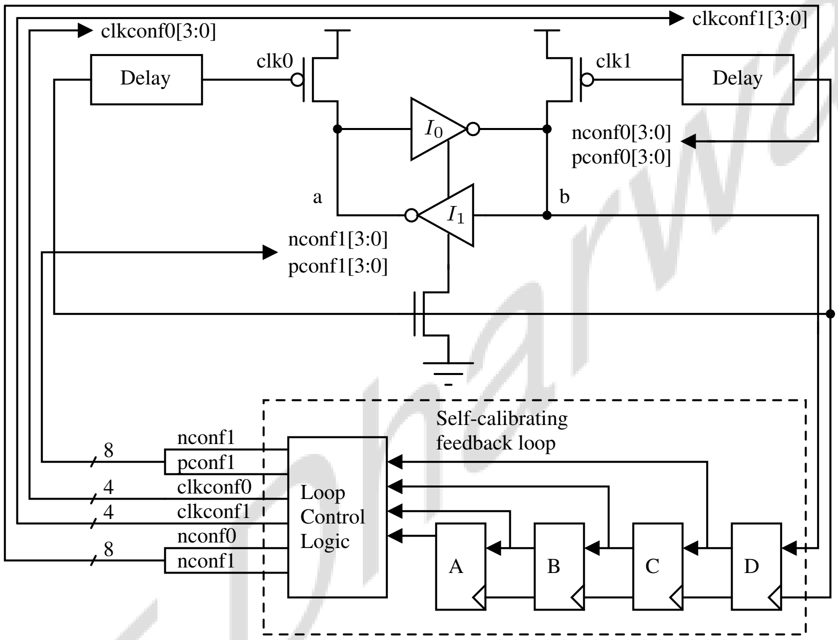

Figure 2.12: All digital variation-tolerant TRNG employing coarse and fine tuning calibration to force metastability without any bias [15].

Coarse-grained Tuning using Programmable Inverters

I_0 and I_1 in Figure 2.12 are converted to programmable inverters, each having eight digital configuration bits, *pconf[3:0]* and *nconf[3:0]*, which control effective drive strengths of PMOS and NMOS transistors, respectively (Figure 2.13). These bits regulate the inverter's Positive/Negative (P/N) skew from the nominal planned value by conditionally turning ON parallel PMOS/NMOS legs. Coarse-grained Tuning counters a wide range of PVT-induced mismatches with the right mix of *pconf[3:0]* and *nconf[3:0]*.

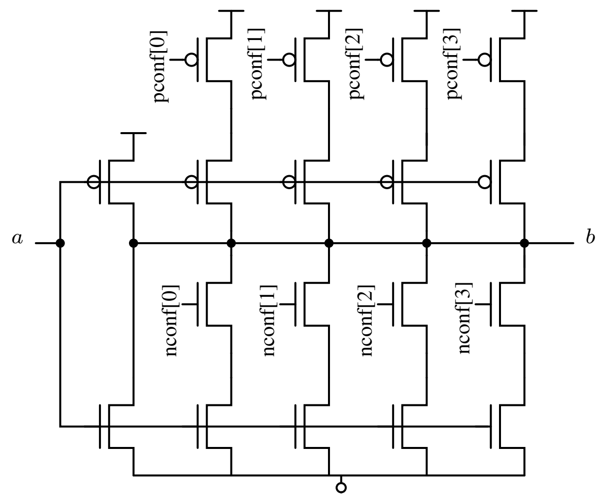

Figure 2.13: Coarse-grained tuning: Inverter with 4-bit programmable PMOS/NMOS strength to counter process variation-induced imbalances [15].

Fine-Grained Tuning Using Delayed Clocks

A relative delay can be introduced between the start of the evaluation phase at each node by separating the pre-charge clocks and introducing a tunable skew between them using programmable delay generators (refer Figure 2.14). This allows for controlling the pre-charge release times for nodes a and b, injecting a directional bias on one side of the bi-stable element. The injected bias is used to compensate for any remaining biases in the system. Suppose node b inclines to resolve to 0. In that case, clk1 can be delayed relative to clk0, keeping node b in pre-charge for a fraction longer than node a. Node b delayed evaluation counteracts its deterministic inclination to resolve to 0. *clkconf0* and *clkconf1* can be used to add or subtract skew on either side of the bi-stable element, resulting in up to 255 permutations of fine-grained configurations.

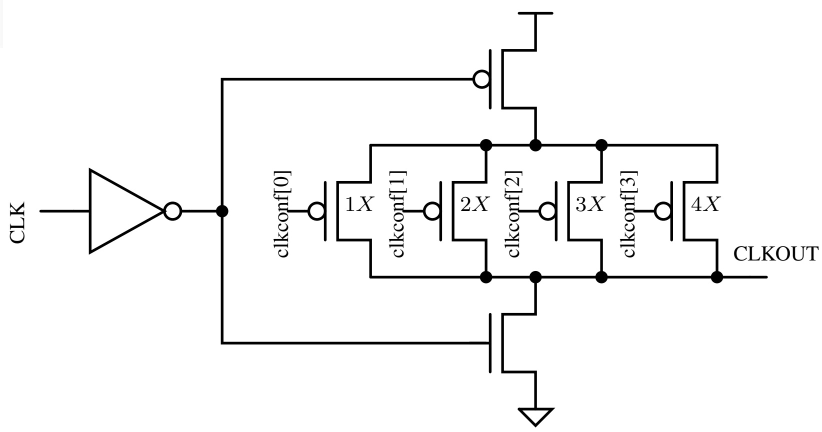

Figure 2.14: Fine-Grained Tuning: Delay Element using 4-bit programmable delay buffers [15].

Self-Calibrating Feedback Loop

The self-calibrating feedback loop is the fundamental component of the reported TRNG that makes it variation-tolerant, continuously monitors the output bit, identifies the presence of $0 - bias$ or $1 - bias$ in the system, and utilizes either coarse-grained or fine-grained tuning to offset the influence of any bias. If the system is discovered to be unbalanced, one of the six 4-bit counters *(nconf0, pconf0, nconf1, pconf1, clkconf0, and clkconf1)* is incremented or decremented to bring it back into balance. The values saved in these counters are used as configuration bits of the tunable inverters and programmable delay buffers. The counter values are left unchanged for the next cycle if no bias is found in a cycle. The reported TRNG was implemented using a 45nm CMOS process that demonstrated full digital entropy extraction and passed all NIST tests at a throughput of 2.4 Gbps. However, the complex digital calibration imposes significant power and area requirements, which were measured at 7 mW and 4004 μm^2, respectively, as reported in [15].

2.6 TRNG Based on common-mode comparator

The TRNG reported in [16] utilizes a common-mode operational comparator and the sampling uncertainty of a DFF to generate random bitstream, as shown in Figure 2.15. It includes a comparator with a Differential to Single amplifier (D2S), a biasing circuit (beta-multiplier voltage reference), a slicer, a DFF, and an output driver. Unlike clocked comparators in meta-stability, which have limitations on data rate due to slower output slope, the proposed design combines a continuous comparator and slicer, achieving a data rate of 3 Gbps. A sampling clock frequency of 3 GHz is chosen based on the minimum and maximum pulse widths of V_{DFFin} to maintain output stability and avoid excessively long run lengths.

The biasing circuit and comparator generate thermal noise, serving as the entropy source for this TRNG. The beta-multiplier voltage-reference bias circuit is designed to establish a bias point at half of V_{DD}. The comparator operates in common mode, as all its inputs are connected to the bias circuit's output. The thermal noise produced by the bias circuit and comparator is combined and amplified by the differential to single amplifier at the metastable point. Based on the thermal noise input, the slicer then categorizes the output voltage into high and low values. A continuous comparator and slicer designed for asynchronous random sequence generation introduce ambiguity when sampled by a $3GHz$ clock signal (CK_{3GHz}). This uncertainty, linked to the arrival moment of data V_{DFFin} in the DFF, enhances the overall randomness. The reported TRNG was implemented using a 65nm CMOS process with a supply voltage of 1.2 volts. The random sequence was sampled at a rate of 3 Gbps, and it passed all the NIST tests. The complete circuit consumed 5 mW power and took 1570 μm^2 area respectively [16].

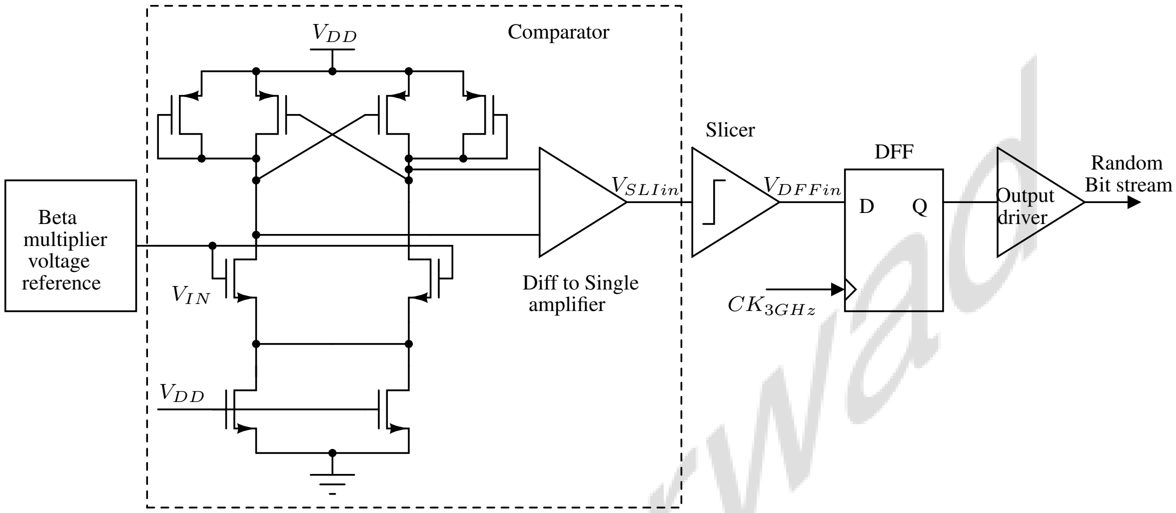

Figure 2.15: TRNG circuit using Common-Mode operating Comparator and sampling uncertainity of D Flip-Flop (DFF) [16].

2.7 TRNG Based on SAR ADC

Figure 2.16 illustrates a TRNG-cum-Successive Approximation Register (SAR) ADC architecture presented in [17]. This architecture functions as both an Analog-to-Digital Converter (ADC) and a TRNG without imposing limitations on the regular SAR operation. Following the completion of all SAR cycles, the SAR residue, which includes quantization noise and thermal noise, is present at the input of the 1-bit quantizer. The TRNG output requires one more clock cycle by reusing the same comparator to quantize this residue into a 1-bit output. The use of the same comparator for SAR conversion and TRNG generation eliminates bias introduced by comparator offset, presenting a notable advantage as comparator offset calibration is not required for bias removal. To ensure the SAR residue acts as an entropy source for random number generation, it is crucial that the SAR quantization noise has a white PSD and low correlation with the input signal.

The ADC resolution plays a key role in achieving low correlation, with higher resolutions resulting in a lower correlation between the quantization noise and the input signal. Within

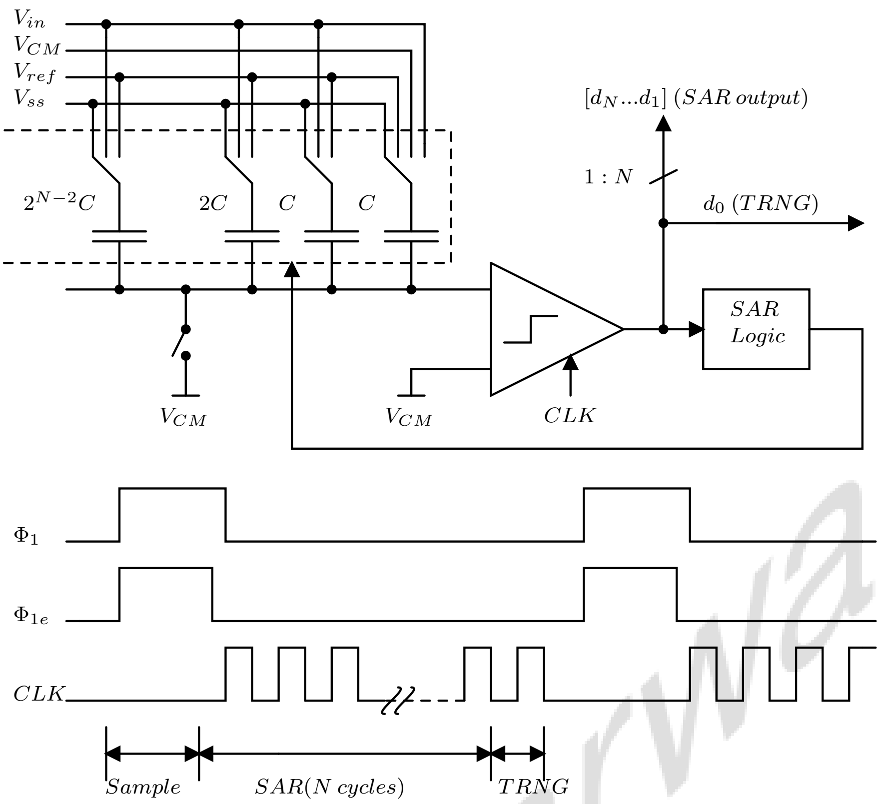

Figure 2.16: Reported SAR ADC based TRNG with its timing diagram [17].

the proposed architecture, operating under this scheme, basic randomness testing is conducted using the Discrete Fourier Transform (DFT) test from the NIST statistical randomness suite. The simulation results demonstrate that for ADC resolutions exceeding 6 bits, the TRNG consistently passes all 150 DFT tests, promising a high confidence in its randomness.

A 10-bit SAR ADC prototype, developed in a 65nm CMOS process, demonstrates that its quantized residue serves as a TRNG output across a broad power supply range (0.6V-1.2V) and temperature range (-5°C to 50°C). This prototype generates the random sequence with a throughput of 1.25 Mbps and consumes 0.22pJ per bit without requiring calibration or post-processing. The generated random sequence successfully passed all recommended NIST tests, validating its statistical randomness. Additionally, the ADC exhibits an Effective Number of Bits (ENOB) of 8.6 bits and a Walden Figure of Merit of 7.8fJ/step. However, the entropy of the random sequence depends on quantization noise, which is white only when the actual ADC inputs are busy [17].

2.8 TRNG Based on Delta-Sigma Modulator

The TRNG architecture described in [18] is shown in Figure 2.17. It proposes a mixed-signal TRNG designed to operate simultaneously as both a TRNG and an ADC utilizing a CTDSM. The TRNG harnesses entropy from both thermal noise and jitter within the ADC. Clock jitter, which can significantly degrade the signal-to-noise ratio (SNR) of the CTDSM, affects the loop in the feedback path, directly impacting the output. The negative feedback loop forces the differential input Voltage Controlled Oscillator (VCO)s in Figure 2.17 to track each other, irrespective of the input signal. This mechanism guarantees that noise, rather than the input signal, determines the input signal of the VCO. The TRNG sequence is generated by sampling the Least-Significant Bit (LSB) of the ADC's outputs. To reduce bias, the raw data sampled from four sub-ADCs are combined using XOR operations for the TRNG bitstream. The reported TRNG was fabricated using a 65nm CMOS process, generated a random bit sequence at a throughput of 52 Mbps, and successfully passed all NIST tests. In this case, as well the entropy of TRNG output could degrade for a non-busy input signal to the ADC [18].

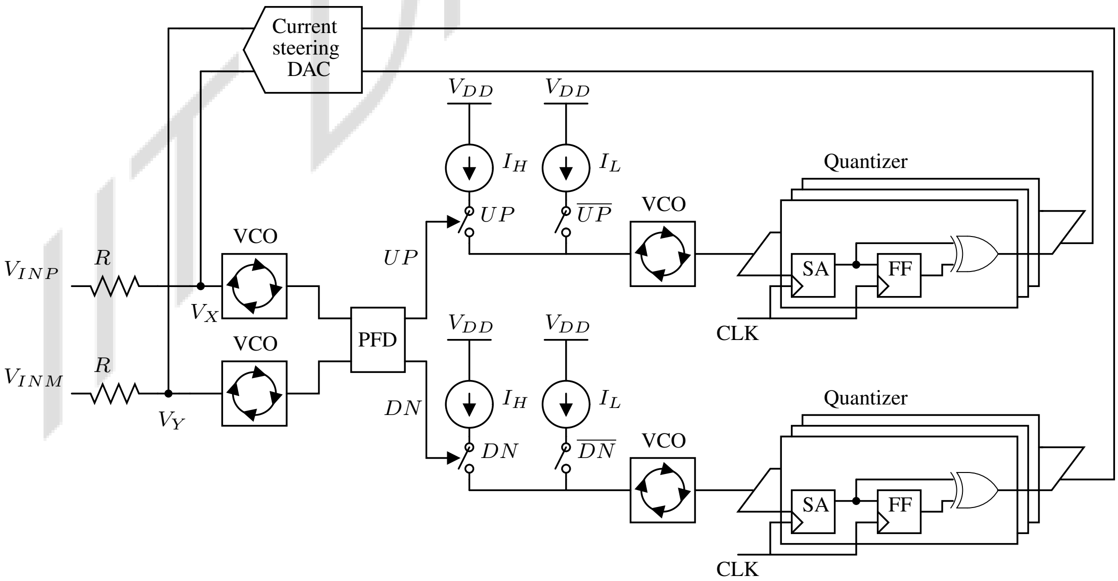

Figure 2.17: TRNG based on Delta-sigma based ADC using jitter from VCO as entropy source [18].

2.9 Post-Processing Techniques

There are several techniques to completely kill or at least reduce the bias of a given bit stream. De-bias using the Von-Neumann approach is popularly used, which completely removes the bias in the sequence at the cost of significantly reduced throughput. The corrector operates on a bit stream, analyzing non-overlapping pairs of successive bits. It generates outputs according to two rules: (1) If the input pair is 00 or 11, it is discarded without producing any output, and (2) If the input pair is 01 or 10, only the first bit appears at the output. This process helps in debiasing the original bit stream by selectively retaining the first bit of pairs with differing values while discarding pairs with identical values. Suppose the input bits have bias e, meaning that for an input bit x,

$$Pr(x=0) = \frac{1}{2} + e \quad \text{and} \quad Pr(x=1) = \frac{1}{2} - e$$

Then, for a given output bit y,

$$Pr(y=1) = Pr(y=1|\text{there is an output}) = \frac{Pr(10)}{Pr(01 \; or \; 10)}$$

$$= \frac{(\frac{1}{2}-e)(\frac{1}{2}+e)}{(\frac{1}{2}+e)(\frac{1}{2}-e)+(\frac{1}{2}-e)(\frac{1}{2}+e)} = \frac{\frac{1}{4}-e^2}{2(\frac{1}{4}-e^2)} = \frac{1}{2}$$

The occurrence rate of a favorable pair (01 and 10) is calculated as:

$$Pr(01) + Pr(10) = 2 \cdot \left(\frac{1}{2} + e\right) \cdot \left(\frac{1}{2} - e\right) = 2 \cdot \left(\frac{1}{4} - e^2\right)$$

However, since each pair contributes an output equal to half its length, the effective rate of the corrector is determined to be $\frac{1}{4} - e^2$. Thus, the Von Neumann corrector achieves, at most, a rate of 1/4, discarding a minimum of 75% of the input bits [19].

A more popular approach is XORing multiple biased but uncorrelated bit streams. This is based on the principle of Piling-up lemma [20]. The lemma states that the bias (deviation of the expected value) of a linear Boolean function of independent binary random variables is related to the product of the input biases given by the equation 2.3.

$$\epsilon\left(X_1 \oplus X_2 \oplus \cdots \oplus X_n\right) = 2^{n-1} \prod_{i=1}^{n} \epsilon\left(X_i\right) \tag{2.3}$$

Here $\epsilon(X_i)$ indicates bias in the individual random process $X_i[n]$. For a given $X_i[n]$, $P(X = 0) = p$ and $P(X = 1) = 1-p$. Then, $\epsilon(X_i)$ can be defined as $\left(\epsilon = \frac{|2p-1|}{2}\right)$. From equation 2.3, we can see that XORing independent binary variables always reduces the bias (or at least does not increase it), and the output becomes unbiased if there is at least one unbiased input variable. Compression based on error-correcting codes is also used to reduce redundancy, thereby maximizing the entropy of the generated sequence. Linear error-correcting codes, such as the BCH code denoted as $[n, k, d]$, are designed to process and decode n-bit data sequences. These codes aim to transform the original biased n-bit data into an unbiased k-bit representation $(k < n)$, utilizing a minimum Hamming distance of d. If the bias in the n-bit sequence is e, the resulting bias in the decoded k-bit bitstream can be expressed as $2^{(d-1)} \cdot (e)^d$. Additionally, the throughput of the decoded bitstream, representing the ratio of useful information to the total information, gets factored by $\frac{k}{n}$. Error correction aids in bias reduction by maximizing compressibility and achieving a good tradeoff between throughput and acceptable bias [19].

Chapter 3

Analysis of Delta-Sigma Modulator for TRNG

Delta-Sigma modulator functions as a closed-loop negative feedback control system, generating a pulse stream with an average value directly proportional to the reference analog input voltage. Delta-sigma modulation inherently involves a form of frequency modulation in which the input signal is oversampled, and the difference between the actual and quantized signals (the delta Δ, or error) is accumulated (sigma Σ). This process effectively pushes most of the quantization noise to higher frequencies.

Based on the type of signal loop filter processes, there are two types of delta-sigma modulators, i.e., Discrete-Time Delta-Sigma Modulator (DTDSM) and Continuous-Time Delta-Sigma Modulator. In DTDSM, the input signal is sampled before the loop filter; thus, the loop filter processes discrete-time signals and is implemented using switched-capacitor integrators. On the other hand, the input signal is sampled after the loop filter in a CTDSM; hence, the loop filter processes continuous-time signals and is often implemented using Active-RC or $G_m - C$ integrators.

This chapter starts with an overview of the fundamental aspects of delta-sigma modulation, covering oversampling, noise shaping, and the synthesis of its transfer functions. This lays the groundwork for the rest of the report. Afterwards, non-idealities, such as clock jitter and quantizer metastability, along with their impact on the overall performance of the modulator, have been discussed. A brief discussion about return-to-zero Digital-to-Analog Converter (DAC) and its ability to compromise the inherent alias rejection property of CTDSM is done in the next section. Based on the concepts developed, a comprehensive noise analysis of the

CTDSM has been conducted for multiple design cases. At last, a takeaway summary has been provided for the design of TRNG based on the insights gained from the noise analysis.

3.1 An Overview of Delta-Sigma Modulator

3.1.1 Sampling and Quantization

Analog-to-digital converters (ADCs) are essential for converting real-world signals like voice, audio, or video, which are continuous in time and amplitude, into digital signals that can be processed by Digital Signal Processors (DSP). The Anti-Aliasing Filter (AAF) is a crucial component in this process. It ensures that the input signal's bandwidth is limited to half of the sampling rate, as required by the Nyquist theorem. This prevents unwanted higher-frequency components from "aliasing" or folding back into the desired frequency range during sampling.

To convert the analog signal $V_{in}(t)$ into its digital version $v[n]$, we start by filtering it using an AAF, resulting in a bandlimited analog signal $x(t)$. The next step would be to sample $x(t)$ at sampling frequency of f_s to get $x[nT_s]$ where $T_s = {f_s}^{-1}$ (refer to Figure 3.1).

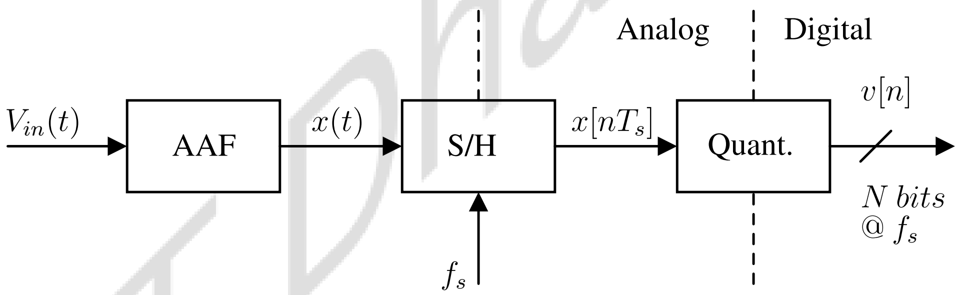

Figure 3.1: The fundamental process of analog-to-digital conversion includes filtering to band limit the signal, sampling to discretize the signal in time, and quantizing to assign digital values to each sample.

Mathematically, it can be modeled as multiplying $x(t)$ with Dirac-delta train having period T_s. Thus we have,

$$x(t) \cdot \sum_{n=-\infty}^{\infty} \delta\left(t - nT_s\right) = \sum_{n=-\infty}^{\infty} x[nT_s]\, \delta\left(t - nT_s\right). \tag{3.1}$$

The Fourier transform (FT) of the sampled signal can be obtained by convolving $x(t)$ and Dirac-delta train in the frequency domain, which is given in the equation 3.2. Here, $X_c(f)$ represents the FT of $x(t)$.

$$\mathcal{F}\left\{\sum_{k=-\infty}^{\infty} x[nT_s]\, \delta\left(t - kT_s\right)\right\} = f_s \sum_{n=-\infty}^{\infty} X_c\left(f - nf_s\right), \tag{3.2}$$

The Discrete-time Fourier transform (DTFT) of the sampled signal $x[nT_s]$, denoted by $X_d(e^{j\omega})$, can be obtained by transforming continuous-time frequency variable "f" to discrete-time frequency variable "ω" using $\omega = 2\pi f T_s$ in equation 3.2. This gives,

$$X_d\left(e^{j\omega}\right) = \mathcal{F}\{x[nT_s]\} = \sum_{n=-\infty}^{\infty} x[n]e^{-j\omega n} = f_s \sum_{n=-\infty}^{\infty} X_c\left(\frac{f_s}{2\pi}\left(\omega - 2\pi n\right)\right). \tag{3.3}$$

We can observe that $X_d(e^{j\omega})$ is periodic, and the spectrum repeats itself with the period of f_s. This gives rise to the Nyquist band limitation that the maximum frequency component in the sampled signal should be equal to or less than half of the sampling frequency in order to prevent aliasing. The subsequent step after sampling in the analog-to-digital conversion process is quantization. Quantization involves approximating a continuous amplitude signal with discrete values, introducing quantization error. This error arises due to the difference between the original continuous signal and its discrete representation. The quantization error is assumed to be *additive noise source*, have *white spectrum* in $[-f_s/2, f_s/2]$, *uniformly distributed* between $\pm V_{LSB}/2$ and *independent* of the input signal. However, one has to note that these assumptions are valid for sufficiently "Busy" input signals. Figure 3.2 shows a linear and practical model of a quantizer where $e[n]$ represents an additive quantization error sequence. z^{-1/f_s} represents one sample delay due to the finite time taken by the practical quantizer to resolve [21] [22].

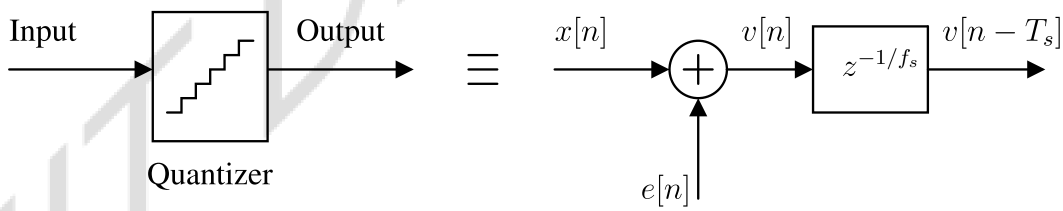

Figure 3.2: Linear model of quantizer as additive noise sequence along with a sample delay.

3.1.2 Oversampling

As the name suggests, oversampling is sampling the signal at a rate greater than the Nyquist rate. *Oversampling Ratio (OSR)* is the number that compares actual sampling rate f_s to Nyquist rate $2B$ [B is the sampling signal bandwidth], and it is defined as $OSR \equiv \frac{f_s}{2B}$. In the case of Nyquist rate sampling, we need brick wall AAF response to prevent high-frequency noise from $[k \cdot f_s - B, k \cdot f_s + B]$ to alias back into signal band $[-B, B]$.

The benefit we get from oversampling the input signal is design relaxation for an anti-aliasing filter. By having higher OSR, the copies are spaced far from each other, thus relaxing the design of the filter to have a wider transition band. Another benefit of oversampling the input, followed by quantization and low pass filtering, is quantization noise reduction, thus improving the SQNR in its Digital representation. For an oversampled signal, the signal bandwidth should be limited to $\frac{f_s}{2 \cdot OSR}$ Hz, whereas quantization noise spectral density remains $\frac{V_{LSB}^2}{12} \cdot \frac{2}{f_s}\, V^2/Hz$ spread up to $\frac{f_s}{2}$ Hz. This results in effectively lower quantization noise floor, thus reducing quantization noise power in signal bandwidth by the factor of OSR [21].

3.1.3 Noise Shaping

Delta-Sigma modulator takes a low-resolution quantizer, making it look like a high-resolution quantizer by improving its SQNR. For the delta-sigma modulator, two processes are responsible for higher SQNR compared to the stand-alone quantizer, namely *Oversampling* and *Noise shaping*. Now, we will look into the conceptual development of the delta-sigma modulator, which performs oversampling and noise shaping in a single negative feedback loop. The output of the linear model of the quantizer has an input sequence in addition to some error sequence. If we put that model into negative feedback with a high gain amplifier having frequency-independent constant gain A (refer Figure 3.3). Here, the intention is to reduce the quantization error altogether using negative feedback by considering the sampled input sequence as a reference.

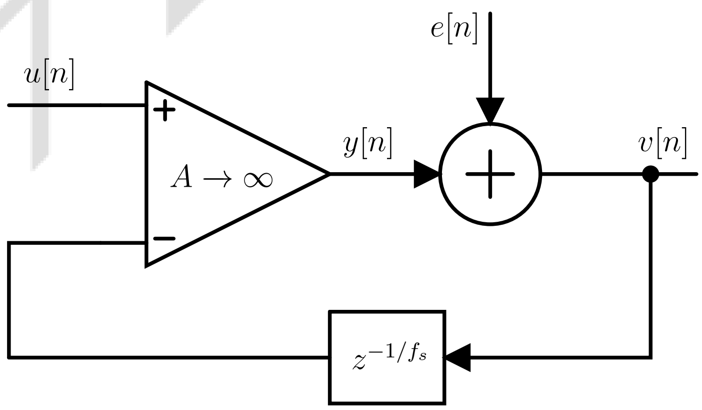

Figure 3.3: Quantizer in negative feedback with a high gain amplifier.

Transfer function from the input $u[n]$ to its quantized version is termed as *Signal Transfer Function (STF)*, whereas additive quantization noise $e[n]$ sees a different transfer function called

Noise Transfer Function (NTF). Analysis of Figure 3.3 results in:

$$V(z) = \underbrace{\left(\frac{A}{1 + Az^{-1/f_s}}\right)}_{\text{STF}} \cdot U(z) + \underbrace{\left(\frac{1}{1 + Az^{-1/f_s}}\right)}_{\text{NTF}} \cdot E(z) \tag{3.4}$$

From the equation, we can observe that in NTF, as $A \rightarrow \infty$, the STF approaches unity, and NTF tends to zero. This suggests $e[n]$ is not affecting the output altogether, and $v[n]$ becomes exact replica of u[n] and behaves as an infinite resolution quantizer. However, if we look closely at the pole location of the above system determined by the characteristic equation, it turns out to be $z = (-A)^{f_s}$. For $A \rightarrow \infty$, the pole of the system lies outside the unit circle, making the overall system unstable. The problem in using frequency-independent gain A along with quantizer in negative feedback is we need $A \rightarrow \infty$ to suppress the additive noise $e[n]$. On the other hand, we need $A < 1$ for the pole of this discrete-time system to lie inside the unit circle to be stable [21].

Clearly, it is not possible to completely eliminate $e[n]$ and have an infinitely precise quantizer. We can exploit the fact that the reference input of this negative feedback system is band limited to $\frac{f_s}{2 \cdot OSR}$ Hz. Instead of suppressing additive quantization noise altogether by using frequency-independent gain $A \rightarrow \infty$, it is possible to shape out $e[n]$ from the signal band, i.e., $\left[0, \frac{f_s}{2 \cdot OSR}\right]$ to out-of-band, i.e., $\left[\frac{f_s}{2 \cdot OSR}, \frac{f_s}{2}\right]$ using frequency-dependent gain. Through noise shaping, the distribution of quantization noise is purposefully manipulated across the frequency spectrum. The quantization noise floor can be lowered by adjusting the loop gain such that it is shaped out from the signal band by putting the quantizer into negative feedback with an appropriate loop filter. Let's suppose A is replaced by a block whose gain is high only in the signal band and roll-off at higher frequencies. This will ensure that NTF has a very low gain in the signal band. Since our reference input is discrete, the lowest-order system that meets all the requirements can be the first-order discrete time accumulator. So A can be replaced by $\frac{1}{1 - z^{-1/f_s}}$. Thus, we have a typical 1st order Discrete-time Delta-Sigma modulator (MOD1 DTDSM) given in Figure 3.4.

The STF and NTF of MOD1 DTDSM can be found by putting $A = 1/(1 - z^{-1/f_s})$ in equation 3.4. We see that the pole of MOD1 DTDSM lies at $z = 0$, which is inside the unit circle, making it completely stable [21].

$$V(z) = \underbrace{1}_{STF} \cdot U(z) + \underbrace{\left(1 - z^{-1/f_s}\right)}_{NTF} \cdot E(z) \tag{3.5}$$

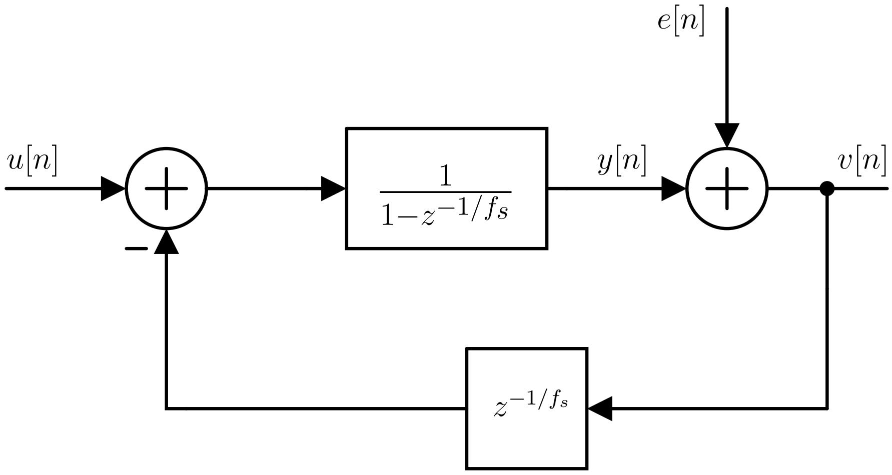

Figure 3.4: 1st order DTDSM loop (MOD1) model [21].

Thus, we have STF as unity and NTF as $(1 - z^{-1/f_s})$, which is a first-order high pass response. This ensures that the output of MOD1 DTDSM, $V(z)$, carries a quantized version of the input along with the shaped quantization noise. Figure 3.5 shows the output spectrum of the quantizer for different cases. If a given signal $u(t)$ gets sampled at Nyquist rate and subsequently quantized, the quantization noise floor appears across its entire bandwidth B, and corresponding noise power is given by σ_q^2. If the same signal gets over-sampled by a factor of OSR, the total quantization noise power gets distributed over a larger bandwidth, resulting in a lower noise floor in the signal band. In this case, total quantization noise power in the signal band gets factored by OSR and becomes $\frac{\sigma_q^2}{OSR}$. In MOD1 DTDSM, the analog input first gets over-sampled, and then the quantization noise floor gets high-pass-shaped, resulting in even lower quantization noise power in the signal band given by $\frac{\sigma_q^2}{OSR} \cdot \int_0^B |NTF|^2 \, df$. Here the quantization noise power σ_q^2 gets factored by OSR as well as shaped out from the signal band, resulting in even better SQNR.

3.1.4 Synthesis of CTDSM

The discrete-time accumulator in DTDSM is realized using switched capacitor integrators. Higher-order transfer functions are implemented in what is commonly referred to as the loop filter to enhance performance and fine-tune the characteristics of delta-sigma modulation. Switched capacitor integrators are commonly employed as fundamental building blocks to implement loop filters in delta-sigma modulators.

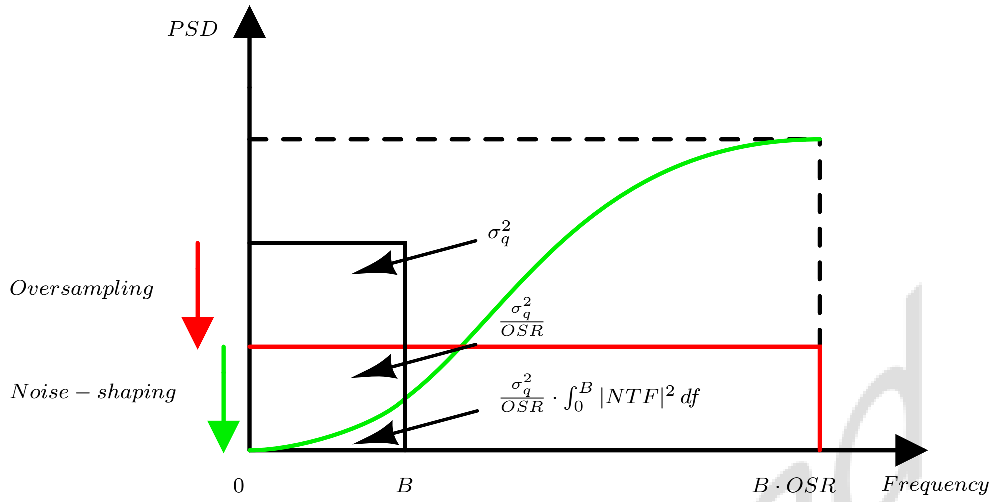

Figure 3.5: In-band quantization noise reduction by oversampling and noise-shaping.

In DTDSM, the input signal is over-sampled upfront, and the difference between the input and fed-back sequence is processed using a discrete-time loop filter. Instead of using a discrete-time loop filter, a continuous-time loop filter can also be employed, which doesn't involve periodic charging-discharging of multiple capacitors. Here, the continuous-time input signal is fed to the modulator, from which the quantizer's output waveform is subtracted and processed by the continuous-time loop filter. Quantizers can be realized as a cascade of ADC & DAC (Figure 3.6). The ADC produces a discrete-time, discrete-valued signal, which gets encoded to digital bits and forms the output of the modulator. DAC generates a continuous-time feedback waveform based on the digital output by associating pulse shape proportional to the input. Figure 3.6 shows a block diagram of CTDSM. It can't be analyzed conventionally since it is part linear (loop-filter), part non-linear (quantizer), part time-invariant (loop-filter), part time-variant (sampler), part continuous-time, part discrete-time, all enclosed in a negative feedback loop [21].

Impulse-invariant transformation

Here, the new topology is expected to shape quantization noise similar to DTDSM, i.e., having the same NTF as DTDSM. For that, *Impulse-invariant transformation* has been used to mimic the same noise-shaping property. The impulse-invariant method converts analog filter transfer functions to digital filter transfer functions so that the impulse response is identical at the sampling instants (Figure 3.7).

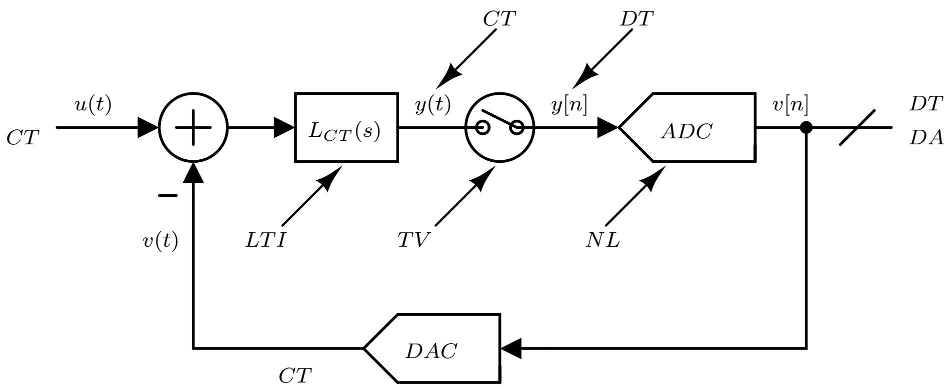

Figure 3.6: Block diagram of CTDSM showing it being part linear (loop-filter), part non-linear (quantizer), part time-invariant (loop-filter), part time-variant (sampler), part continuous-time, part discrete-time, all enclosed in a negative feedback loop [21].

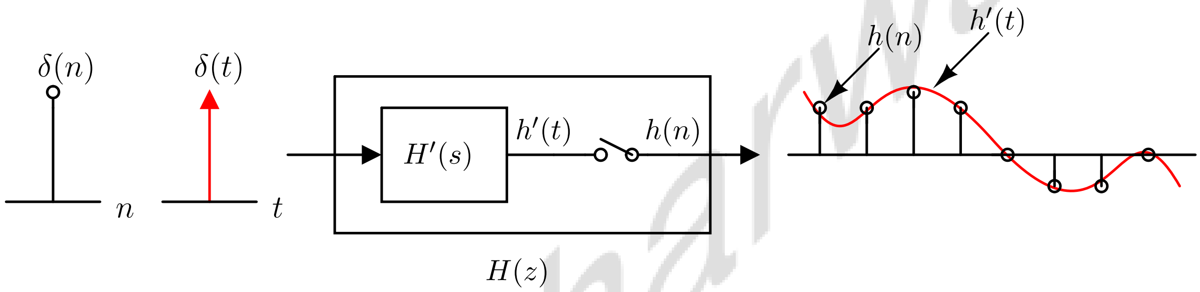

Figure 3.7: Impulse-invariant transformation

Even though we have to implement the loop filter using continuous-time circuitry in CTDSM, we have to ensure that the quantization noise sees the same NTF. To demonstrate it, we will try to replicate the MOD1 DTDSM loop response using a continuous-time loop filter. The aim is to ensure that the quantizer gets the same impulse response after sampling as DTDSM.

Loop impulse response in DTDSM

We will take a MOD1 DTDSM model developed in section 3.1.3. The single sample delay of the quantizer is merged with the loop filter's transfer function so that it has the same loop gain. To find out the loop impulse response at quantizer input, we have to break the loop and excite it with unit impulse after deactivating the input source, as shown in Figure 3.8.

In Figure 3.8, the DT unit impulse has been passed through a cascade of one sample delay followed by the discrete-time accumulator. Thus, we will get MOD1 DTDSM loop impulse response $l_{DT} = 0, 1, 1, 1...$

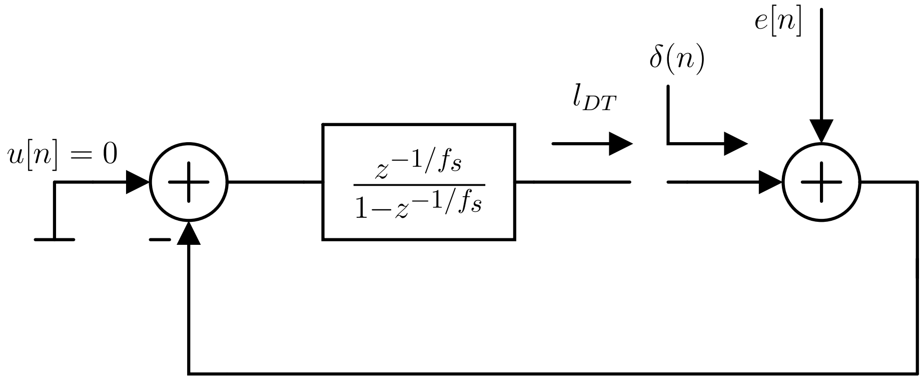

Figure 3.8: Exciting DTDSM loop with discrete-time unit impulse

The first sample of the sequence is zero due to one sample delay z^{-1/f_s}, but after that, unit impulse keeps on accumulating due to $\frac{1}{1-z^{-1/f_s}}$ as given in Figure 3.9.

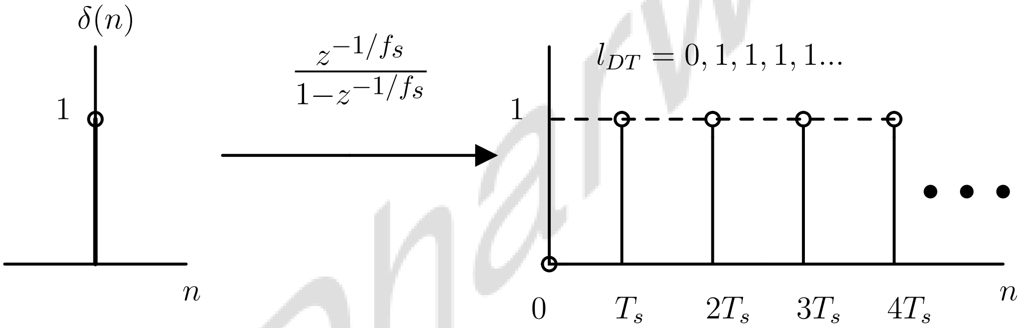

Figure 3.9: Loop impulse response in DTDSM

Loop impulse response in CTDSM

Again, a similar exercise must be done in CTDSM to determine what $L(s)$ should be to ensure the same loop impulse response. We will break the loop at quantizer input after the sampler and apply the discrete-time impulse (Figure 3.10). What comes back after the sampler must be identical to whatever we got in the case of DTDSM, i.e., $l_{CT} = l_{DT}$. By ensuring this, the quantizer will see the same loop response and won't know whether the CT or DT loop filter has been used [21].

DAC Impulse response

Every DAC is associated with a pulse shape $p(t)$ based on its implementation. These pulses wiggle at each clock edge such that their average corresponds to the final analog output. A

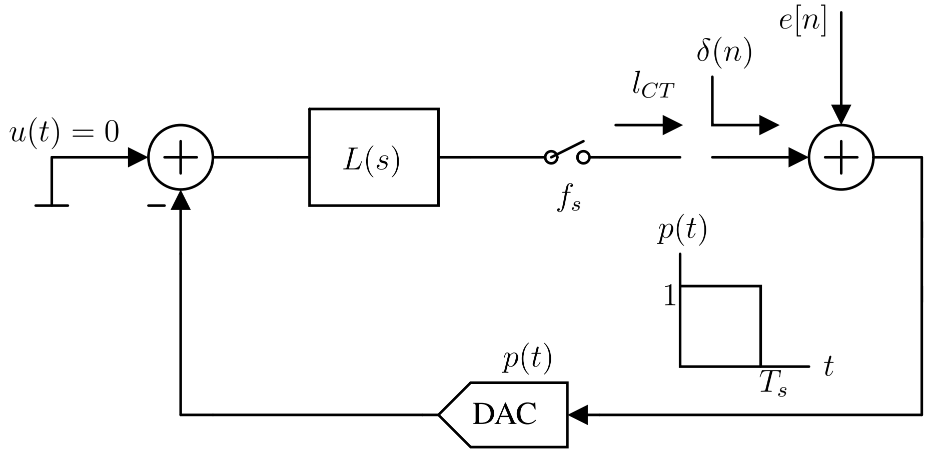

Figure 3.10: Exciting CTDSM loop with discrete-time unit impulse

DAC's operation can be mathematically modeled in two steps. It takes DT and DA impulses as input. First, the DT impulse sequence gets converted to the CT impulse train. Then, that impulse train passes through a filter impulse response $p(t)$. The impulse response of some commonly used pulse shapes is shown in Figure 3.11.

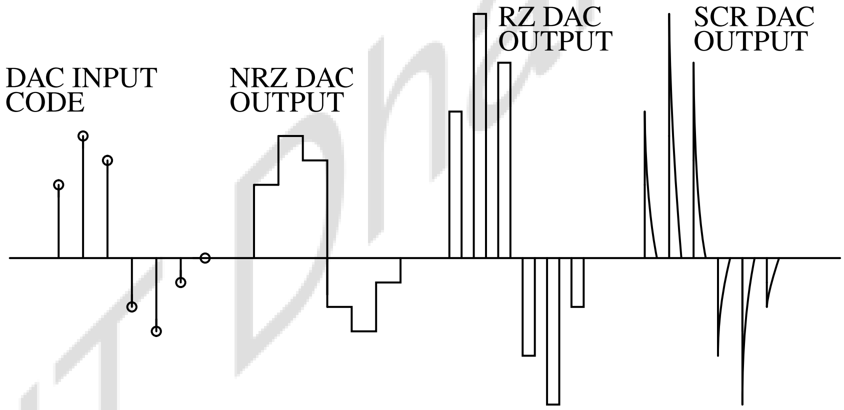

Figure 3.11: Pulse shape associated with DACs

DAC output associates $p(t)$ for an incoming DAC input code such that $p(t) = 1, 0 < t < T_s$ for Non-Return to Zero (NRZ) DAC, $p(t) = 2, 0 < t < 0.5T_s$ for Return to Zero (RZ) DAC and $p(t) = 2, t = nT_s$ and decays exponentially with time period much less than $T_s/2$ for Switched-Capacitor Resistor (SCR) DAC. These DAC outputs can be further averaged out using a low-pass filter to get the analog signal that is linearly related to the incoming DAC input code [21].

Now, we will go back to replicating the loop impulse response of CTDSM l_{CT} with l_{DT}. The DT unit impulse sees the input of the DAC having NRZ impulse response $p(t)$ followed by the loop filter $l(t)$ (impulse response of $L(s)$) and gets sampled at a rate of f_s as shown in Figure

3.12. $\delta(n)$ will be inherently converted to CT unit impulse $\delta(t)$ and sees the filter $p(t)$which result in the function $[u(t) - u(t - T_s)]$. Again $[u(t) - u(t - T_s)]$ passes through $l(t)$ to produce the trace $[f_s \cdot r(t) - f_s \cdot r(t - T_s)]$, so that after sampling at T_s, it should give $l_{CT} = l_{DT}$.

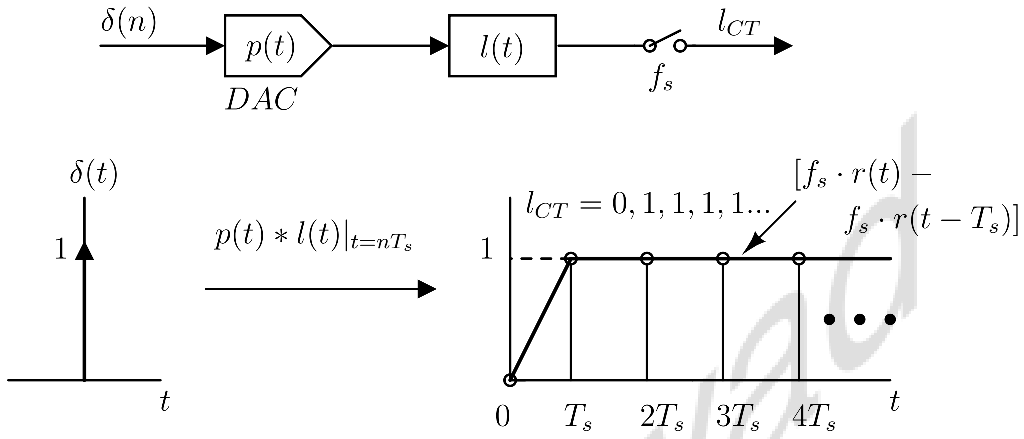

Figure 3.12: Deriving loop impulse response for CTDSM by forcing the loop to mimic the same loop response as of MOD1 DTDSM.

As apparent, $L(s)$ should be an ideal integrator with time constant $T_s = 1/f_s$. The resulting CTDSM has been shown in Figure 3.13. This is derived from MOD1 and designed to mimic the same NTF as its discrete-time counterpart [21].

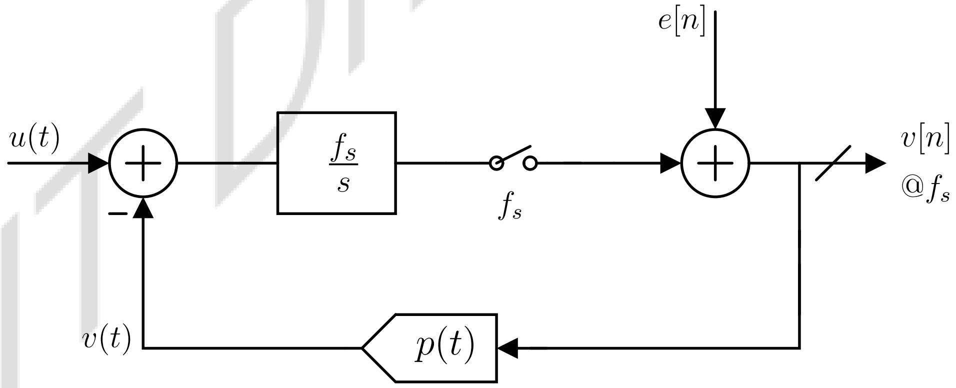

Figure 3.13: Block diagram model for MOD1 CTDSM [21].

3.1.5 STF and NTF of MOD1 CTDSM

CTDSM has both continuous-time and discrete-time signals flowing through the loop. To get the expression of STF & NTF for MOD1 CTDSM with sampling frequency f_s, the loop is re-arranged to analyze these signals separately. In Figure 3.14, the input signal $u(t)$ first sees continuous-time loop filter f_s/s and becomes $y_1(t)$. After being sampled at the clock frequency f_s, $y_1(t)$ becomes $y_1[n]$. The additive quantization noise $e(n)$ and $y_1[n]$ both will face 1st order high-pass NTF $(1 - z^{-1/f_s})$. Here, the transfer function from $u(t)$ to $v[n]$ can be referred to as STF, whereas the transfer function from $y_1[n]$ or $e(n)$ to $v[n]$ can be called NTF. The resultant NTF and STF will be as follows [21] [23]:

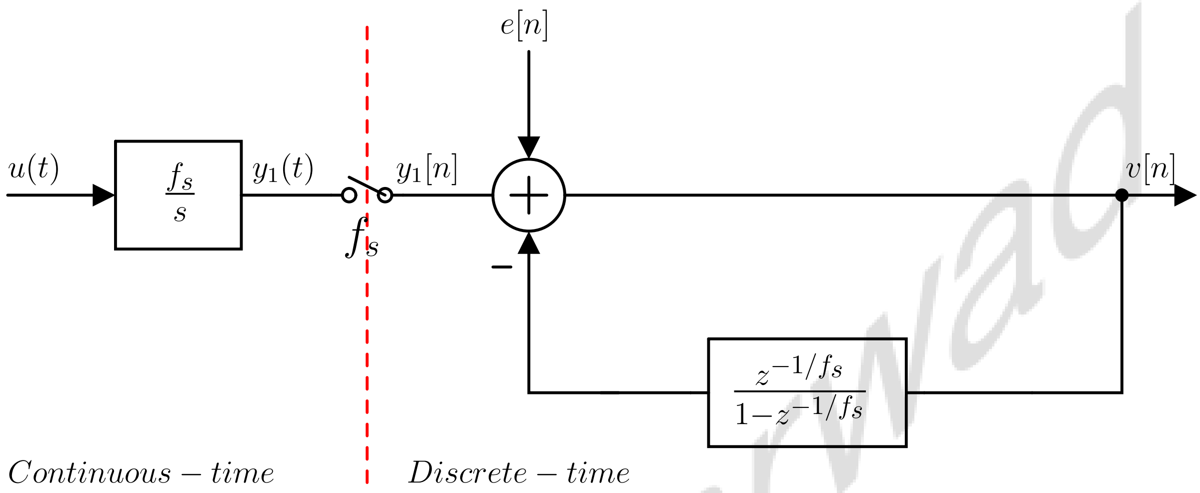

Figure 3.14: CTDSM simplified into continuous-time and discrete-time to visualize its STF and NTF.

$$NTF(z) = \frac{V(z)}{E(z)} = \frac{1}{1 - \frac{z^{\frac{-1}{f_s}}}{1-z^{\frac{-1}{f_s}}}} = \left(1 - z^{\frac{-1}{f_s}}\right) \tag{3.6}$$

$$\mathrm{STF}(f) = \underbrace{\frac{f_s}{j2\pi f}}_{\text{loop-filter}} \underbrace{\left(1 - e^{-j2\pi\frac{f}{f_s}}\right)}_{NTF} = e^{-j\pi\frac{f}{f_s}} \sin c(\frac{f}{f_s}) \tag{3.7}$$

In Figure 3.15, the magnitude response of the loop filter along with NTF & STF is plotted together for $f_s = 100MHz$. In the modulator's output spectrum (up to $f_s/2 = 50MHz$), we can notice that the quantization noise is shaped out with the gain of 2 at $f_s/2$. STF has unity gain near DC with first-order roll-off at high frequency. The CTDSM shows excellent alias rejection, as the loop filter response highly suppresses the possible alias band around the multiple of f_s.

3.2 Non-Idealities in CTDSM Design

This section discusses multiple non-idealities in CTDSM design that can be leveraged to introduce noise from multiple uncorrelated sources into the delta-sigma loop. By incorporating

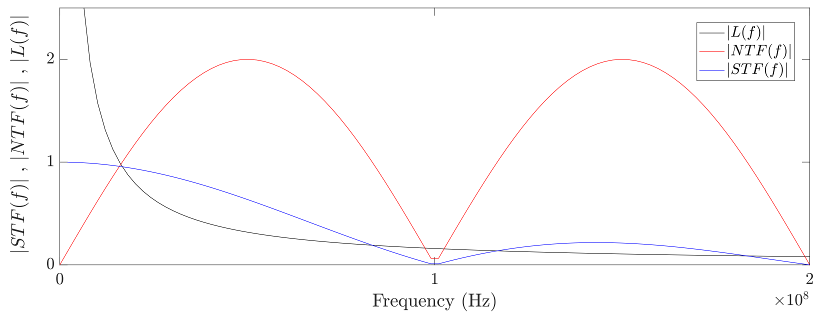

Figure 3.15: Magnitude response of Loop-filter, NTF, and the resulting STF of MOD1 CTDSM.

such noise sources, the aim is to enhance the noise variance in the loop, which is to be used for TRNG design. We will examine the impact of clock jitter and the comparator's metastability on the noise floor of the delta-sigma modulator. Additionally, we will qualitatively analyze noise modeling and the equivalent error sequences injected into the loop as a result of it. We will also explore how the utilization of RZ DAC compromises the natural alias rejection of the delta-sigma modulator. This non-ideality can be utilized to fold back additional high-frequency noise into the band of interest, consequently increasing the noise variance within the loop.

3.2.1 Clock-jitter

Ideally, the clock transitions should happen at integer multiples of the time period. In the actual clock, the transition deviates from the ideal edges due to random phase modulation of the ideal clock signal because of noise present in the circuit. These timing errors are denoted by the sequence $\Delta t[n]$ and are called jitter. In Figure 3.16, $\Delta t[1]$, $\Delta t[2], ..., \Delta t[5]$ are assumed to be uncorrelated, Gaussian distributed, and white spectrum having $\sigma_{\Delta t}$ as RMS value.

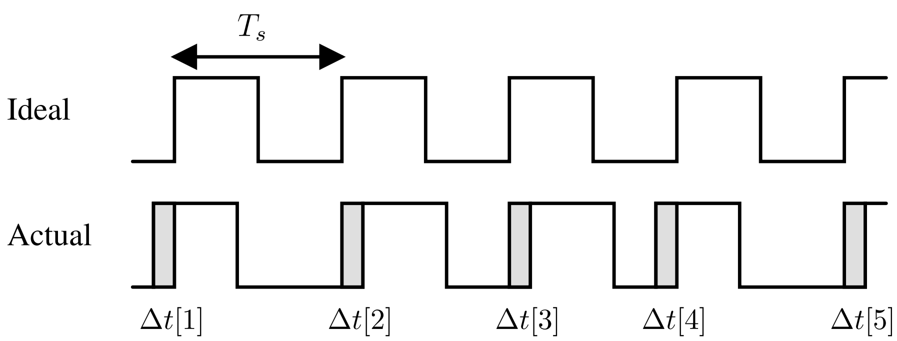

Figure 3.16: White uncorrelated error sequence due to clock jitter.

The noise floor of the CTDSM is directly affected by jitter in clk_{adc} and clk_{dac}, respectively. The error induced by jitter in clk_{adc} can be represented by an ADC operating with a clean clock, where an additive sampling error sequence $e_{adc}[n]$ is injected into the loop at the ADC's input. Similarly, the error induced by jitter in clk_{dac} can be modeled by the DAC operating with a jitter-free clock, with an induced additive error sequence at the DAC's input given by $e_{dac}[n]$, as shown in Figure 3.17.

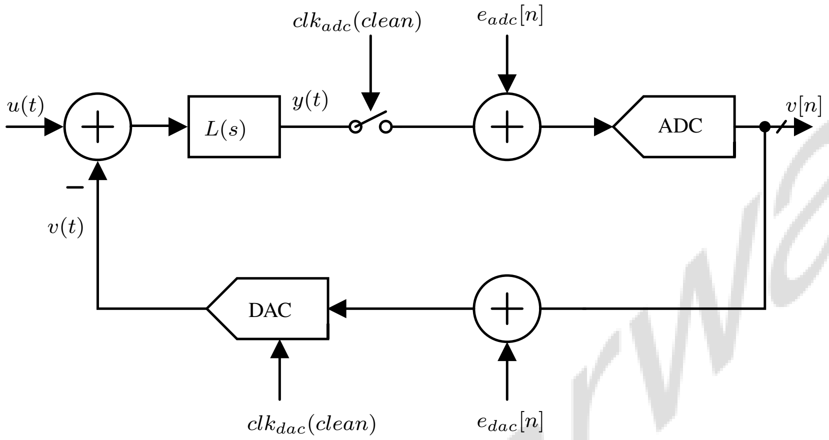

Figure 3.17: Clock jitter induced error model in CTDSM [21].

$e_{adc}[n]$ is modeled as the first-order approximation of the difference between $y(t)$ sampled through an ideal clock and a jittery clk_{adc}, as given in Equation 3.8. The sequence $\Delta t[n]$ represents the timing errors, assumed to be uncorrelated, Gaussian-distributed, and with a white spectrum characterized by an RMS value of $\sigma_{\Delta t}$. $e_{adc}[n]$ is referred to the input of the ADC, similar to additive quantization noise, and it exhibits a Gaussian distribution and white PSD analogous to $\Delta t[n]$. The effect of $e_{adc}[n]$ on the noise floor of the delta-sigma modulator appears at high frequencies since it undergoes high-pass noise shaping due to NTF [21].

$$e_{adc}[n] = \Delta t[n] \cdot \left.\frac{dy}{dt}\right|_{nT_s} \tag{3.8}$$

Random jitter in clk_{dac} causes timing error in the DAC output signal transitions, which induces an additive error sequence. It can be modeled as $e_{dac}[n]$, which is effectively referred to the input of the DAC (output of the CTDSM). $e_{dac}[n]$ is also a Gaussian-distributed error sequence and exhibits a uniformly distributed (white) spectrum analogous to $\Delta t[n]$. Since it is referred to the output of the modulator, it affects the entire spectrum and raises the noise floor of CTDSM uniformly. Different DACs are associated with different pulse shapes like NRZ, RZ, Exponentially Decaying Return to Zero (EDRZ) etc. The error sequence $e_{dac}[n]$ varies across different

DAC pulses employed in CTDSM. As a result, the contribution of $e_{dac}[n]$ from different DACs varies, influencing the overall noise variance at the output differently. The equivalent error sequence resulting from random jitter in clk_{dac} for different DACs utilized in CTDSM is provided by Equations 3.9, 3.10, and 3.11, respectively [21] [24].

NRZ DAC:

$$e_{dac}[n] = (v[n] - v[n-1])\frac{\Delta t[n]}{T_s} \tag{3.9}$$

RZ DAC:

$$e_{dac}[n] = (2v[n])\frac{(\Delta t_{rise}[n] + \Delta t_{fall}[n])}{T_s} \tag{3.10}$$

SCR DAC:

$$e_{dac}[n] = \frac{v[n]}{R_d}\exp\left(\frac{-T_s}{2R_dC_d}\right)\left(\Delta t[n] - \Delta t\left[n + \frac{1}{2}\right]\right) \tag{3.11}$$

SCR DAC generates Exponentially Decaying Return to Zero pulses for which the equivalent error sequence model is given in Equation 3.11. Here, C_d & R_d is switched capacitor feedback capacitance and discharging resistance, T_s is the pulse width, and $v[n]$ is the input to the DAC, respectively. By comparing the error sequence due to clock jitter in different DACs, we can conclude that RZ DAC-based CTDSM is most prone to jitter noise, followed by NRZ DAC-based CTDSM. SCR DAC-based CTDSM is the least affected since the error reduces exponentially with the discharge time constant R_dC_d. The use of SCR DAC compromises the alias rejection of CTDSM, making the loop filter a Linear Periodic Time-Varying system [25]. Increasing the discharging time constant R_dC_d of SCR DAC pulses amplifies $e_{dac}[n]$, increasing its sensitivity to clock jitter noise (Refer to Equation 3.11). Therefore, we opt for a pulse generated through the weighted combination of RZ and SCR DAC, resulting in a high magnitude of jitter-induced error sequence and noise folding due to compromised alias rejection. This approach enriches the entropy of CTDSM by increasing the noise variance in the loop, making it suitable for TRNG design.

3.2.2 Comparator's Metastability

Figure 3.18 shows StrongArm latch used in MOD1 CTDSM design. The StrongArm latch is often used alongside the RS latch to implement the comparator (1-bit ADC). A practical ADC takes a finite time to resolve and has a delay (Δt_d) that depends on the magnitude of its inputs ($V_{IN}1$) and ($V_{IN}2$). In Equation 3.12, we can observe t_{delay} as the function of the magnitude of the comparator's input $|V_{IN}1 - V_{IN}2|$. Here, αV_{DD} indicates the output level the latch has to resolve to recognize the change in the logical state, and β is the gain from $(V_{IN}1, 2)$ to $(V_{X,Y})$ and τ is the time constant for resolution.

$$\Delta t_d \approx \tau \ln \left(\frac{\alpha V_{DD}}{\beta |V_{IN}1 - V_{IN}2|} \right) \ where, \ \tau = \frac{C_{X,Y}}{g_{m_{3,4}} \left(1 - C_{X,Y}/C_{P,Q}\right)} \tag{3.12}$$

Resolution of the latch is defined as the minimum possible difference $|V_{IN}1 - V_{IN}2|$ up to which it can be resolved to V_{DD} or ground. In case of $|V_{IN}1 - V_{IN}2| \leq$ resolution, the comparator enters into a metastable state, only to be resolved by virtue of random noise present at regenerative nodes $V_{X,Y}$.

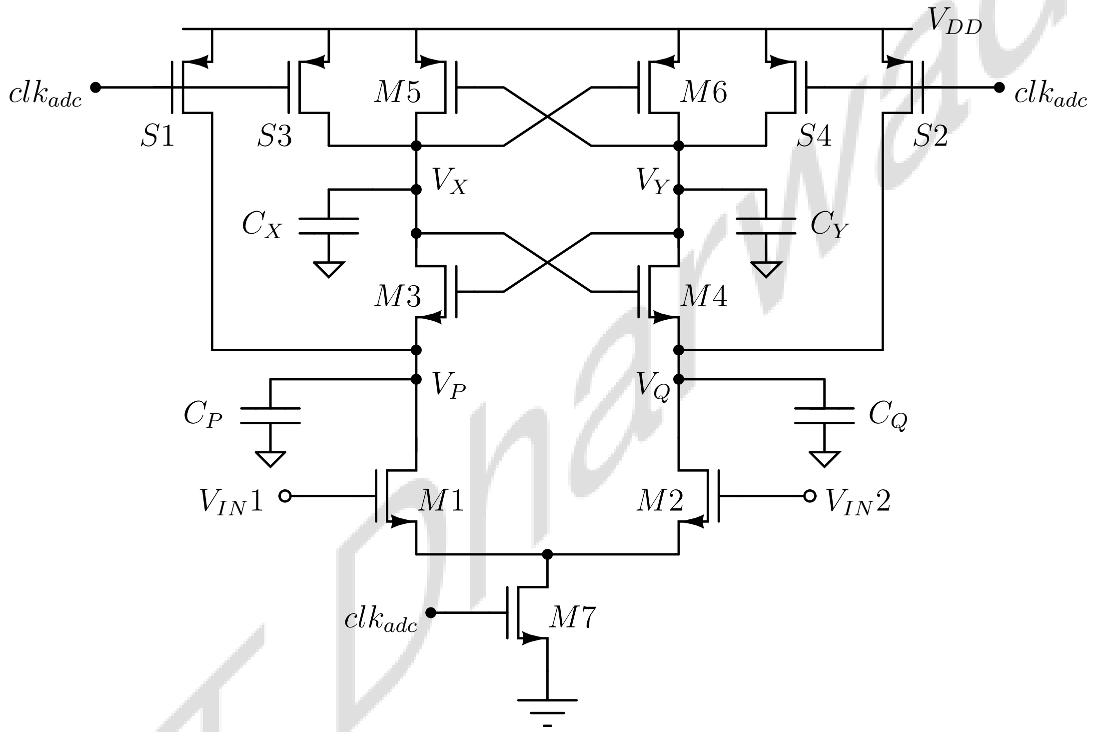

Figure 3.18: The StrongArm latch used in 1-bit ADC for MOD1 CTDSM.

Analysis of error due to metastability

In Figure 3.19, we can observe, in case of $|V_{IN}1| \approx |V_{IN}2|$, the regenerative nodes $V_{X,Y}$ fall very slowly and during regeneration phase, they get randomly resolved to the supply rails. Δt_d in case of metastability can be modeled as a Poisson random process. For 1-bit ADC design in the case of MOD1 CTDSM, the comparator's metastability manifests in two ways. Due to very high loop gain, the input of ADC has a Gaussian distribution with a mean at the threshold value of the latch. This results in the samples at ADC's input, causing the latch to be stuck at a metastable state with a high probability of causing random resolution, adding

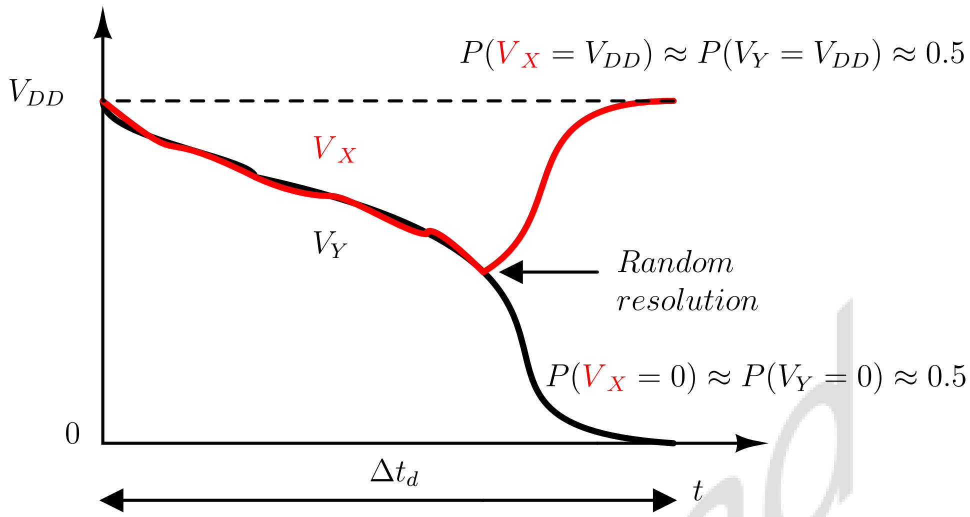

Figure 3.19: Latch stuck in metastability resolves randomly with random delays.

randomness to the output sequence. Also, the resolution time (Δt_d) is random, which modulates the phase of feedback pulses. This random pulse width modulation of feedback causes an error sequence $e_{meta}[n]$, similar to jitter in the clock of DAC (Refer to Figure 3.17). (Δt_d) is called signal-dependent delay, responsible for $e_{meta}[n]$ given by Equation 3.13. This error sequence is modeled exactly in the same way as $e_{dac}[n]$ and follows similar characteristics [21, 24, 26, 27].

$$e_{meta}[n] = \frac{\Delta t_d}{T_s}(v[n] - v[n-1]) \tag{3.13}$$

3.2.3 Alias Rejection in CTDSM

CTDSM eliminates the need for an Anti-alias filter, as evident from the STF given in Equation 3.10. Frequencies in the range of $[k \cdot f_s - f_b, k \cdot f_s + f_b]$ Hz are known as alias bands where f_s and f_b represent sampling frequency and maximum signal frequency, respectively. We know that STF is $L(j2\pi f) \cdot NTF$ in the case of CTDSM. We can define Alias Rejection (AR) as the ratio of the magnitude response of STF in the signal band (f_b) to the alias range $(k \cdot f_s \pm f_b)$. Let's pick an arbitrary frequency in the signal band, $0 < \Delta f < f_b$, and try to define and quantify alias rejection for it. Thus, Alias rejection $AR(k \cdot f_s + \Delta f)$ will tell us how strong the suppression is at the corresponding alias frequencies $(k \cdot f_s + \Delta f)$ by the virtue of the STF of CTDSM.

Since NTF at Δf and $(k \cdot f_s + \Delta f)$ is same (both rises as $2\pi\Delta f$), $AR(k \cdot f_s + \Delta f)$ can be equivalently written as,

$$\begin{aligned} AR\left(kf_s + \Delta f\right) &= \frac{STF(j2\pi\Delta f)}{\text{STF}\left(j2\pi\left(kf_s + \Delta f\right)\right)} \\ &= \frac{L(j2\pi\Delta f)}{L\left(j2\pi\left(kf_s + \Delta f\right)\right)} \end{aligned} \tag{3.14}$$

We can observe that $|AR\left(kf_s + \Delta f\right)|$ is large since $L(j2\pi\Delta f) \gg L\left(j2\pi\left(kf_s + \Delta f\right)\right)$. Also, STF($j2\pi\Delta f) \approx 1$, we can write:

$$\begin{aligned} |STF\left(j2\pi\left(kf_s + \Delta f\right)\right)| &= \left|\frac{L\left(j2\pi\left(kf_s + \Delta f\right)\right)}{L(j2\pi\Delta f)}\right| \\ &= \left|\frac{\Delta f}{kf_s + \Delta f}\right| \end{aligned} \tag{3.15}$$

Since $\Delta f << f_s$ and Δf is of order $\frac{f_s}{OSR}$, it leads to the very strong rejection of all the potential alias frequencies for Δf, i.e., $(kf_s + \Delta f)$. Topologies such as switched capacitor resistors are used to implement RZ DAC, which has a two-phase operation. The strength of inherent alias rejection of CTDSM those designed to use these DACs gets compromised. The two-phase switching operation of such DACs makes the loop filter Linear and periodic time-varying (LPTV). We can write $STF(f) = L(j2\pi f) \cdot NTF(e^{j2\pi f})$ only in the case of loop filter being time-invariant. If we consider a delta-sigma modulator, using practical G_{ota} to implement the loop filter, having C_d as equivalent switched capacitor in feedback RZ DAC, by applying the principle discussed in [28], an equivalent time-invariant loop filter transfer function $L_{eq,ct}(s)$ is derived, and it is given as,

$$L_{eq}(j2\pi f) \approx L(j2\pi f) - \underbrace{\frac{f_s C_d}{G_{\text{ota}}} \sum_k L\left(j2\pi\left(f - kf_s\right)\right)}_{E(j2\pi f)}. \tag{3.16}$$

We can observe from equation 3.18 that periodically varying $E(j2\pi f)$ is degrades the loop filter $L(j2\pi f)$ near the alias band. Since the $L_{eq}(j2\pi f)$ represents the equivalent time-invariant transfer function of the loop filter, we can write STF in this case as $STF(f) = L_{eq}(j2\pi f) \cdot NTF(e^{j2\pi f})$. Again, to quantify alias rejection, we will evaluate the new STF at $(kf_s + \Delta f)$. This results in:

$$L_{eq}(j2\pi f) \approx (f_s C_d / G_{ota}) L_{\text{ideal}}(j2\pi\Delta f) \tag{3.17}$$

$$NTF(e^{j2\pi(f+\Delta f)}) = NTF(e^{j2\pi\Delta f}) \approx 1/L_{ideal}(j2\pi\Delta f) \tag{3.18}$$

$$|STF(j2\pi(kf_s + \Delta f))| \approx \frac{f_s C_d}{G_{ota}} \tag{3.19}$$

For a given sampling frequency f_s, C_d and C_i is fixed to ensure first-order noise shaping. Hence, STF can be seen degraded by practical G_{ota}. This is considerably higher than $|(\Delta f)/(kf_s+\Delta f)|$ that we got for CTDSM with NRZ DAC. This leads to poor alias rejection for the modulators designed with RZ DAC topology [21] [25].

3.3 Noise analysis of CTDSM

Our aim for TRNG design using CTDSM is to enhance the noise variance within the loop by simultaneously incorporating multiple uncorrelated noise sources. In order to leverage the randomness from all sources of entropy, the design must ensure that the variance of each noise source is similar. In order to analyze this, we first model and characterize each of these noise sources and analyze their impact on the CTDSM noise floor.

In this section, we will examine various noise models of CTDSM and utilize these models and transfer functions to estimate the noise variance contribution from each source. Additionally, we will provide theoretical justification for the design choices aimed at ensuring similar noise contribution from each noise source onto the overall noise signal variance within CTDSM. We will use the simplest single-bit, 1st-order (MOD1) CTDSM as a reference for all analyses.

3.3.1 Thermal Noise

Thermal noise introduced at the modulator's input directly affects the entire spectrum of the modulator, resulting in an overall rise in the noise floor of CTDSM. It can be introduced via a resistor R at the modulator's input and characterized by its power spectral densities given by $S_{v_ni} = (4KTR)\ V^2/Hz$. The thermal noise, which possesses a white spectrum, directly elevates the noise floor of the CTDSM across all frequencies. This occurs because the STF of the CTDSM, denoted by $STF(f) = \frac{f_s}{j2\pi f}\left(1 - e^{-j2\pi \frac{f}{f_s}}\right)$, typically demonstrates nearly unity gain and attenuates with 20 dB per decade within the modulator's bandwidth across the range of 0 to $f_s/2$.

Figure 3.20 illustrates the thermal noise spectrum appearing at the output of the MOD1 CTDSM, where we can observe it to be low-pass shaped. To estimate the maximum thermal noise variance that could pass through the MOD1 CTDSM and raise its noise floor, the first question to address is the maximum value of resistance that we can utilize to generate thermal noise at the modulator's input. We imposed a 1 GHz bandwidth limitation during the transient noise simulation in cadence virtuoso to speed up the simulation to tolerable time limits.

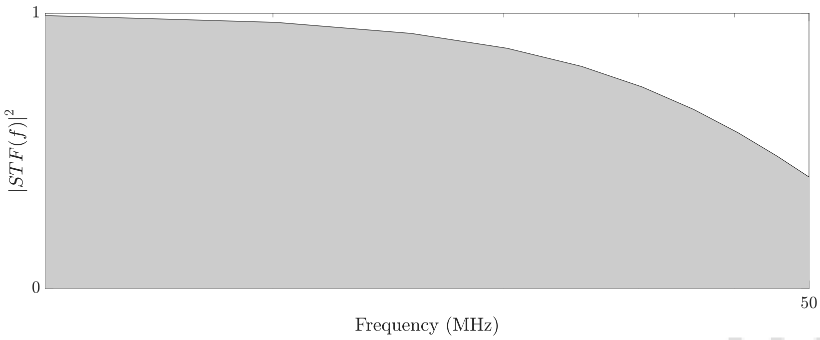

Figure 3.20: PSD of thermal noise appearing at the output of the modulator.

Since MOD1 CTDSM design has to be done in TSMC 65nm CMOS process with VDD of 1.2 volts, the maximum swing it can see would be $\frac{VDD}{2}$, i.e., 600 mV. For $R = 2.5\ G\Omega$ with with $S_{v_ni} = (4KTR)\ V^2/Hz$ band-limited to 1 GHz, the σ_{v_ni} comes out to be 200 mV. From here, we can get $3\sigma_{v_ni}$ to be 600 mV. The maximum stable amplitude limitation and the effect of dead zones due to the least probable spikes in thermal noise voltage swings are neglected here. This is done because we are only trying to estimate the maximum variance of thermal noise that can be passed through STF to the output of the modulator. The thermal noise variance low-pass shaped by the STF appearing at the output of MOD1 CTDSM is given by $\sigma^2_{v_no}$ in Equation 3.20.

$$
\begin{aligned}
S_{v_ni} =& 4kTR \quad V^2/Hz \\
\sigma^2_{v_no} =& \int_0^{\frac{f_s}{2}} S_{v_ni} \cdot |(\mathrm{STF}(f))|^2 \, df \quad V^2 \\
=& 4KTR \cdot \int_0^{\frac{f_s}{2}} \left| \frac{1}{j2\pi\frac{f}{f_s}} \left(1 - e^{-j2\pi\frac{f}{f_s}}\right) \right|^2 df \quad V^2 \\
S_{v_ni} =& 4KTR \cdot \int_0^{\frac{f_s}{2}} \left(\frac{2 - 2\cos\left(2\pi\frac{f}{f_s}\right)}{\left(\frac{2\pi}{f_s}\right)^2 \cdot f^2} \right) df \quad V^2 \\
=& 4KTR \cdot \left(3.8685 \times 10^7\right) \quad V^2 \\
=& \left(40 \times 10^{-12}\right) \cdot \left(3.8685 \times 10^7\right) \quad V^2 \\
=& 1.5474 \times 10^{-3} \quad V^2
\end{aligned}
\tag{3.20}
$$

Taking Account of Aliases

The input of MOD1 CTDSM is wideband noise up to 1 GHz. Since it gets sampled in the path from the input of the STF to the output, the alias will be folded back from $(K \cdot f_s \pm f_s/2)$ to $(0\ ,\ f_s/2)$. Let's consider an ideal scenario in which sampling is assumed to be done by an ideal periodic impulse train with time period T_s. In that case, we can assume all the aliases will fold back as-it-is with unity gain. With that assumption, we have estimated the equivalent thermal noise variance in $(0,\ f_s/2)$. In Figure 3.21, we can notice that the thermal noise components are highly suppressed at higher frequencies.

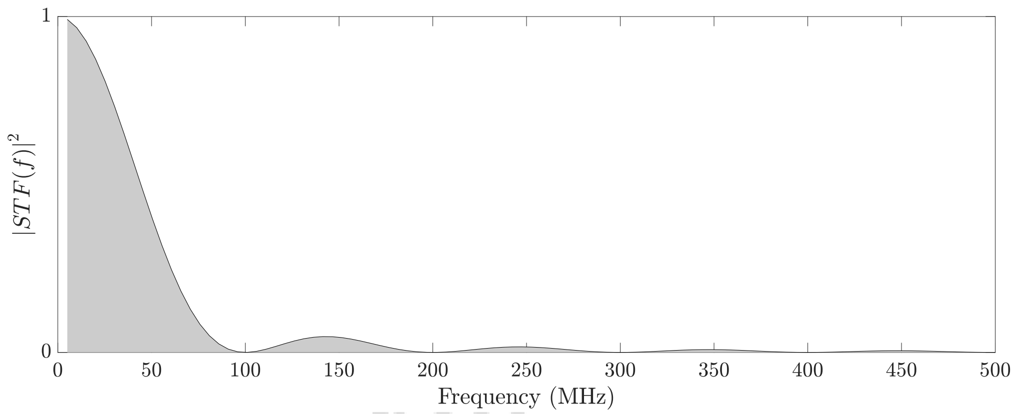

Figure 3.21: Input-referred thermal noise and aliases at higher frequencies.

Here, the loop filter is doing the band limitation of wideband thermal noise, and its variance becomes almost zero after 4-5 alias bands. In the Table 3.1, we can observe that noise folding after $(5 \cdot f_s \pm f_s/2)$ almost becomes negligible, and the overall variance of thermal noise from 0 to $f_S/2$ saturates to $1.94\ mV^2$ with about 25% increase.

3.3.2 Quantization Noise

There are a few assumptions regarding quantizer error sequence, which holds fairly for analytical purposes. It is assumed to be *additive*, *white*, *uniformly distributed*, and *independent of input* for a busy input signal. Using these assumptions, variance and single-sided power per bandwidth of quantization noise can be written as $\sigma^2_{q_n i} = \left(\frac{\Delta^2}{12}\right) V^2$ and $S_{q_n i} = \left(\frac{\Delta^2}{12} \cdot \frac{2}{f_s}\right) V^2/Hz$ respectively, where Δ is step-size of a given quantizer. The delta-sigma modulator has a white quantization noise spectrum before noise shaping (refer Figure 3.22), which is modeled to be added at the input of the ADC within the loop. The input Quantization noise spectrum is shaped

$K \cdot f_s \pm f_s/2$	$\sigma^2_{vnalias}$ at $K \cdot f_s \pm f_s/2$	σ^2_{vno}
$100MHz \pm 50MHz$	$3.14 \times 10^{-4}\, V^2$	$1.85 \times 10^{-3}\, V^2$
$200MHz \pm 50MHz$	$5.60 \times 10^{-5}\, V^2$	$1.90 \times 10^{-3}\, V^2$
$300MHz \pm 50MHz$	$2.35 \times 10^{-5}\, V^2$	$1.92 \times 10^{-3}\, V^2$
$400MHz \pm 50MHz$	$1.29 \times 10^{-5}\, V^2$	$1.93 \times 10^{-3}\, V^2$
$500MHz \pm 50MHz$	$8.20 \times 10^{-6}\, V^2$	$1.93 \times 10^{-3}\, V^2$
$600MHz \pm 50MHz$	$5.68 \times 10^{-6}\, V^2$	$1.94 \times 10^{-3}\, V^2$
$700MHz \pm 50MHz$	.	$1.94 \times 10^{-3}\, V^2$
$800MHz \pm 50MHz$	.	$1.94 \times 10^{-3}\, V^2$
$900MHz \pm 50MHz$	.	$1.94 \times 10^{-3}\, V^2$
$1000MHz \pm 50MHz$	.	$1.94 \times 10^{-3}\, V^2$

Table 3.1: Thermal noise variance in $(0\ ,\ f_s/2)$ considering noise folding due to aliasing.

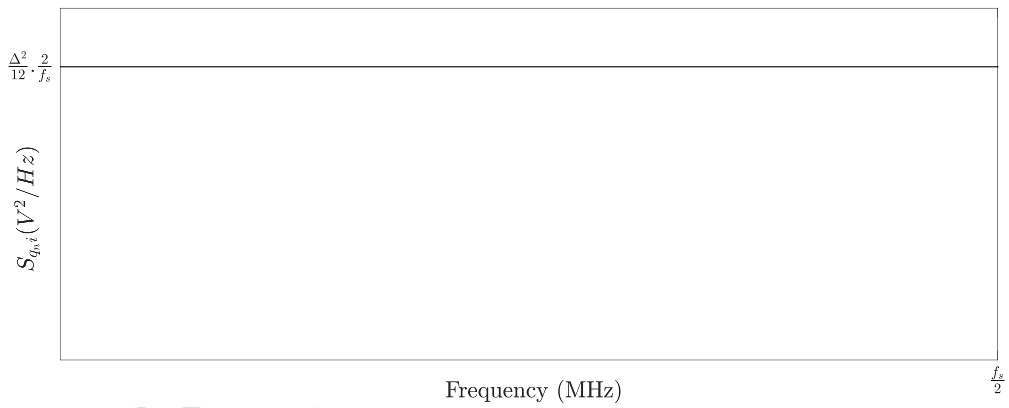

Figure 3.22: Single-sided PSD of quantization noise, modeled to be added at the input of the ADC within the loop.

by first-order high-pass NTF $\left(1 - e^{-j2\pi \frac{f}{f_s}}\right)$ as shown in Figure 3.23. It's apparent that the quantization noise spectrum undergoes shaping due to this NTF, resulting in a frequency-dependent characteristic. Thus, it tends to contribute more towards the high-frequency components in the overall noise profile.

The variance of shaped quantization noise for MOD1 CTDSM is given by σ^2_{qno} in Equation 3.21. We can notice an increase in the overall variance of shaped quantization noise due to the gain NTF possesses at high frequencies. Since we are dealing with a single-bit quantizer in

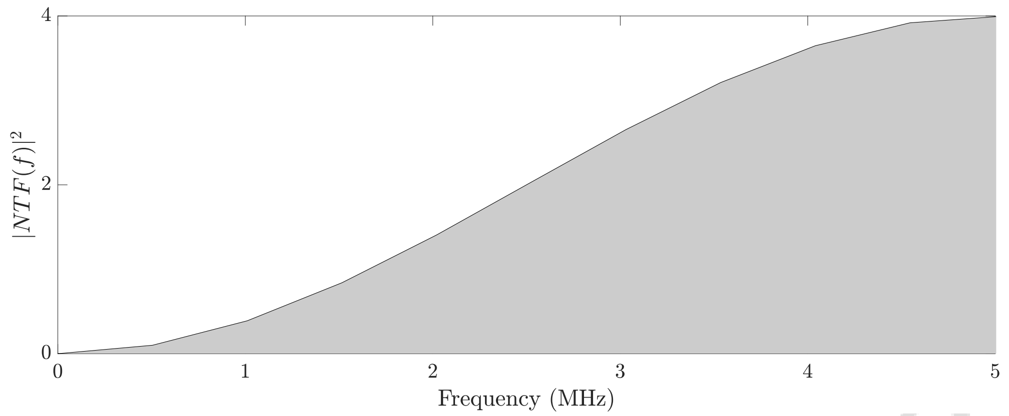

Figure 3.23: Shaped quantization noise PSD.

1st order Delta-Sigma loop, the step-size $\Delta = \frac{VDD}{2}$. We have VDD = 1.2 volts in TSMC65 technology, thus $\sigma^2_{q_n o} = 60\ mV^2$.

$$\begin{aligned}
S_{q_n i} =& \frac{\Delta^2}{12} \cdot \left(\frac{2}{f_s}\right) \quad V^2/Hz \\
\sigma^2_{q_n o} =& \int_0^{\frac{f_s}{2}} S_{q_n i} \cdot |(\mathrm{NTF}(f))|^2\, df \quad V^2 \\
=& \frac{\Delta^2}{12} \cdot \frac{2}{f_s} \int_0^{\frac{f_s}{2}} \left|\left(1 - e^{-j2\pi \frac{f}{f_s}}\right)\right|^2 df \quad V^2 \\
=& \frac{\Delta^2}{12} \cdot \frac{2}{f_s} \int_0^{\frac{f_s}{2}} 2 \cdot \left(1 - \cos\left(2\pi \frac{f}{f_s}\right)\right) df \quad V^2 \\
=& 2 \cdot \left(\frac{\Delta^2}{12}\right) \quad V^2 \\
=& 60 \times 10^{-3} \quad V^2
\end{aligned} \tag{3.21}$$

3.3.3 Phase Noise (clk_{adc})

The ADC, sampling with a jittery clock, can be modeled as one working with a jitter-free clock, but with an error $e_{adc}[n]$ added at its input, as shown in Figure 3.24.

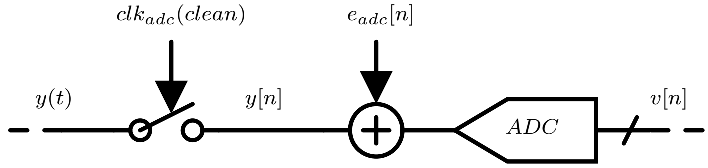

Figure 3.24: $e_{adc}[n]$ is error induced due to jitter in clk_{adc}, which is modeled to be added at the input of the ADC within CTDSM.

The additive error sequence $e_{adc}[n]$ can be written as a difference of $y(t)$ sampled through an ideal and jittery clk_{adc}. This gives a first-order approximation of $e_{adc}[n]$ given by Equation 3.22 [21]. $\Delta t[n]$ is a white sequence with an rms value $\sigma_{\Delta t}$. The next step should be to model $y(t)$ in terms of known parameters of MOD1 Delta-Sigma modulator. $e_{adc}[n]$ can be modeled in terms of the input signal $y(t)$ and timing error $\Delta t[n]$ in clk_{adc}.

$$\begin{aligned}
&y(t) \rightarrow y\left(nT_s\right) \{\text{Ideal } clk_{adc}\} \\
&y(t) \rightarrow y\left(nT_s+\Delta t[n]\right) \approx y\left(nT_s\right)+\Delta t[n] \cdot \left.\tfrac{dy}{dt}\right|_{nT_s} \{\text{Jittery } clk_{adc}\} \\
&e_{adc}[n]=y\left(nT_s+\Delta t[n]\right)-y\left(nT_s\right)=\Delta t[n] \cdot \left.\tfrac{dy}{dt}\right|_{nT_s}
\end{aligned} \tag{3.22}$$

For a given signal $y(t)$, input to an ADC, getting sampled with clk_{adc} with the period $(nT_s + \Delta t[n])$, the additional error sequence on top of the ideal samples of $y(t)$ is modeled as shown in the Figure 3.25.

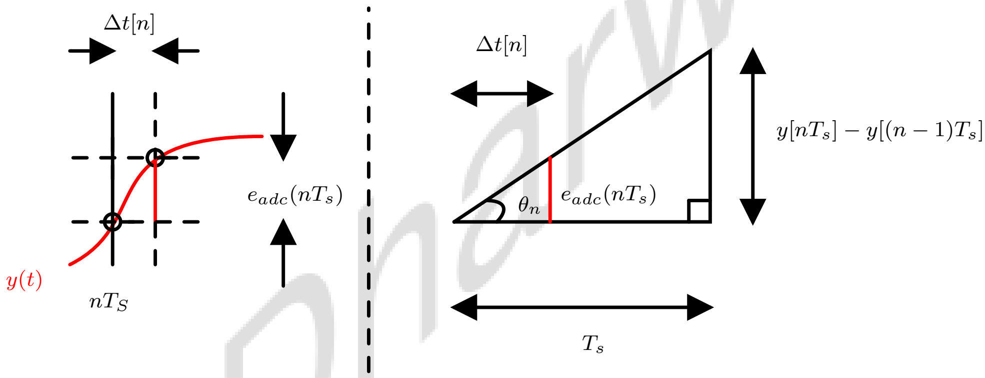

Figure 3.25: e_{adc} modeled as the slope of sampled signal at ideal instances multiplied with random timing error.

$e_{adc}[n]$ can be written as the product of the slope of $y(t)$ at nT_s with the random timing error $\Delta t[n]$ given in equation 3.23. Since $\Delta t[n]$ is a zero-mean Gaussian random process, $e_{adc}[n]$ is also white and has a variance given by $\sigma^2_{e_{adc}i}$. The RMS value of timing error, $\sigma_{\Delta t}$ in the clk_{adc}, was measured to be $2ns$ for $T_s = 10ns$. This results in $\sigma^2_{e_{adc}i}$ to be $\frac{1}{25}$th of σ^2_{dv}, where σ^2_{dv} is mean-square value of $y[n] - y[n-1]$.

$$\begin{aligned}
&e_{adc}[n]=\Delta t[n] \cdot tan\left(\theta_n\right)=\Delta t[n] \cdot \tfrac{y[n]-y[n-1]}{T_s} \\
&\sigma_{e_{adc}i}^2=\sigma_{\Delta t}^2 \cdot\left(\tfrac{\sigma_{dv}}{T_s}\right)^2=\tfrac{\sigma_{dv}^2}{25} \quad V^2
\end{aligned} \tag{3.23}$$

For a CTDSM, the loop filter's output can be written as $Y(f) = U(f) \cdot STF(f) + E(f) \cdot (NTF(f) - 1)$. This can be used to model $y[n] - y[n-1]$ as, $u(t) = v_n i$ and $e[n] = q_n i$, passes through $STF(f)$ and $(NTF(f) - 1)$ followed by "first-order difference" transfer function, $(1-$

$e^{-j2\pi\frac{f}{f_s}}$) as shown in the Figure 3.26. Thus, we can write mean square value of $y[n] - y[n-1]$ as σ_{dv}^2 given in equation 3.24.

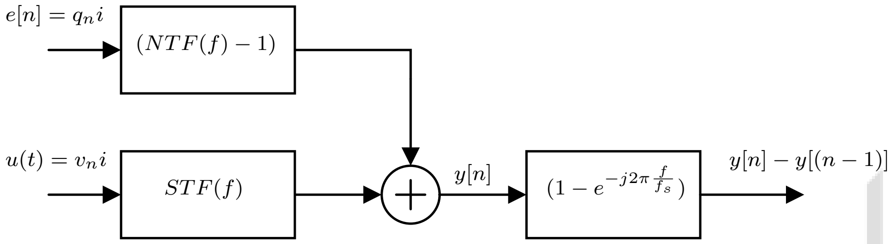

Figure 3.26: Modeling $y[n] - y[n-1]$ with known variables of MOD1 CTDSM.

$$\begin{aligned} \sigma_{dv}^2 = S_{v_n i} \cdot \int_0^{\frac{f_s}{2}} \left|(STF(f))\left(1 - e^{-j2\pi\frac{f}{f_s}}\right)\right|^2 df \\ + \ S_{q_n i} \int_0^{\frac{f_s}{2}} \left|(NTF(f) - 1)\left(1 - e^{-j2\pi\frac{f}{f_s}}\right)\right|^2 df \end{aligned} \tag{3.24}$$

Now that we know σ_{dv}^2, we have calculated the variance of noise due to $e_{adc}[n]$ as per Equation 3.23. Here, $\sigma_{e_{adc}i}^2$ is written as ($\sigma_{e_{adc}i}^2 = \sigma_{e_{adc}i1}^2 + \sigma_{e_{adc}i2}^2$). The noise variance $\sigma_{e_{adc}i}^2$ is determined by Equation 3.25.

$$\begin{aligned} \sigma_{e_{adc}i1}^2 &= \frac{1}{25} \cdot 4KTR \cdot \int_0^{\frac{f_s}{2}} \left|\frac{1}{j2\pi\frac{f}{f_s}}\left(1 - e^{-j2\pi\frac{f}{f_s}}\right)^2\right|^2 df \quad V^2 \\ &= \frac{1}{25} \cdot 4KTR \cdot \int_0^{\frac{f_s}{2}} \frac{f_s^2}{(2\pi f)^2}\left(6 - 8\cos(2\pi\frac{f}{f_s}) + 2\cos(4\pi\frac{f}{f_s})\right) df \quad V^2 \\ &= \frac{1}{25} \cdot (40 \times 10^{-12}) \cdot (6.44 \times 10^7) \\ &= 0.1 \times 10^{-3} \quad V^2 \end{aligned}$$

$$\begin{aligned} \sigma_{e_{adc}i2}^2 &= \frac{1}{25} \cdot \frac{\Delta^2}{12} \cdot \frac{2}{f_s} \int_0^{\frac{f_s}{2}} \left|\left(-e^{-j2\pi\frac{f}{f_s}}\right)\left(1 - e^{-j2\pi\frac{f}{f_s}}\right)\right|^2 df \quad V^2 \\ &= \frac{1}{25} \cdot \frac{\Delta^2}{12} \cdot \frac{2}{f_s} \int_0^{\frac{f_s}{2}} \left(2 - 2\cos(2\pi\frac{f}{f_s})\right) df \quad V^2 \\ &= \frac{1}{25} \cdot \frac{\Delta^2}{12} \cdot \frac{2}{f_s} \cdot f_s = 2.4 \times 10^{-3} \quad V^2 \end{aligned} \tag{3.25}$$

$$\begin{aligned} \sigma_{e_{adc}i}^2 &= \sigma_{e_{adc}i1}^2 + \sigma_{e_{adc}i2}^2 \\ &= 2.5 \times 10^{-3} \quad V^2 \end{aligned}$$

Given that $\Delta t[n]$ is a Gaussian-distributed white random process, $e_{adc}[n]$ also follows the same distribution. Since $e_{adc}[n]$ is also white, we can write its power spectral density as $S_{e_{adc}i}$ given in equation 3.26. $e_{adc}[n]$ gets added to the loop at the input of the ADC, so it sees NTF and gets high-pass shaped similar to quantization noise. From this, the noise variance due to $e_{adc}[n]$, which is denoted by $\sigma^2_{e_{adc}o}$ is calculated in Equation 3.26 which came out to be $\sigma^2_{e_{adc}o} = 5\,mV^2$.

$$
\begin{aligned}
S_{e_{adc}i} =& 2.5 \times 10^{-3} \cdot \left(\frac{2}{f_s}\right) \quad V^2/Hz \\
\sigma^2_{e_{adc}o} =& \int_0^{\frac{f_s}{2}} S_{e_{adc}i} \cdot |(\mathrm{NTF}(f))|^2\, df \quad V^2 \\
=& 2.5 \times 10^{-3} \cdot \frac{2}{f_s} \int_0^{\frac{f_s}{2}} \left|\left(1 - e^{-j2\pi \frac{f}{f_s}}\right)\right|^2 df \quad V^2 \\
=& 2.5 \times 10^{-3} \cdot \frac{2}{f_s} \int_0^{\frac{f_s}{2}} 2 \cdot \left(1 - \cos\left(2\pi \frac{f}{f_s}\right)\right) df \quad V^2 \\
=& 2.5 \times 10^{-3} \cdot \frac{2}{f_s} \cdot f_s = 5 \times 10^{-3} \quad V^2
\end{aligned}
\tag{3.26}
$$

3.3.4 Phase Noise (clk_{dac})

The noise floor of a CTDSM is sensitive to the jitter present in the clk_{dac} pulses. The jitter induces random phase modulation of feedback pulses, resulting in variation in the amount of charge fed back in each clock cycle. This variation induces $e_{dac}(t)$, which can be modeled as the additional noise added to the modulator at the output of a jitter-free DAC. We will examine the noise modeling in a CTDSM for jittery clk_{dac}, considering the DAC generating NRZ pulses for simplicity in the analysis. Subsequently, we will derive the modeling of $e_{dac}[n]$, as introduced in Equation 3.9. Although we have chosen to implement a DAC that generates a weighted combination of RZ and exponentially decaying pulses, this analysis will provide us with a fairly good estimate of the noise variance contribution due to jittery clk_{dac}. Additionally, prior work in [24] suggests that the jitter noise in the case of RZ DACs is typically 4 to 5 dB higher than that in NRZ DACs.

Let's consider the case of NRZ DAC with an input code $v[n]$ as shown in Figure 3.27. Due to the jitter in clk_{dac}, an additional error sequence $e_{dac}(t)$ gets introduced into the modulator at the output of a jitter-free DAC. $e_{dac}(t)$ can be modeled as the sum of slivers having width $\Delta t[n]$ and height $v[n] - v[n-1]$ occurring at nT_s.

$e_{dac}(t)$ can be further modeled as an error sequence $e_{dac}[n]$ referred to the input of the DAC (Figure 3.28), doing amplitude modulation to the incoming pulses. This can be interpreted as each impulse of the error sequence $e_{dac}(t)$ having area $\Delta t[n](v[n] - v[n-1])$ to be spread over the entire period T_s of the pulse.

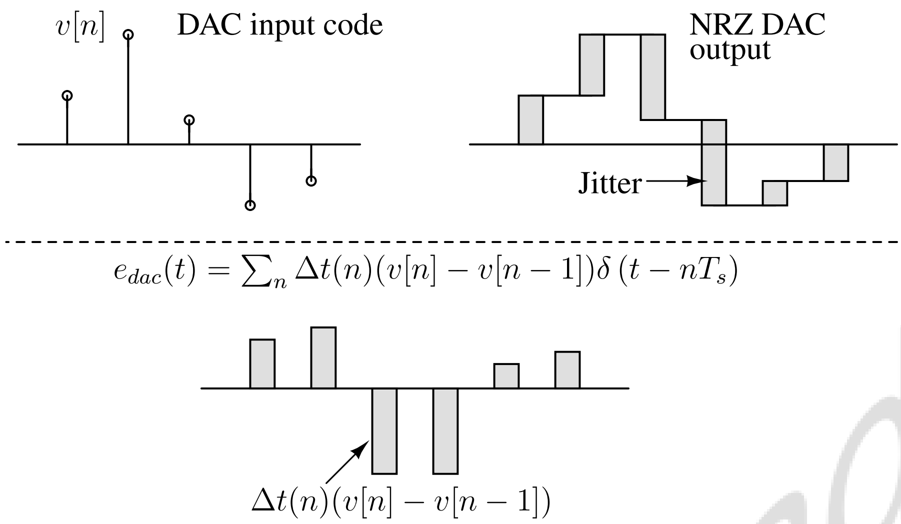

Figure 3.27: NRZ DAC input sequence, output waveform with clock jitter, and equivalent error waveform.

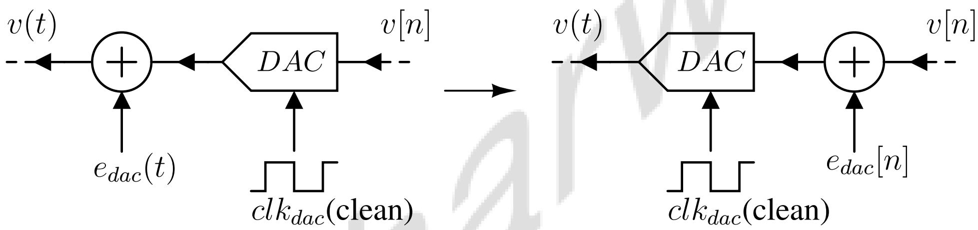

Figure 3.28: $e_{dac}(t)$ referred at the output of DAC, whereas $e_{dac}[n]$ referred at the input of DAC, i.e., the output of the modulator.

Mathematically, $e_{dac}(t)$ is equivalently represented as $e_{dac}[n]$ by band-limiting $e_{dac}(t)$ up to $f_s/2$ through a filter with an NRZ impulse response as shown in Figure 3.29. Thus, $e_{dac}[n]$ is modeled as an error sequence exhibiting NRZ pulse shape, referred to DAC's input.

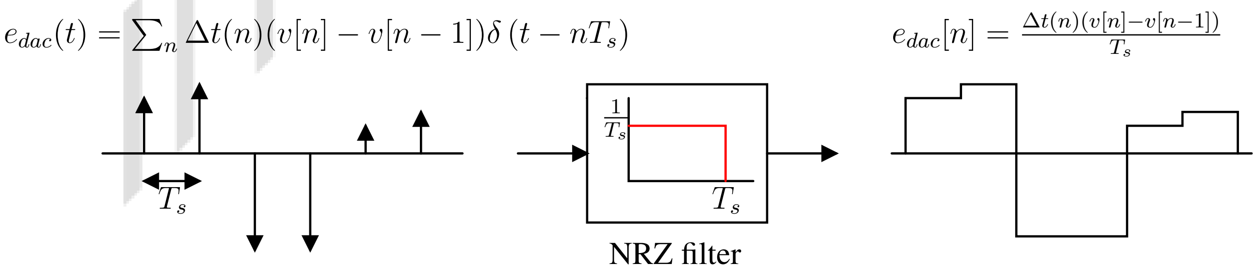

Figure 3.29: Low-frequency approximation of the error $e_{dac}(t)$ as NRZ pulses referred to the input of DAC.

We assume $\Delta t[n]$ to be zero mean, Gaussian distributed, white sequence. This results in $e_{dac}[n]$ also being a zero-mean Gaussian random process appearing directly along with output $v[n]$.

We want to quantify the total variance contribution of $e_{dac}[n]$ at the output of the Delta-Sigma modulator. Since it is a zero mean process, the mean squared value of $e_{dac}[n]$ will directly give the equivalent variance $\sigma^2_{e_{dac}}$ given in the equation 3.27. We have already estimated $\sigma_{\Delta t}$ to be $2ns$ for clk_{dac} with period $T_s = 10ns$. This results in $\sigma^2_{e_{dac}}$ to be $\frac{1}{25}$th of σ^2_{dv}. Here, σ^2_{dv} represents mean-square value of $(v[n] - v[n-1])$.

$$\begin{aligned} e_{dac}[n] &= \frac{\Delta t[n]}{T_s} \cdot (v[n] - v[n-1]) \\ \sigma^2_{e_{dac}} &= \left(\frac{\sigma_{\Delta t}}{T_s}\right)^2 \cdot {\sigma_{dv}}^2 = \frac{\sigma^2_{dv}}{25} \quad V^2 \end{aligned} \tag{3.27}$$

In CTDSM, we know that $V(f) = STF(f) \cdot U(f) + NTF(f) \cdot E(f)$. From this, we can model $v[n] - v[n-1]$ as $u(t) = v_n i$ and $e[n] = q_n i$, passes through $STF(f)$ and $NTF(f)$ followed by "first-order difference" transfer function, $(1 - e^{-j2\pi\frac{f}{f_s}})$ as shown in the Figure 3.30. Thus, we can write mean square value of $v[n] - v[n-1]$ as σ^2_{dv} given in Equation 3.28.

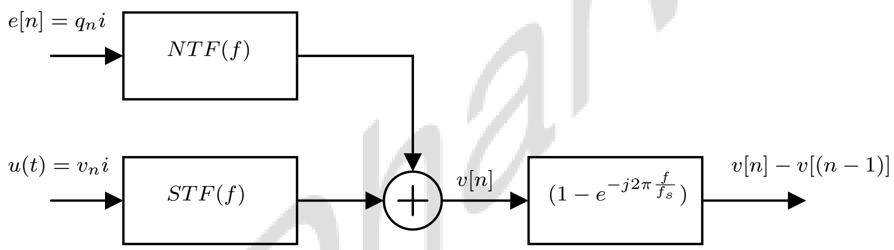

Figure 3.30: Modeling $v[n] - v[n-1]$ with known variables of MOD1 CTDSM.

$$\begin{aligned} \sigma^2_{dv} = S_{v_n i} \cdot \int_0^{\frac{f_s}{2}} \left|STF(f)\left(1 - e^{-j2\pi\frac{f}{f_s}}\right)\right|^2 df \\ + \; S_{q_n i} \int_0^{\frac{f_s}{2}} \left|NTF(f)\left(1 - e^{-j2\pi\frac{f}{f_s}}\right)\right|^2 df \end{aligned} \tag{3.28}$$

Now that we know σ^2_{dv}, we have calculated the variance of noise due to $e_{dac}[n]$ as per Equation 3.27. Here $\sigma^2_{e_{dac}}$ is written as $(\sigma^2_{e_{dac}} = \sigma^2_{e_{dac}1} + \sigma^2_{e_{dac}2})$. The noise variance $\sigma^2_{e_{dac}}$ is determined in Equation 3.29. The overall noise variance at the output of MOD1 CTDSM comes out to be 7.3 mV^2 distributed uniformly across $(0\ ,\ f_s/2)$.

This analysis also shows that jitter in clk_{dac} pulses generates noise at the output of the DAC only if $v[n] \neq v[n-1]$. Successive v[n] values can differ either due to quantization noise or due to thermal noise at the input, as shown in Figure 3.30. Also, it can be seen from the analysis given in Equation 3.29 that the contribution of quantization noise is almost 700 times the contribution of thermal noise for a 1-bit modulator.

$$
\begin{aligned}
\sigma^2_{e_{dac}1} &= \frac{1}{25} \cdot 4KTR \cdot \int_0^{\frac{f_s}{2}} \left| \frac{1}{j2\pi\frac{f}{f_s}} \left(1 - e^{-j2\pi\frac{f}{f_s}}\right)^2 \right|^2 df \\
&= \frac{1}{25} \cdot 4KTR \cdot \int_0^{\frac{f_s}{2}} \frac{f_s^2}{(2\pi f)^2} \left(6 - 8\cos(2\pi\frac{f}{f_s}) + 2\cos(4\pi\frac{f}{f_s})\right) df \\
&= (\frac{1}{25}) \cdot (40 \times 10^{-12}) \cdot (6.44 \times 10^7) = 0.1 \times 10^{-3} \quad V^2
\end{aligned}
$$

$$
\begin{aligned}
\sigma^2_{e_{dac}2} &= \frac{1}{25} \cdot \frac{\Delta^2}{12} \cdot \frac{2}{f_s} \int_0^{\frac{f_s}{2}} \left| \left(1 - e^{-j2\pi\frac{f}{f_s}}\right) \left(1 - e^{-j2\pi\frac{f}{f_s}}\right) \right|^2 df \\
&= \frac{1}{25} \cdot \frac{\Delta^2}{12} \cdot \frac{2}{f_s} \int_0^{\frac{f_s}{2}} \left(6 - 8\cos(2\pi\frac{f}{f_s}) + 2\cos(4\pi\frac{f}{f_s})\right) df \\
&= (\frac{1}{25}) \cdot (6 \times 10^{-10}) \cdot (3 \times 10^8) = 7.2 \times 10^{-3} \quad V^2
\end{aligned}
\tag{3.29}
$$

$$
\sigma^2_{e_{dac}} = \sigma^2_{e_{dac}1} + \sigma^2_{e_{dac}2} = 7.3 \times 10^{-3} \quad V^2
$$

3.3.5 Noise Due To Comparator's Metastability

We have discussed the origin and characteristics of noise-induced due to comparator in delta-sigma modulator. Metastability exit time (signal-dependent delay) Δt_d of the comparator is the random variable, which is modeled as a white random process similar to timing error sequence $\Delta t[n]$. Δt_d causes random phase modulation of feedback pulses, which manifest similarly to $e_{dac}[n]$. This results in an additional error sequence $e_{meta}[n]$ getting added into the loop as shown in Figure 3.31 and given by equation 3.30.

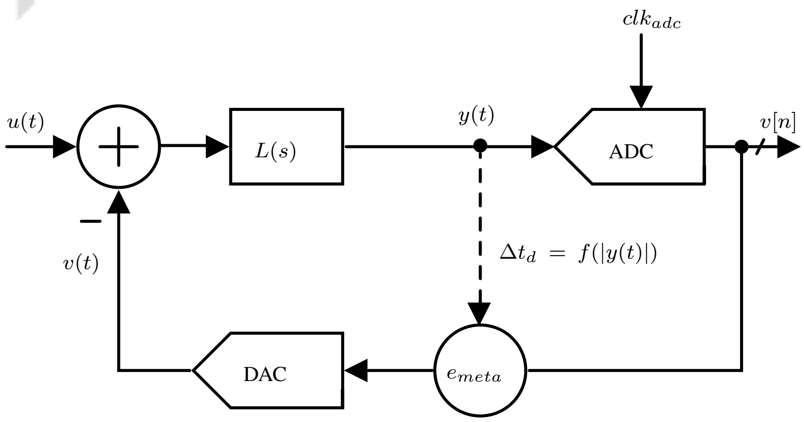

Figure 3.31: Metastability induced error model in CTDSM [21].

$$e_{meta}[n] = \frac{\Delta t_d}{T_s}(v[n] - v[n-1]) \tag{3.30}$$

It must be noted here that if the comparator is close to metastability, the clock-to-Q delays are affected. In such situations, the clock-to-Q delays will follow a Gaussian distribution with a mean at the nominal clock-to-Q value and a standard deviation that depends on the speed of the flip-flop. However, when the comparator actually enters metastability, the time to resolve can take any value, and a reasonable assumption is that it follows a Poisson distribution.

To find the variance of $e_{meta}[n]$ at the output of the modulator, $\sigma_{\Delta t_d}$ was measured by plotting eye diagrams of the comparator's output. The $\frac{1}{6}th$ spread of the eye diagram of $v[n]$ taken over a sufficiently long interval can assure $\sigma_{\Delta t_d}$ measurement accuracy. We measured $\sigma_{\Delta t_d}$ to be approximately equal to $400\ ps$. We followed exact-similar steps as given in Equations 3.27 to 3.29 to find out $\sigma^2_{e_{meta}}$ which came out to be $0.3\ mV^2$. Please note that this doesn't include noise variance due to the random resolution of the comparator when it actually enters metastability due to samples falling near its threshold value.

3.3.6 Comparison of the noise sources

We considered 1st-order 1-bit CTDSM with a sampling frequency of f_s to estimate the noise variance contribution due to each source. We observed that the noise spectrum, which arises from the quantization process within the modulator and is shaped by the 1st order high-pass $NTF(f) = \left(1 - e^{-j2\pi \frac{f}{f_s}}\right)$. On the other hand, Thermal noise introduced at the input stage directly affects the entire spectrum of the modulator, resulting in an overall rise in the noise floor. This is because the STF of the CTDSM, given by $STF(f) = \frac{f_s}{j2\pi f}\left(1 - e^{-j2\pi \frac{f}{f_s}}\right)$, typically exhibits high gain within the bandwidth of the loop filter and can be designed to maintain a positive gain between 0 to $f_s/2$. We have thermal noise and quantization noise characterized by their respective power spectral densities given by $S_{v_n i} = (4KTR)\ V^2/Hz$ and $S_{q_n i} = \left(\frac{\Delta^2}{12} \cdot \frac{2}{f_s}\right) V^2/Hz$. Additionally, the error sequences due to clock jitter and the metastability of the quantizer in CTDSM are modeled in the time domain given by $e_{adc}[n]$, $e_{dac}[n]$ and $e_{meta}[n]$ respectively. They also possess white spectral characteristics and impact the entire spectrum of CTDSM based on the transfer function they encounter [21, 24].

We used these noise models and transfer functions to estimate the noise variance contribution due to each source. Noise contribution from multiple sources in the case of the 1-bit and

Noise sources	**1st Order 1 Bit**	**1st Order 4 Bit**
Quantization Noise $(q_n i)$	$60\ mV^2$	$937.5\ \mu V^2$
Thermal Noise $(v_n i)$	$1.94\ mV^2$	$1940\ \mu V^2$
Phase Noise $(e_{adc}[n])$	$5\ mV^2$	$281\ \mu V^2$
Phase Noise $(e_{dac}[n])$	$7.3\ mV^2$	$215.5\ \mu V^2$
Metastability $(e_{meta}[n])$	$0.24\ mV^2$	$8.56\ \mu V^2$

Table 3.2: Noise variance of CTDSM using single-bit and 4-bit quantizer.

4-bit CTDSM are summarized in Table 3.2. For the 1st-order 1-bit modulator, we can observe that the overall noise power is dominated by quantization noise, and contribution from all other sources remains negligible. A multi-level quantizer, with its finer resolution, enables other uncorrelated noise sources, such as thermal noise from the resistor at the input, clock jitter, and random delays taken by the comparator to resolve, to make more substantial contributions to the output spectrum by reducing quantization noise variance, as evident from the Table 3.2.

Chapter 4

System Design of CTDSM Based TRNG

We intend to use CTDSM as an entropy extractor for TRNG design. By exploiting the negative feedback and non-idealities of CTDSM, noise from multiple uncorrelated sources can be introduced into the loop. For a single-bit CTDSM, the ADC's output pulses can be sampled to generate raw binary data for the TRNG output. In the case of multi-bit CTDSM, sampling through the LSB output provides an extra layer of filtering. The LSB is least affected by circuit offset and exhibits a whiter behavior than higher-order bitstreams. The LSB is more sensitive to random noise than higher-order bits since they represent the smallest changes in the input value. Higher-order bits, representing larger values, can mask these small fluctuations, making them less suitable for directly capturing randomness at a high rate. The probability of the LSB being 1 or 0 is determined by the probability density of the input signal at the threshold between the two smallest quantization levels. Circuit offsets typically shift the entire probability density function of the input signal, but they don't significantly alter its shape or distribution around that threshold. This means that the probability of the LSB is a 1 or 0 remains relatively unchanged in the presence of circuit offsets, making it a preferable choice to sample entropy from CTDSM.

It is also crucial to note that raw LSB samples alone may not meet stringent randomness criteria and could exhibit biases or patterns due to circuit imperfections or environmental factors. This can be attributed to practical loop filter speed limitations, such as input-referred offsets, finite loop gain, and bandwidth. To generate an unbiased random bit stream, independent and Independent and Identically Distributed (IID) samples of LSB from multiple instances of CTDSM can be XORed together, as shown in Figure 4.1. Since all the instances run independently, overall bias gets factored by the product of individual biases based on Piling-up approximation [20].

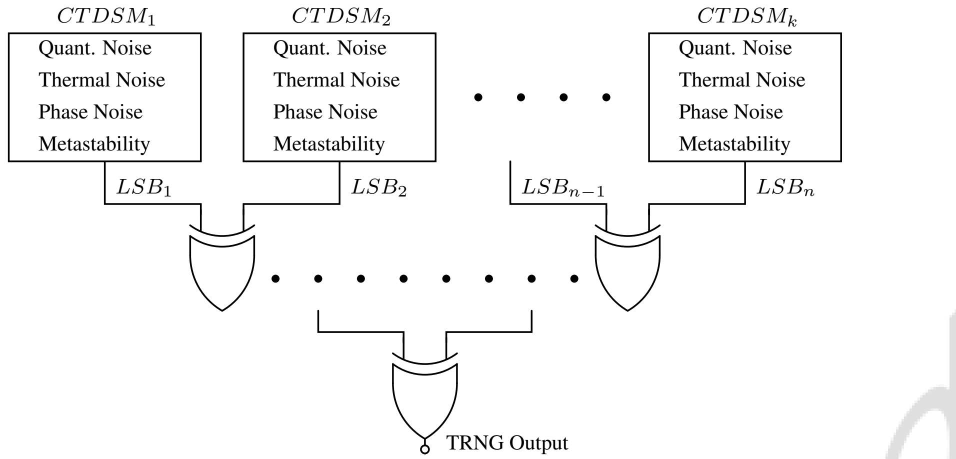

Figure 4.1: Independent and identically distributed samples of LSB from multiple instances of CTDSM are XORed together to generate unbiased TRNG output.

4.1 Single-bit CTDSM based TRNG

We started with the design of the simplest architecture, MOD1 CTDSM, for TRNG design. Figure 4.2 shows 1st order single-bit Switched Capacitor Return to zero DAC-based CTDSM (SC-CTDSM). The sampling rate of clk_{adc} and clk_{dac} are given as by $f_s = 1/T_s$. The integrator is of the OTA-RC type. ϕ_1 and ϕ_2 are non-overlapping phases of clk_{dac}.

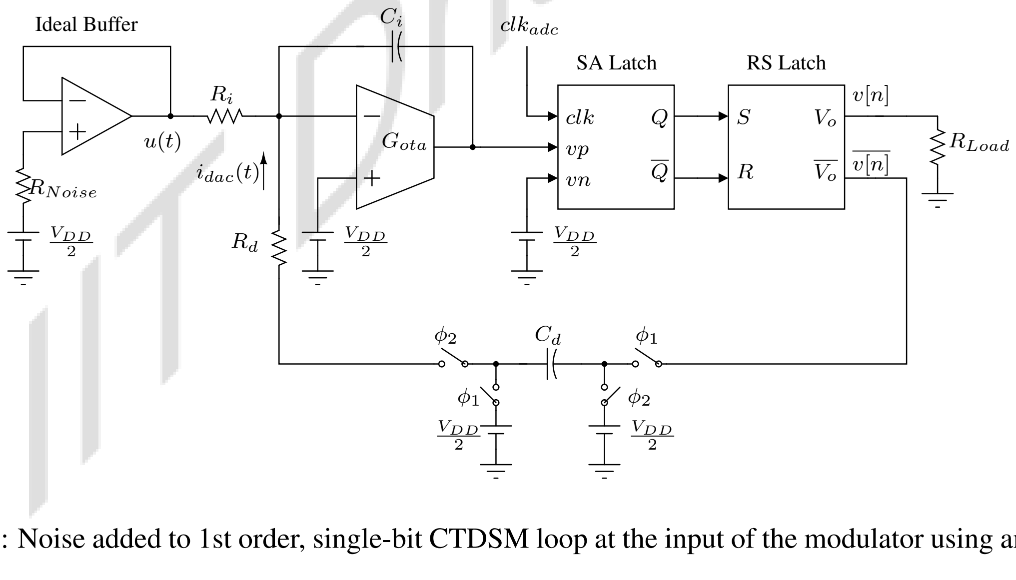

Figure 4.2: Noise added to 1st order, single-bit CTDSM loop at the input of the modulator using an ideal buffer, phase noise using $clkadc$, $clkdac$ and through metastability of practical comparator. 1-bit ADC output $v[n]$ used to sample raw binary random data for TRNG.

The goal was to observe the extent to which the overall noise variance could be elevated. We made certain design choices to maximize the effect of all the noise in the loop. SCRZ DAC was chosen because of its ability to compromise alias rejection of CTDSM (Refer to section

3.2.3). To increase the jitter sensitivity of SCRZ DAC due to random timing error in clk_{dac}, the discharging time constant R_dC_d of the feedback pulse was set up such that the pulse doesn't completely discharge to zero much before the next clock edge comes. The Comparator was slowed down to induce metastability by scaling up the transistor's length and load capacitance of the regenerative latch, increasing τ (refer to equation 3.14). A $2.5G\Omega$ resistor was added through an ideal buffer at the input of MOD1 CTDSM. As discussed in the section, we limited the transient noise simulation bandwidth up to 1 GHz, giving rise to the thermal noise having 6σ approximately equal full-scale input dynamic range of the modulator. Jittery clk_{adc} and clk_{dac} were generated separately through an ideal Voltage-Controlled Oscillator (VCO) injected with white noise from the resistor. The RMS value of the timing error $\sigma_{\Delta t}$ of clk_{adc} and clk_{dac} was measured, which came out approximately up to 20% of the clock period. The full design was done using TSMC 65nm ($V_{DD} = 1.2volts$) technology except for buffer using an ideal Voltage Controlled Voltage Source (VCVS) and Verilog-A model for VCO for clock generation.

4.1.1 Design of 1-bit CTDSM

In the Figure 4.2, Since the average current through the integrating capacitor C_i is zero, the average input current $\left(\frac{\overline{u(t)}}{R_i}\right)$ equals the average DAC current $\overline{i_{dac}}$. Thus, $C_d f_s = 1/R_i$ to achieve $STF(0) = 1$. That implies unity gain is achieved when the switched capacitor feedback resistor is made equal to the input resistor. Also, to ensure 1st-order noise shaping, C_d can be set equal to C_i, for which the theoretical background can be found in [23].

Figure 4.3 shows the AC response of the practical integrator designed to handle 100 MHz sampling frequency. It has a 3 dB and unity gain bandwidth of 440 KHz and 100 MHz, respectively, with a DC gain of 47.92 dB.

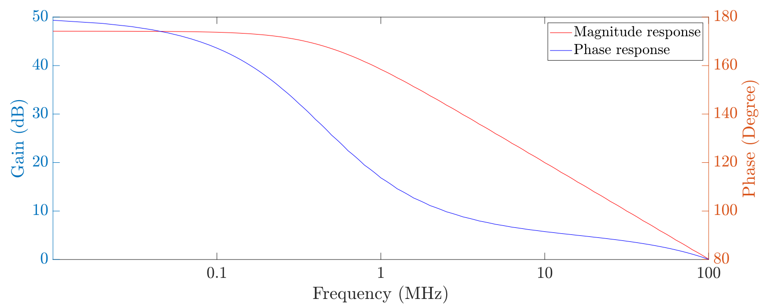

Figure 4.3: AC response of the loop filter (practical integrator) designed to handle sampling rate of $f_s = 100MHz$ to ensure 1st order noise shaping.

4.1.2 Testing 1-bit CTDSM based TRNG

Initially, we evaluated the MOD1 SC-CTDSM design by stimulating the loop with a full-scale sinusoidal signal at a frequency of 50 kHz. Figure 4.4 illustrates the FFT spectrum of the resulting sequence $v[n]$. A distinct peak at 50 kHz is evident, and quantization noise is shaped out beyond the $3dB$ bandwidth of the loop filter, confirming the effective 1st-order Delta-Sigma operation of the MOD1 SC-CTDSM.

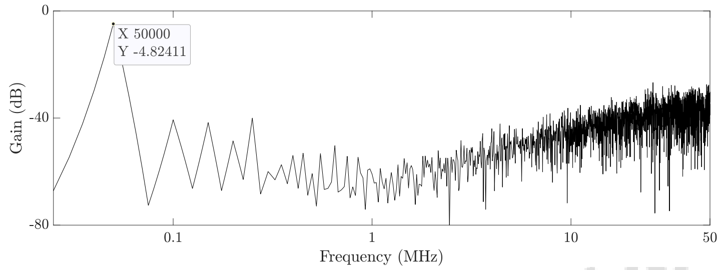

Figure 4.4: MOD1 CTDSM response to a single tone input at 50 KHz.

Following a test simulation with a single-tone sinusoidal signal, the MOD1 CTDSM underwent simulation considering all noise sources. The clock signals, clk_{adc} and clk_{dac}, were generated through two instances of an ideal VCO injected with white thermal noise. The RMS $\sigma_{\Delta t}$ jitter of the generated clock was measured to be $\approx$ 20% of T_s. Thermal noise was introduced at $u(t)$ using a $2.5G\Omega$ resistor using an ideal buffer. One million bits were sampled and XORed from the outputs of the four instances of MOD1 denoted as $v_k[n]$ (k = 1, 2, 3, 4). Figure 4.5 shows the FFT of a single instance output of MOD1 $v[n]$ alongside the result of $(v_1[n] \oplus v_2[n] \oplus v_3[n] \oplus v_4[n])$.

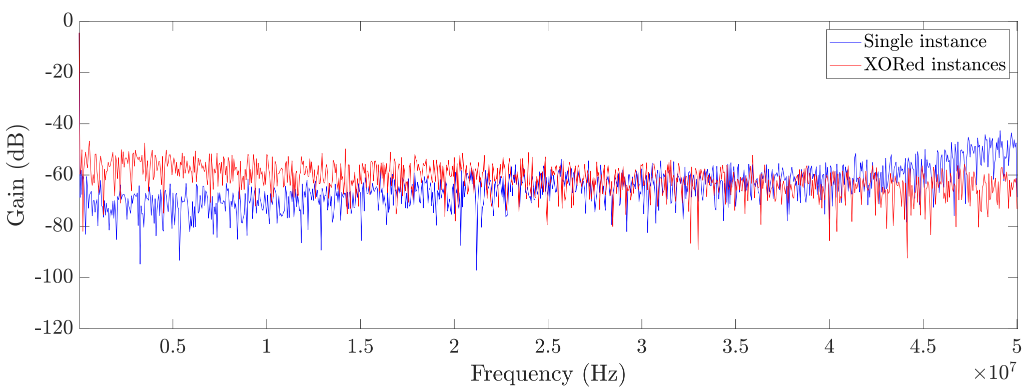

Figure 4.5: Spectrum of a single instance of MOD1 CTDSM output $v[n]$ showing frequency-dependent behavior, whereas the spectrum of sampled data from four XORed instances appears white.

The spectrum of one million sampled bits at the rate of 50 MHz from a single instance of MOD1 SC-CTDSM output appeared high-pass shaped, as expected. This is also evident from the noise analysis we did for MOD1 CTDSM in the section, where shaped quantization noise dominated the overall noise budget of the modulator despite injecting the maximum possible thermal noise and phase noise into the loop. After XORing four IID outputs, the resultant spectrum tended to become "whiter". This gave us the confidence to do further testing on one million-bit samples of $(v_1[n] \oplus v_2[n] \oplus v_3[n] \oplus v_4[n])$. Although the XORed spectrum appeared white, there might be some correlation among the samples buried under small tones in the spectrum. Figure 4.6 shows the auto-correlation function of the data, revealing spikes at non-zero lags, indicating strong positive correlations in the subsequent bits.

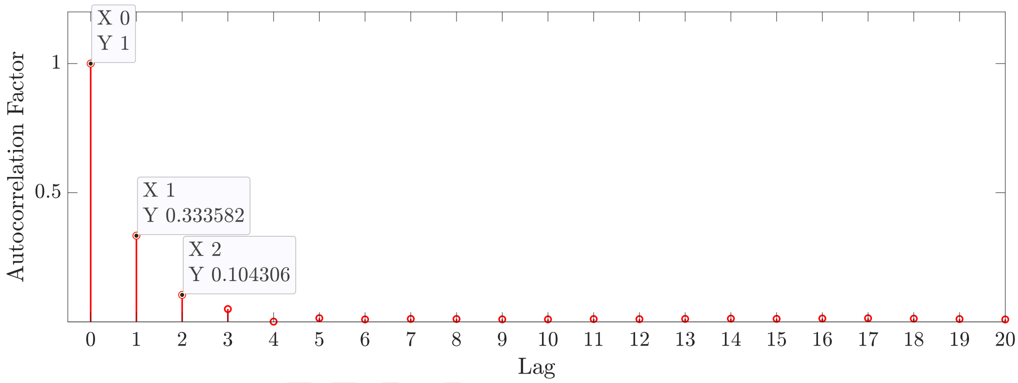

Figure 4.6: Auto correlation function plotted for sampled data after doing XOR operation on four instances of MOD1 CTDSM output $v[n]$, showing correlation.

We also performed NIST 800-22 randomness tests on one million bit samples generated through $(v_1[n] \oplus v_2[n] \oplus v_3[n] \oplus v_4[n])$ for which the results are summarized in Table 4.1. It is evident that the generated bitstream failed nine out of fifteen tests, making it unsuitable for cryptographic applications. So, our first attempt to generate truly random data sampled through MOD1 SC-CTDSM was not successful.

A single-bit Delta-Sigma modulator can function as a higher-resolution ADC for a limited bandwidth input signal. By adjusting the DC gain of the loop filter, we can fine-tune the SQNR around the input signal bandwidth. Further filtering out this signal from the shaped spectrum results in a higher Effective Number of Bits (ENOB), providing multi-bit resolution even from a less precise quantizer. However, using a single-bit quantizer in a delta-sigma modulator for TRNG design poses challenges. The resolution of a single-bit quantizer (only two levels) is too coarse to effectively convert wideband input-referred noise into truly random bits. Moreover,

Serial No.	Test Name	Result
1.	Frequency monobit	Pass
2.	Block frequency	Pass
3.	Runs test	Fail
4.	Longest run of ones	Fail
5.	Binary matrix rank	Pass
6.	DFT	Fail
7.	Non-overlapping	Pass
8.	Overlap matching	Fail
9.	Maurers universal	Fail
10.	Linear complexity	Pass
11.	Serial test	Fail
12.	Approximate entropy	Fail
13.	Cumulative sums	Pass
14.	Random excursions	Fail
15.	Random excursions variant	Fail

Table 4.1: The result of the NIST 800-22 randomness tests conducted on one million bit samples obtained from four instances of MOD1 CTDSM output ($v_k[n],\ k = 1,\ 2,\ 3,\ 4$), and post-processed through the XOR corrector by doing ($v_1[n] \oplus v_2[n] \oplus v_3[n] \oplus v_4[n]$).

resolving zero-mean noise with just two levels is sensitive to systematic bias and also introduces periodicity (patterns), leading to idle tones in the spectrum responsible for the correlation, as observed in Figure 4.6.

Shifting to a multi-bit quantizer in the delta-sigma modulator seems beneficial in overcoming these challenges. A multi-level quantizer, with its finer resolution, aids in eliminating tones and does not mask small noise fluctuations. It also enables other uncorrelated noise sources, such as thermal noise from the resistor at the input, clock jitter, and random delays taken by the comparator to resolve, to make more substantial contributions to the output spectrum by reducing quantization noise variance, as evident from the Table 3.2. Additionally, sampling through the LSB provides an extra layer of filtering, mitigating the impact of circuit non-idealities and further improving the quality of the generated random bitstream.

4.2 Multi-bit CTDSM based TRNG

To test this hypothesis, we designed a 1st order, 4-bit CTDSM using ideal Verilog-A models for a 4-bit ADC and an NRZ-DAC. The integrator in the loop filter was implemented using the same OTA-RC structure with a unity gain bandwidth of 100 MHz. Using an ideal buffer, we injected thermal noise at the input of the modulator generated from a 1 $G\Omega$ resistor. The entire circuit operated with a clock frequency of 100 MHz for both clk_{adc} and clk_{dac} generated from a noisy VCO with an RMS timing error ($\sigma_{\Delta t}$) of approximately 2 ns.

We did the transient noise simulation of 4-bit CTDSM with all the noise sources exciting the loop at once. By plotting the continuous-time transient signal at the output of the loop filter, we observed that the signal excursion stayed within the ± 150 mv range, centered around the circuit's common mode voltage of 800 mv. However, despite having a full-scale dynamic range of 1.2 volts, the quantizer (ADC + DAC) effectively perceives a random process with $6\sigma = 300mv$. This leads to poor resolution of the delta-sigma modulator which results in the masking of a significant amount of noise variance from different sources due to the coarse conversion by the quantizer.

To resolve this, we modified the Verilog-A models for a 4-bit ADC and an NRZ-DAC to have a full-scale dynamic range well within $6\sigma = 300mv$. By doing that, we ensured the loop's operation with better granularity. Now that the loop has been enriched with high-entropy noise, one million bits were sampled from the LSB output at a rate of 100 MHz from 4 separate instances of 4-bit CTDSM. Figure 4.7 shows the spectrum of $v_{LSB}[n]$ and four instances of $v_{LSB}[n]$ XORed together.

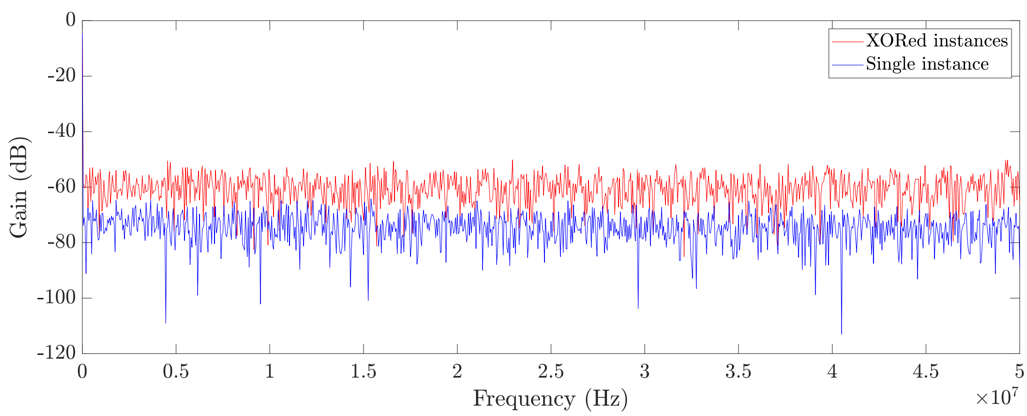

Figure 4.7: Spectrum of a single instance of 4 Bit CTDSM LSB output (blue) along with its four XORed instances (red).

We can observe that the spectrum appears white, while the spectrum of binary random data obtained from $v_{LSB1} \oplus v_{LSB2} \oplus v_{LSB3} \oplus v_{LSB4}$ has even higher variance and fewer tones. The next step was to check for any correlation in the subsequent samples of the binary random data. We plotted the auto-correlation factors at different lags, taking one million-bit samples of the generated data. In Figure 4.8, we can see an ACF equal to 1 at $Lag = 0$ and within ± 0.002 with a 95% confidence bound for all non-zero Lag values. This justifies that samples of $v_{LSB1} \oplus v_{LSB2} \oplus v_{LSB3} \oplus v_{LSB4}$ show no observable correlation so far.

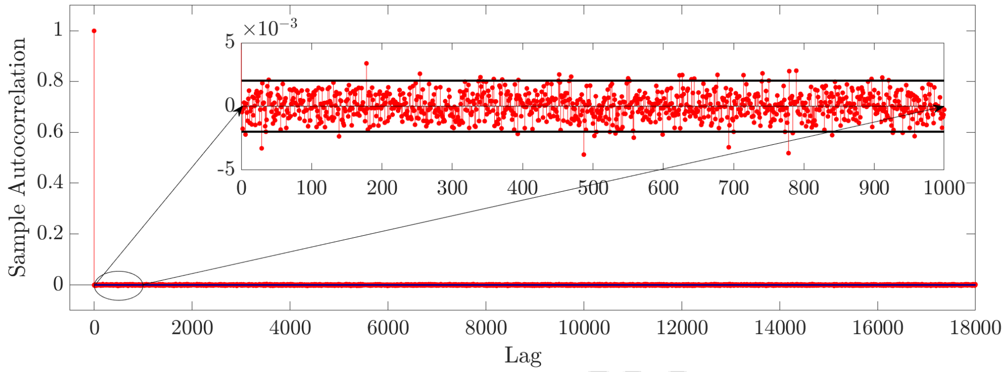

Figure 4.8: Unity auto-correlation at zero lag and ACF bounded to zero at all other Lag for $v_{LSB1} \oplus v_{LSB2} \oplus v_{LSB3} \oplus v_{LSB4}$.

For the final validation, NIST 800-22 randomness tests were performed on one million-bit samples of binary random data generated through XORing four IID instances of a 4-bit CTDSM. The generated bitstream consistently passed 12 to 13 tests out of a total of 15 tests, and the results are summarized in Table 4.2.

4.2.1 Design of 4-bit CTDSM

Building on the promising results obtained from our Verilog-A model simulations of a 4-bit CTDSM for TRNG, we are now taking a practical leap forward. To further enhance the practicality and applicability of our findings, we intend to transition from idealized Verilog-A models to a fully-fledged CMOS-level implementation of the 4-bit CTDSM. This shift will involve replacing all idealized elements and Verilog-A models with a comprehensive CMOS design, allowing us to validate our insights in a more realistic and hardware-oriented context. Before we discuss circuit implementation, it is worthwhile to note some important aspects of the ideal model simulations.

Serial No.	Test Name	Result
1.	Frequency monobit	Pass
2.	Block frequency	Pass
3.	Runs test	Fail
4.	Longest run of ones	Pass
5.	Binary matrix rank	Pass
6.	DFT	Pass
7.	Non-overlapping	Pass
8.	Overlap matching	Pass
9.	Maurers universal	Pass
10.	Linear complexity	Pass
11.	Serial test	Pass
12.	Approximate entropy	Pass
13.	Cumulative sums	Pass
14.	Random excursions	Fail
15.	Random excursions variant	Fail

Table 4.2: The result of the NIST 800-22 randomness tests conducted on one million bit samples obtained from $v_{LSB1} \oplus v_{LSB2} \oplus v_{LSB3} \oplus v_{LSB4}$

1. We employed an ideal VCVS buffer at the modulator's input to inject thermal noise, simulated with impractical resistor values (in the range of Gigaohms). While an ideal buffer possesses infinite input impedance, zero output impedance, and no delay, a real-world buffer, implemented through an operational amplifier, introduces bandwidth constraints and finite output impedance, which loads the subsequent stage.

2. Additionally, generating clock signals (clk_{adc} and clk_{dac}) with timing errors around 20% of the clock period is essential for a significant noise contribution. We used the ideal Verilog-A model of VCO injected with thermal noise to generate a high-jittered clock.

3. We utilized a 4-bit Verilog-A model for ADC and DAC, adjusting the dynamic range based on the maximum signal excursion encountered by the quantizer in the loop. Replicating all these behaviors using TSMC 65nm CMOS poses multiple design challenges for the implementation of a fully functional, hardware-based True Random Number Generator.

Figure 4.9 shows a 1st order, 4-bit CTDSM implemented using TSMC 65nm technology with a supply voltage of 1.2 volts. Stochasticity has been incorporated into the loop by introducing noise through various means. Here, thermal noise is generated by $R_{1,2}$, and its noise power is amplified using a high-gain fully-differential single-stage OTA. We used the OTA-RC topology for designing the 1st order loop filter. CRO have been designed to generate clk_{adc} and clk_{dac} with amplified jitter. A 4-bit Flash ADC and a Switched Capacitor RZ DAC have been designed to implement the quantizer. The two-phase operation of the SCRZ DAC ensures poor alias rejection, as discussed earlier. A Non-Overlapping Clock Generator (NOCG) generates ϕ_1 and ϕ_2 to clock the two-phase operation of the Switched Capacitor RZ DAC. T2B is a Thermometer Binary encoder used to convert from thermometer codes $(T_0, \ldots, T_{14})$ to binary codes $(D_0, \ldots, D_3)$. The whole circuit is being operated around the DC voltage reference $V_{DC} = 800$ mV. The LSB output, D_0 from 4-bit Flash ADC, is sampled to generate raw random binary data, which subsequently undergoes post-processing. We will go through the design details of each stage of 1st order, 4-bit CTDSM for TRNG application.

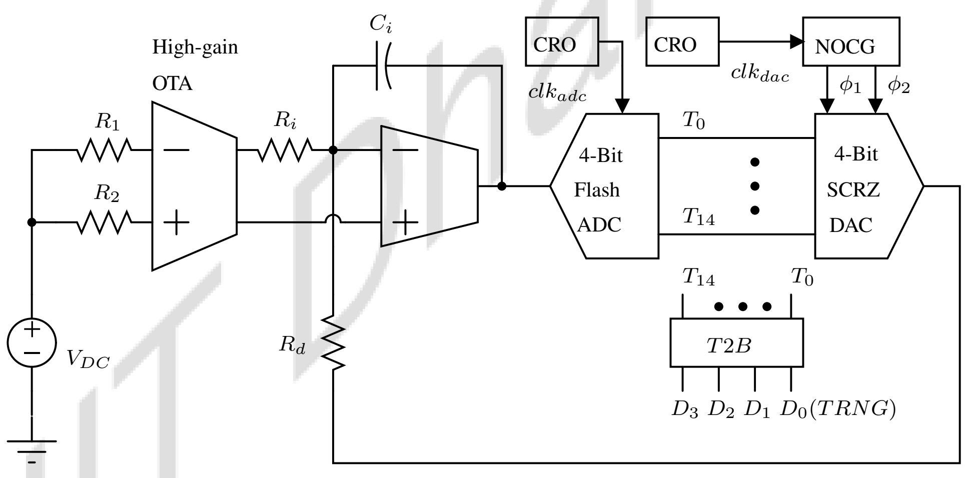

Figure 4.9: 1st order, 4-bit CTDSM implemented using TSMC 65nm technology in which thermal noise generated by $R_{1,2}$ used as a reference input whereas Chaotic Ring oscillators are used to clock 4-bit ADC and DAC.

We used a High-gain Operational Transconductance Amplifier (OTA), which calibrates a negative conductance load for impedance cancellation to achieve 54 dB gain and unity gain-bandwidth of 5.4 GHz [29]. This OTA was used in an open-loop configuration, where thermal noise from R_1 and R_2 served as inputs at a DC bias voltage of V_{DC} = 800 mV. The input differential thermal noise exhibited a standard deviation (σ_{in}) of 1.2 mV, which underwent a 26 dB amplification in noise power. The standard deviation, (σ_{out}) of resulting differential thermal noise at the output was measured to be approximately 24.5 mv (Refer Figure 4.10).

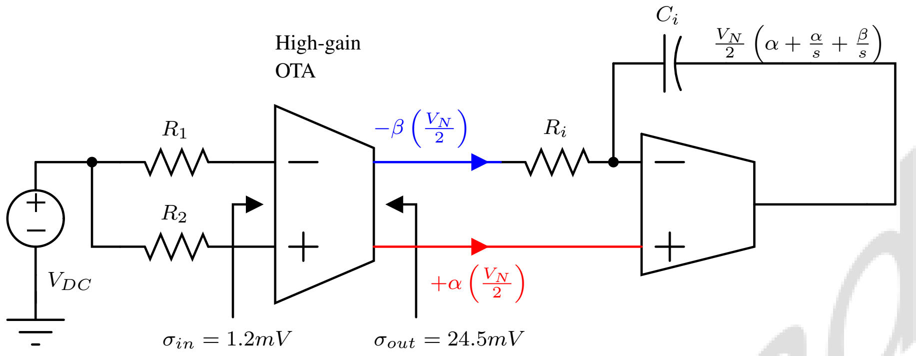

Figure 4.10: High-gain OTA used in an open-loop configuration boosts thermal noise generated through $R1$ and $R2$, which gets buffered at CTDSM input.

The differential thermal noise was directly fed to the positive and negative terminals of the OTA-RC-based integrator in the next stage, as shown in Figure 4.10. The output common mode voltage from the previous stage biases both the terminal of the OTA used in the next stage. As a result of this setup, the differential output of the high-gain OTA becomes imbalanced because each terminal experiences a different load impedance. So, instead of getting a fully differential thermal noise signal at the High-gain OTA's output, say $(\pm\frac{V_N}{2})$, we get $(+\alpha \cdot \frac{V_N}{2})$ and $(-\beta \cdot \frac{V_N}{2})$. Further, the time constant of the integrator $(\tau_{integrator})$ increases due to the high output impedance of the high-gain OTA. Let's consider having $(+\alpha \cdot \frac{V_N}{2})$ at the positive terminal and $(-\beta \cdot \frac{V_N}{2})$ at the negative terminal of the OTA-RC integrator. Due to the different transfer functions each noise signal encounters, $(+\alpha \cdot \frac{V_N}{2})$ directly appears at the output of the integrator, alongside its integrated version. In contrast, $(-\beta \cdot \frac{V_N}{2})$ is directly integrated but with an inverted gain. Thus, effectively, we have the same delta-sigma negative feedback operation within the loop with the reference input as $(+\alpha \cdot \frac{V_N}{2} - \beta \cdot \frac{V_N}{2})$, along with additional thermal noise $(+\alpha \cdot \frac{V_N}{2})$ added at input of the ADC. The reduction in the integrator's time constant constrains the bandwidth of noise within the loop. When this noise is sampled through clk_{adc} with $T_{sampling} < \tau_{integrator}$, it can lead to longer-than-expected strings of 1's or 0's in the raw binary data. However, the direct entry of high-frequency thermal noise $(+\alpha \cdot \frac{V_N}{2})$ into the loop partially compensates for this effect. Additionally, since we generate raw binary random data from multiple instances, which are independent and identically distributed, the impact of this non-ideality diminishes further after XORing them.

Figure 4.11 shows the quantizer implementation using 4-bit Flash ADC and Switched capacitor RZ DAC. The comparators for the ADC are implemented using the cascade of the StrongArm latch and symmetric RS latch. Unit DAC element was designed using a switched capacitor resistor with a Transmission gate as the switch. Transient simulations revealed that the maximum signal excursion of the random process $y(t)$, $6\sigma_y$, did not cover the full-scale dynamic range of the quantizer (ADC + DAC). This could result in the masking of a significant amount of noise variance due to the coarse conversion by the quantizer, which effectively degrades entropy within the loop and could lead to idle patterns/tones.

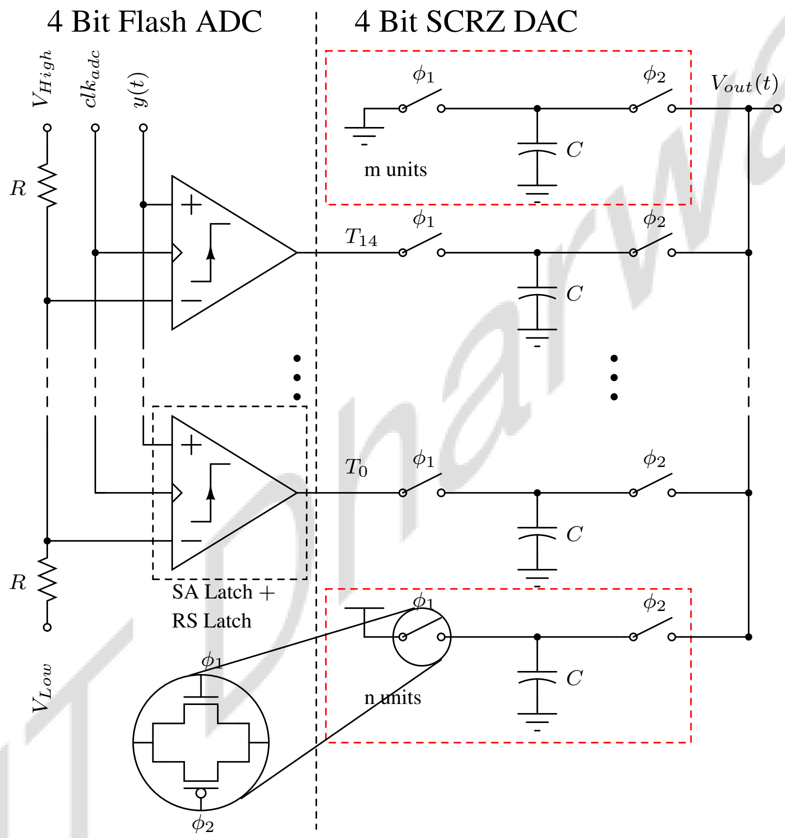

Figure 4.11: Cascade of 4 Bit Flash ADC with 4 Bit DAC based on switched-capacitor resistors used in CTDSM. The input dynamic range of ADC is controlled by V_{High} and V_{Low}, whereas DAC's output dynamic range is controlled by adjusting the number of always ON and OFF (m,n) unit elements.

To address this issue, the ADC and DAC's dynamic range was calibrated to handle the measured full-scale swing of the noise input with an additional headroom to prevent quantizer overloading. The biasing for V_{High} and V_{Low} in the Flash ADC was set based on the value $V_{CM} \pm 3\sigma_y$ where V_{CM} indicated DC reference voltage of the loop. Meanwhile, control over the dynamic range in the SCRZ DAC was attained by using additional always ON (m units) and always OFF (n units) DAC elements. This configuration also aided in managing the offset of the quantizer, which could be mitigated by adjusting the number of always ON (OFF) DAC elements.

Figure 4.12 shows a chaotic ring oscillator circuit used to generate clock signals for clk_{adc} and clk_{dac}. It is based on the design proposed in [30] and fine-tuned to meet the specific requirements of our design. We measured the clock frequencies at V_{RO1} and V_{RO2} to be 98.4 MHz and 100.6 MHz, respectively, with an RMS jitter-to-period ratio of approximately 20%. The reported circuit boosts jitter using the chaotic dynamics generated by the non-linear coupling of two-ring oscillators. The design features two identical seven-stage ring oscillators, diode-connected transistors, a capacitor (C) for oscillation frequency control, and two resistors (R1 & R2) to control the oscillation amplitude. The non-linearity of this circuit comes from the diode-connected transistors configured in an anti-parallel connection.

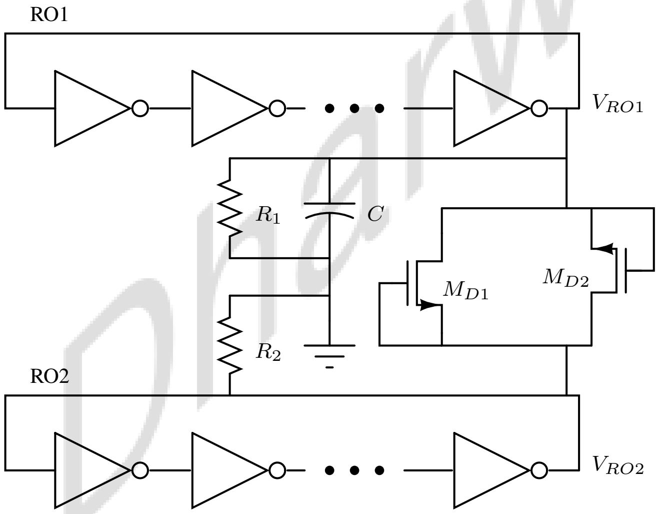

Figure 4.12: Chaotic ring oscillator used to amplify the jitter in clk_{adc}, clk_{dac} by non-linear coupling of two diode-connected equal-length ring oscillators.

A phase portrait helps us understand how a system evolves over time. The phase portrait of a deterministic system reveals regular and predictable patterns, with the system following specific trajectories in an orderly manner. In contrast, the phase portrait of a chaotic system displays irregular and unpredictable behavior. The trajectories in a chaotic system are highly sensitive to initial conditions. Small changes in the starting state can lead to vastly different outcomes, resulting in complex and random patterns in the phase portrait. We plotted V_{RO1} against V_{RO2} as annotated in Figure 4.12 to see the evolution of the designed chaotic ring oscillator. The phase portrait of the chaotic ring oscillator is depicted in Figure 4.13. This portrait reveals the chaotic behavior of the system, showcasing no observable fixed pattern and unpredictable trajectories, which manifests itself in the form of jitter in the resulting clocks.

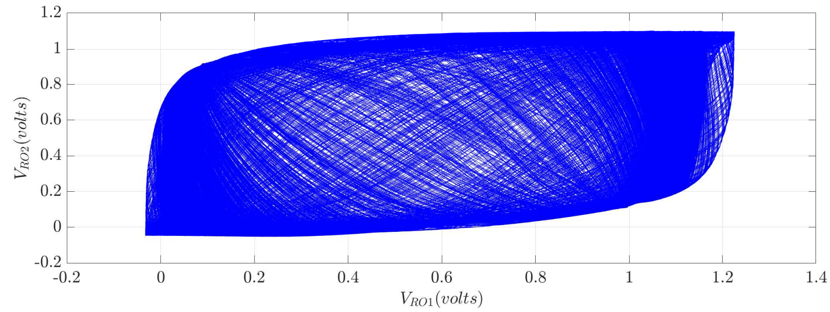

Figure 4.13: Phase portrait obtained by plotting V_{RO1} against V_{RO2} showing the chaotic behavior of the system, with no observable fixed pattern and unpredictable trajectories.

4.2.2 Testing 4-bit CTDSM based TRNG

We did a transient simulation of 4 separate instances of our 4-bit CTDSM design shown in Figure 4.9 excited with all the noise sources simultaneously. The LSB output of the 4-bit ADC represented as D_0, was sampled at a frequency of 100.6 MHz. This process led to the generation of four IID binary random bitstreams, namely $D_{0,x=1,2,3,4}$, each consisting of one million samples. Let's denote the XORed outcomes of these bitstreams as $D_{XOR,4}$, obtained by performing $D_{0,1} \oplus D_{0,2} \oplus D_{0,3} \oplus D_{0,4}$. Similarly, $D_{XOR,3}$ and $D_{XOR,2}$ have their respective meanings. Figure 4.14 displays the FFTs of both $D_{0,x}$ and $D_{XOR,3}$. Both spectra appear flat with no visible frequency dependence and overlap each other, demonstrating similar strength across the entire bandwidth. Additionally, $D_{XOR,3}$ appears to have fewer tones in its spectrum compared to $D_{0,x}$.

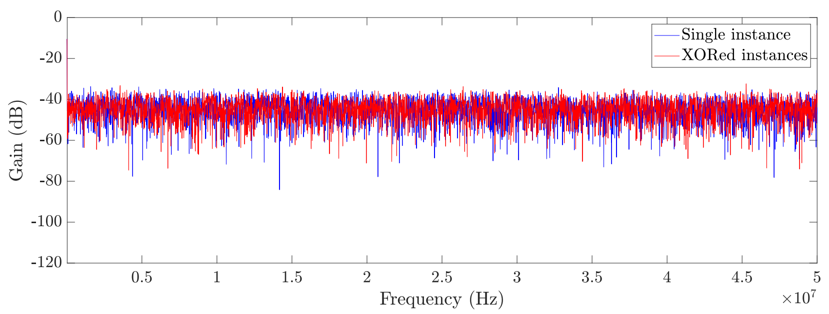

Figure 4.14: FFTs of $D_{0,x}$ and $D_{XOR,3}$ both appears flat with no visible frequency dependence.

We plotted the ACF for $D_{0,x=1,2,3}$ and $D_{XOR,3}$ which is shown in Figure 4.15. We can observe the effectiveness of post-processing based on XOR operation on i.i.d binary random data. The autocorrelation factors of $D_{0,x=1,2,3}$ at non-zero lags are reasonably close to zero. However, upon comparing with the ACF of post-processed data $D_{XOR,3}$, we can clearly observe the presence of micro-correlations in the raw, unprocessed data. This emphasizes the effectiveness of XOR-based post-processing in reducing correlations and enhancing randomness in the generated binary data.

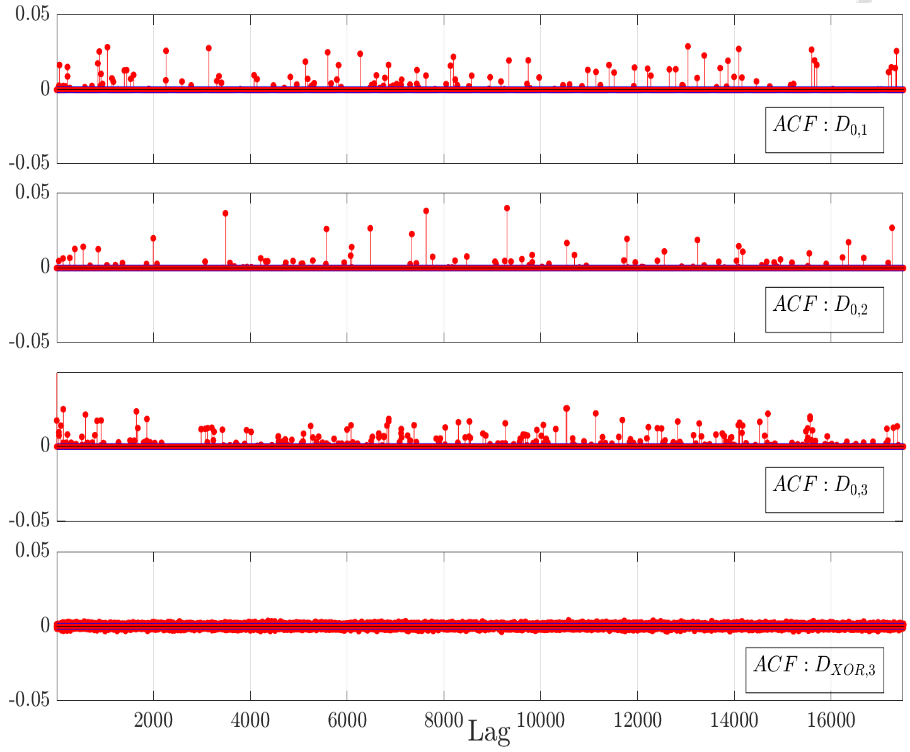

Figure 4.15: Autocorrelation function plotted for $D_{0,x=1,2,3}$ and $D_{XOR,3}$. We can observe the presence of micro-correlations in the raw, unprocessed data, whereas $D_{0,x=1,2,3}$ shows no observable patterns.

For completeness, Figure 4.16 presents a more clearly observable auto-correlation plot for our TRNG output data, $D_{XOR,3}$. Here, we observe an ACF equal to 1 at *Lag=0* and within ± 0.002 with a 95% confidence bound for all non-zero Lag values. This justifies that the samples of $D_{XOR,3}$ exhibit no observable correlation so far. Near-zero autocorrelation at non-zero lags is an essential metric for confirming the randomness of the data, as described in Section 1.1 and shown in Figure 1.2. This outcome partially demonstrates the potential of our approach to produce reliable and secure random numbers for various technological and scientific applications.

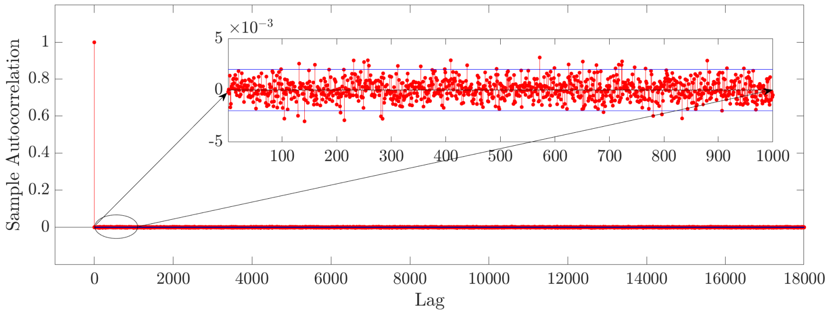

Figure 4.16: Auto-correlation plot for TRNG output data, $D_{XOR,3}$ where ACF equal to 1 at *Lag=0* and within ± 0.002 with a 95% confidence bound for all non-zero Lag values.

After doing spectral analysis and ACF test on the generated samples, the final step involves subjecting the data to the NIST 800-22 randomness test. NIST 800-22 randomness tests were conducted on one million-bit samples of $D_{0,x=1,2,3,4}$, as well as on $D_{XOR,2}$, $D_{XOR,3}$. $D_{0,x=1,2,3,4}$ passed 7-8 tests out of 15 attributed to the micro-correlation revealed in Figure 4.15, whereas all possible combinations of $D_{XOR,2}$ passed around ten to eleven tests. Moving ahead, we examined different combinations of $D_{XOR,3}$, which successfully passed all 15 NIST tests as outlined in the results presented in Table 4.3.

Serial No.	**Test Name**	**Result**
1.	Frequency mono bit	Pass
2.	Block frequency	Pass
3.	Runs test	Pass
4.	Longest run of ones	Pass
5.	Binary matrix rank	Pass
6.	DFT	Pass
7.	Non-overlapping	Pass
8.	Overlap matching	Pass
9.	Maurers universal	Pass
10.	Linear complexity	Pass
11.	Serial test	Pass
12.	Approximate entropy	Pass
13.	Cumulative sums	Pass
14.	Random excursions	Pass
15.	Random excursions variant	Pass

Table 4.3: One million binary random data samples of $D_{XOR,3}$ passing all NIST 800-22 test.

This success validates our initial motivation to generate multiple uncorrelated noise sources, effectively combining their variance to create a cryptography-grade random number generator using the Delta-Sigma modulator.

Chapter 5

Conclusions and Future work

5.1 Conclusions

This thesis proposes a CTDSM-based TRNG that harnesses entropy from multiple noise sources. It showcases utilizing CTDSM to combine entropy from multiple uncorrelated noise sources for TRNG design. The thesis delves into the conceptual development of a CTDSM, and analyzes noise modeling of error induced due to clock jitter and quantizer's metastability within a CTDSM. Based on the insights gained, it offers theoretical justification for the design choices aimed at ensuring a similar amount of noise from each source.

The thesis discussed circuit implementation of CTDSM-based TRNG, exploring some circuit innovations for improving performance. These include generating a weighted combination of RZ and exponentially decaying pulses to simultaneously leverage poor alias rejection and high jitter sensitivity. Additionally, methods for controlling the quantizer's dynamic range based on noise signal excursion are proposed to improve spectral characteristics of quantization noise. An effective method of dithering white noise at the input of the ADC within the CTDSM loop is also suggested and implemented, which resulted in a reduction of tones and enhancement of the overall noise variance.

These design choices are evaluated through extensive simulations, demonstrating their effectiveness. Simulation results of the proposed design, based on 1st-order, 4-bit CTDSM implemented using TSMC 65nm technology, demonstrate the successful generation of cryptographic-grade random sequences at a throughput of 100.6 MHz.

5.2 Future work

The proposed TRNG, utilizing a 1st-order, 4-bit (CTDSM), passes all NIST tests at a supply voltage of 1.2 V and nominal room temperature. Future research directions to enhance performance and ensure reliability in practical scenarios include:

- **Post-layout verification:** While the proposed TRNG has been verified at the schematic level, it's crucial to validate its performance in silicon through post-layout simulations. These simulations assess the design's functionality, ensuring its effectiveness and reliability in practical applications.

- **Verification across PVT:** It is important to conduct thorough verification across Process, Voltage, and Temperature (PVT) induced device mismatch to ensure the robustness of the TRNG's random behavior.

- **Design optimization for power:** Optimize the TRNG design to minimize power consumption while maintaining performance.

- **Environmental impact analysis:** TRNG can become vulnerable to side-channel interferences, which can compromise its security. Extensive testing is needed to investigate the impact of environmental factors such as temperature variations, electromagnetic interferences, and power supply fluctuations on the entropy of the TRNG, ensuring its resilience to external influences.

- **Post-Processing enhancement:** Explore and implement more efficient post-processing techniques to enhance the quality of generated random numbers. This may involve an extensive survey to identify methods that mitigate area and energy penalties, thus improving throughput without compromising security.

These future works aim to address various aspects of TRNG based on CTDSM, ensuring its reliability, efficiency, and resilience to external influences.

Appendix A

Simulation Setup and Associated Challenges

Analysis of TRNG circuits requires long simulations to generate 1 million bits, which is the minimum required for some of the NIST tests. The performance of the computer used (along with the settings and simulation methods) for the simulations determines the simulation time and accuracy. Following are the specifications of the computer used for this work and the specific simulator settings are discussed thereafter.

- **System configuration:**

  1. Processor: Intel Core i7 (9th generation)
  2. Graphics Card: Nvidia GeForce GT 730
  3. Operating System: CentOS 7
  4. Memory: 16 GB RAM, 1 TB storage
  5. Simulator: Cadence Virtuoso, using Spectre with Analog Design Environment L (ADE L).

- **Setup:** To validate a TRNG design according to the specific requirements of individual tests in NIST SP 800-22, a minimum of 1 million bit samples must be generated. After capturing the schematic and verifying the functionality of the proposed CTDSM-based TRNG, we conducted Transient Analysis up to 10 ms to obtain the required 1 million bit samples. We imposed a 1 GHz noise bandwidth limitation during the transient analysis setup to speed up the simulation to practically feasible time steps. The 10 ms transient analysis was performed using high-performance simulation settings with APS++ (Accelerated Parallel Simulator) enabled to ensure multithreading. These settings allowed for dynamically large time steps and the use of multiple processor cores for multiple instances, facilitating faster simulation with reasonably high accuracy.

- **Simulation time:** To gather 1 million bit samples for NIST randomness test validation, the proposed CTDSM-based TRNG circuit needed to be simulated for 10 ms using transient analysis. Despite enabling high-performance simulation settings, one complete cycle of simulation used to take approximately 2 weeks. The long simulation time necessitated periodically saving states to ensure the simulation could be resumed from the last saved state in case of any unwanted interruptions.

- **Net simulation data:** While performing transient analysis of the CTDSM-based TRNG in high-performance mode with a noise bandwidth up to 1 GHz, saving data from every node resulted in approximately 600 GB of data for just 650 μs of simulation time. One of the issues encountered was that if we selectively saved data from only the required nodes, Cadence Virtuoso did not allow resuming the simulation from the last saved state and always threw the error: "No VT data available on node xxx."

  This forced us to run the transient analysis with data saving enabled for all nodes to ensure the simulation could be resumed from the last saved state in case of any interruption. Consequently, we had to run the simulation for each 650 μs interval, sample the data, clear the memory, and then resume the simulation from the last saved state. After completing the transient analysis, we concatenated each fragment of sampled data to obtain 1 million bit samples for TRNG validation.

References

[1] L. E. Bassham, A. L. Rukhin, J. Soto, J. R. Nechvatal, M. E. Smid, E. B. Barker, S. D. Leigh, M. Levenson, M. Vangel, D. L. Banks, N. A. Heckert, J. F. Dray, and S. Vo, "Sp 800-22 rev. 1a. a statistical test suite for random and pseudorandom number generators for cryptographic applications," Gaithersburg, MD, USA, Tech. Rep., 2010.

[2] M. Alioto, "Trends in hardware security: From basics to asics," *IEEE Solid-State Circuits Magazine*, vol. 11, no. 3, pp. 56–74, 2019.

[3] L. Gong, J. Zhang, H. Liu, L. Sang, and Y. Wang, "True random number generators using electrical noise," *IEEE Access*, vol. 7, pp. 125 796–125 805, 2019.

[4] V. R. Pamula, X. Sun, S. M. Kim, F. u. Rahman, B. Zhang, and V. S. Sathe, "A 65-nm cmos 3.2-to-86 mb/s 2.58 pj/bit highly digital true-random-number generator with integrated de-correlation and bias correction," *IEEE Solid-State Circuits Letters*, vol. 1, no. 12, pp. 237–240, 2018.

[5] W. Chang, B. Fang, X. Yun, S. Wang, and X. Yu, "Randomness testing of compressed data," *Journal of Computing*, vol. 2, no. 1, pp. 44–55, 2010.

[6] S. Yasuda, H. Satake, T. Tanamoto, R. Ohba, K. Uchida, and S. Fujita, "Physical random number generator based on mos structure after soft breakdown," *IEEE journal of solid-state circuits*, vol. 39, no. 8, pp. 1375–1377, 2004.

[7] F. Crupi, R. Degraeve, G. Groeseneken, T. Nigam, and H. Maes, "On the properties of the gate and substrate current after soft breakdown in ultrathin oxide layers," *IEEE Transactions on Electron Devices*, vol. 45, no. 11, pp. 2329–2334, 1998.

[8] K. Okada and K. Taniguchi, “Electrical stress-induced variable range hopping conduction in ultrathin silicon dioxides,” *Applied Physics Letters*, vol. 70, no. 3, pp. 351–353, 1997. [Online]. Available: https://doi.org/10.1063/1.118411

[9] A. Mohanty, K. B. Sutaria, H. Awano, T. Sato, and Y. Cao, “RTN in scaled transistors for on-chip random seed generation,” *IEEE Transactions on Very Large Scale Integration (VLSI) Systems*, vol. 25, no. 8, pp. 2248–2257, 2017.

[10] B. Sunar, W. J. Martin, and D. R. Stinson, “A provably secure true random number generator with built-in tolerance to active attacks,” *IEEE Transactions on computers*, vol. 56, no. 1, pp. 109–119, 2006.

[11] K. Wold and C. H. Tan, “Analysis and enhancement of random number generator in fpga based on oscillator rings,” in *2008 International Conference on Reconfigurable Computing and FPGAs*. IEEE, 2008, pp. 385–390.

[12] N. Nalla Anandakumar, S. K. Sanadhya, and M. S. Hashmi, “FPGA-based true random number generation using programmable delays in oscillator-rings,” *IEEE Transactions on Circuits and Systems II: Express Briefs*, vol. 67, no. 3, pp. 570–574, 2020.

[13] S. N. Dhanuskodi, A. Vijayakumar, and S. Kundu, “A chaotic ring oscillator-based random number generator,” in *2014 IEEE International Symposium on Hardware-Oriented Security and Trust (HOST)*. IEEE, 2014, pp. 160–165.

[14] A. T. Do and X. Liu, “25 fJ/bit, 5Mb/s, 0.3 V true random number generator with capacitively-coupled chaos system and dual-edge sampling scheme,” in *2017 IEEE Asian Solid-State Circuits Conference (A-SSCC)*. IEEE, 2017, pp. 61–64.

[15] S. K. Mathew, S. Srinivasan, M. A. Anders, H. Kaul, S. K. Hsu, F. Sheikh, A. Agarwal, S. Satpathy, and R. K. Krishnamurthy, “2.4 Gbps, 7 mW all-digital PVT-variation tolerant true random number generator for 45 nm CMOS high-performance microprocessors,” *IEEE Journal of Solid-State Circuits*, vol. 47, no. 11, pp. 2807–2821, 2012.

[16] S.-G. Bae, Y. Kim, Y. Park, and C. Kim, “3-Gb/s high-speed true random number generator using common-mode operating comparator and sampling uncertainty of d flip-flop,” *IEEE Journal of Solid-State Circuits*, vol. 52, no. 2, pp. 605–610, 2017.

[17] A. Jayaraj, N. Nitin Gujarathi, I. Venkatesh, and A. Sanyal, “0.6–1.2 V, 0.22 pJ/bit true random number generator based on SAR ADC,” *IEEE Transactions on Circuits and Systems II: Express Briefs*, vol. 67, no. 10, pp. 1765–1769, 2020.

[18] S. T. Chandrasekaran, V. E. G. Karnam, and A. Sanyal, “0.36-mW, 52-Mbps true random number generator based on a stochastic delta–sigma modulator,” *IEEE Solid-State Circuits Letters*, vol. 3, pp. 190–193, 2020.

[19] S. Kwok, Y. Ee, G. Chew, K. Zheng, K. Khoo, and C. Tan, “A comparison of post-processing techniques for biased random number generators,” in *Information Security Theory and Practice. Security and Privacy of Mobile Devices in Wireless Communication. WISTP 2011*, ser. Lecture Notes in Computer Science, C. Ardagna and J. Zhou, Eds., vol. 6633. Berlin, Heidelberg: Springer, 2011.

[20] Z. Kukorelly, “The piling-up approximation in linear cryptanalysis,” *IEEE Transactions on Information Theory*, vol. 47, no. 7, pp. 2812–2823, 2001.

[21] S. Pavan, R. Schreier, and G. C. Temes, *Understanding Delta-Sigma Data Converters*, second edition ed. Wiley-IEEE Press, 2017.

[22] P. Aziz, H. Sorensen, and J. vn der Spiegel, “An overview of sigma-delta converters,” *IEEE Signal Processing Magazine*, vol. 13, no. 1, pp. 61–84, 1996.

[23] S. Pavan and R. S. Rajan, *IEEE Transactions on Circuits and Systems II: Express Briefs, title=Simplified Analysis and Simulation of the STF, NTF, and Noise in Continuous-Time $\Delta\Sigma$ Modulators*, vol. 61, no. 9, pp. 681–685, 2014.

[24] J. Cherry and W. Snelgrove, “Clock jitter and quantizer metastability in continuous-time delta-sigma modulators,” *IEEE Transactions on Circuits and Systems II: Analog and Digital Signal Processing*, vol. 46, no. 6, pp. 661–676, 1999.

[25] S. Pavan, “Alias Rejection of Continuous-Time $\Delta\Sigma$ Modulators With Switched-Capacitor Feedback DACs,” *IEEE Transactions on Circuits and Systems I: Regular Papers*, vol. 58, no. 2, pp. 233–243, 2011.

[26] B. Razavi, “The strongarm latch [a circuit for all seasons],” *IEEE Solid-State Circuits Magazine*, vol. 7, no. 2, pp. 12–17, 2015.

[27] A. Abidi and H. Xu, “Understanding the regenerative comparator circuit,” in *Proceedings of the IEEE 2014 Custom Integrated Circuits Conference*, 2014, pp. 1–8.

[28] S. Pavan and R. S. Rajan, “Interreciprocity in linear periodically time-varying networks with sampled outputs,” *IEEE Transactions on Circuits and Systems II: Express Briefs*, vol. 61, no. 9, pp. 686–690, 2014.

[29] M. S.M and N. K., “Mismatch Tolerant Negative Conductance Load Tuning for High Gain OTAs,” in *2023 IEEE Asia Pacific Conference on Circuits and Systems (APCCAS)*, 2024, p. to appear.

[30] Çiçek and G. Dündar, “A chaos based integrated jitter booster circuit for true random number generators,” in *2013 European Conference on Circuit Theory and Design (ECCTD)*, 2013, pp. 1–4.

List of Publications

[1] Apurv Pandey, and Naveen Kadayinti "TRNG Based on Multiple Entropy Sources Using CTDSM", **under review** *in IEEE Asia Pacific Conference On Circuits and Systems 2024 (APCCAS 2024)*

Acknowledgments

I am deeply grateful to my supervisor, Prof. Naveen Kadayinti, for his guidance, encouragement, and patience at every stage of my work. His mentorship and expertise have been invaluable assets throughout this journey. I am thankful and honored to have had the opportunity to work with him. I sincerely thank my Research Progress Committee members, Prof. Naveen M B, Prof. Nagaveni S, and Prof. R S Ashwin Kumar from IIT Kanpur. Their valuable feedback and advice have been integral to my research and academic progress.

I also want to express gratitude to my grandparents, parents, and brother for their constant love and support. Their encouragement has been a source of strength for me. I am also thankful to my dear friends, Aakarshya and Priya, for always being there for me. I feel truly blessed to have such a caring family and friends by my side.

I would also like to thank my labmates and colleagues for their moral support and encouragement throughout my research work. Finally, I thank my friends from Hostel Asavari and Saveri for making this journey beautiful and memorable.

Apurv Pandey